

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии Воронкин Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Изучение архитектуры и принципов работы рекуррентных и одномерных свёрточных нейронных сетей для задачи классификации художественных текстов по авторам.

Цель работы: освоение теоретических основ и приобретение практических навыков построения рекуррентных (RNN, GRU, LSTM) и одномерных свёрточных (Conv1D) нейронных сетей для обработки текстовых данных, исследование влияния гиперпараметров (размер словаря, размер окна, шаг окна) на качество классификации.

Репозиторий: https://github.com/bickrosss/NN_lab5

Порядок выполнения работы:

Задание 1 Решение задачи классификации текстов шести писателей (Булгаков, Брэдбери, Стругацкие, Макс Фрай, Клиффорд Саймак, О. Генри) с помощью одномерной свёрточной нейронной сети Conv1D, которая показала лучший результат на практическом занятии. Проводится серия экспериментов по изучению влияния размера словаря (VOCAB_SIZE) и параметров скользящего окна (WIN_SIZE, WIN_HOP) на итоговое качество распознавания.

1.1 Реализация средствами библиотеки Keras (TensorFlow).

1.1.1 Подключение необходимых для работы библиотек.

Импортированы все необходимые модули: NumPy для работы с массивами, Keras-утилиты для категориальных данных и построения модели Sequential, базовые слои (Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding, Flatten, Activation), а также специализированные слои для обработки последовательностей: SimpleRNN, GRU, LSTM, Bidirectional, Conv1D, MaxPooling1D, GlobalMaxPooling1D. Для предобработки текстов подключён Tokenizer, для оценки качества – функция confusion_matrix и класс ConfusionMatrixDisplay из scikit-learn. Также подключены вспомогательные модули gdown (загрузка датасета из облака), os, time, re и matplotlib.

```

# Работа с массивами данных
import numpy as np

# Функции-утилиты для работы с категориальными данными
from tensorflow.keras import utils

# Класс для конструирования последовательной модели нейронной сети
from tensorflow.keras.models import Sequential

# Основные слои
from tensorflow.keras.layers import Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding, Flatten, Activation
from tensorflow.keras.layers import SimpleRNN, GRU, LSTM, Bidirectional, Conv1D, MaxPooling1D, GlobalMaxPooling1D

# Токенизатор для преобразования текстов в последовательности
from tensorflow.keras.preprocessing.text import Tokenizer

# Рисование схемы модели
from tensorflow.keras.utils import plot_model

# Матрица ошибок классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Загрузка датасетов из облака google
import gdown

# Функции операционной системы
import os

# Работа со временем
import time

# Регулярные выражения
import re

# Отрисовка графиков
import matplotlib.pyplot as plt

# Вывод объектов в ячейке colab
from IPython.display import display

%matplotlib inline

```

Рисунок 1.1.1 – Подключение библиотек

1.1.2 Загрузка датасета.

Датасет с текстами шести писателей загружается из облачного хранилища с помощью библиотеки `gdown` в виде ZIP-архива и распаковывается в локальную папку `writers`. Также определяются константы для дальнейшей загрузки данных: имя папки с файлами и признаки обучающей/тестовой выборки в имени файла («обучающая» / «тестовая»).

```

# Загрузим датасет из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/17/writers.zip', None, quiet=True)

# Папка с текстовыми файлами
FILE_DIR = 'writers'

# Признаком обучающей выборки в имени файла
SIG_TRAIN = 'обучающая'

# Признаком тестовой выборки в имени файла
SIG_TEST = 'тестовая'

# Распакуем архив в папку writers
!unzip -qo writers.zip -d writers/

```

Рисунок 1.1.2 – Загрузка и распаковка датасета

1.1.3 Формирование классов и выборок.

Для каждого файла из папки `writers` с помощью регулярного выражения извлекается имя автора и признак выборки (обучающая или тестовая). Если

автор встречается впервые, он добавляется в общий список классов CLASS_LIST, а соответствующие пустые строки – в text_train и text_test. Затем содержимое файла дописывается в строку нужного класса и нужной выборки. В результате получаются два списка строк: text_train и text_test, в каждом из которых индекс соответствует номеру класса.

```

# Подготовим пустые списки
CLASS_LIST = [] # Список классов
text_train = [] # Список для обучающей выборки
text_test = [] # Список для тестовой выборки

# Получим список файлов в папке
file_list = os.listdir(FILE_DIR)

for file_name in file_list:
    # Выделим имя класса и тип выборки из имени файла
    m = re.match('\(((.+)\) (\S+)_', file_name)
    # Если выделение получилось, то файл обрабатываем
    if m:
        class_name = m[1]
        subset_name = m[2].lower()

        is_train = SIG_TRAIN in subset_name
        is_test = SIG_TEST in subset_name

        if is_train or is_test:
            # Добавим новый класс, если его еще нет в списке
            if class_name not in CLASS_LIST:
                print(f'Добавление класса "{class_name}"')
                CLASS_LIST.append(class_name)
                text_train.append('')
                text_test.append('')

            cls = CLASS_LIST.index(class_name)
            print(f'Добавление файла "{file_name}" в класс "{CLASS_LIST[cls]}"', {subset_name} выборка.')

            with open(f'{FILE_DIR}/{file_name}', 'r') as f:
                text = f.read()
                subset = text_train if is_train else text_test
                subset[cls] += ' ' + text.replace('\n', ' ')

*** Добавление класса "Макс Фрай"
Добавление файла "(Макс Фрай) Тестовая_2 вместе.txt" в класс "Макс Фрай", тестовая выборка.
Добавление класса "Рэй Брэдбери"
Добавление файла "(Рэй Брэдбери) Обучающая_22 вместе.txt" в класс "Рэй Брэдбери", обучающая выборка.
Добавление класса "Клиффорд Саймак"
Добавление файла "(Клиффорд Саймак) Обучающая_5 вместе.txt" в класс "Клиффорд Саймак", обучающая выборка.
Добавление класса "О. Генри"
Добавление файла "(О. Генри) Обучающая_50 вместе.txt" в класс "О. Генри", обучающая выборка.
Добавление класса "Булгаков"
Добавление файла "(Булгаков) Обучающая_5 вместе.txt" в класс "Булгаков", обучающая выборка.
Добавление файла "(О. Генри) Тестовая_20 вместе.txt" в класс "О. Генри", тестовая выборка.
Добавление файла "(Булгаков) Тестовая_2 вместе.txt" в класс "Булгаков", тестовая выборка.
Добавление файла "(Рэй Брэдбери) Тестовая_8 вместе.txt" в класс "Рэй Брэдбери", тестовая выборка.
Добавление файла "(Макс Фрай) Обучающая_5 вместе.txt" в класс "Макс Фрай", обучающая выборка.
Добавление класса "Стругацкие"
Добавление файла "(Стругацкие) Тестовая_2 вместе.txt" в класс "Стругацкие", тестовая выборка.
Добавление файла "(Стругацкие) Обучающая_5 вместе.txt" в класс "Стругацкие", обучающая выборка.
Добавление файла "(Клиффорд Саймак) Тестовая_2 вместе.txt" в класс "Клиффорд Саймак", тестовая выборка.
<:11: SyntaxWarning: invalid escape sequence '\('
<:11: SyntaxWarning: invalid escape sequence '\('
/tmp/ipykernel_637/2666206858.py:11: SyntaxWarning: invalid escape sequence '\('
m = re.match('\(((.+)\) (\S+)_', file_name)

```

Рисунок 1.1.3 – Результат формирования классов и выборок

В результате выявлено 6 классов писателей: Макс Фрай, Рэй Брэдбери, Клиффорд_Саймак, О. Генри, Булгаков, Стругацкие. Общее количество классов выводится для контроля корректности обработки.

```
1) # Определим количество классов
   CLASS_COUNT = len(CLASS_LIST)
   print('Классы:', CLASS_LIST)
   print('Количество классов:', CLASS_COUNT)

... Классы: ['Макс Фрай', 'Рэй Брэдберри', 'Клиффорд Саймак', 'О. Генри', 'Булгаков', 'Стругацкие']
   Количество классов: 6

1) # Контекстный менеджер для измерения времени операций
   class timex:
       def __enter__(self):
           self.t = time.time()
           return self

       def __exit__(self, type, value, traceback):
           print('Время обработки: {:.2f} c'.format(time.time() - self.t))
```

Рисунок 1.1.4 – Список классов и их количество

1.1.4 Универсальные функции для подготовки и обучения.

Чтобы запускать эксперименты с разными значениями VOCAB_SIZE, WIN_SIZE и WIN_HOP, вся подготовка данных и создание нейросети обёрнуты в функции. Это позволяет переиспользовать код и единообразно обрабатывать данные во всех экспериментах.

Листинг 1.1.1 – Универсальные функции подготовки данных

```
# Функция разбиения последовательности на отрезки скользящим окном
def split_sequence(sequence, win_size, hop):
    return [sequence[i:i + win_size] for i in range(0, len(sequence) -
win_size + 1, hop)]

# Функция формирования выборок (отрезки + ONE-метки)
def vectorize_sequence(seq_list, win_size, hop):
    class_count = len(seq_list)
    x, y = [], []
    for cls in range(class_count):
        vectors = split_sequence(seq_list[cls], win_size, hop)
        x += vectors
        y += [utils.to_categorical(cls, class_count)] * len(vectors)
    return np.array(x), np.array(y)

# Функция полной подготовки датасета по заданным параметрам
def prepare_dataset(vocab_size, win_size, win_hop):
    print(f'\n=== Подготовка датасета: VOCAB_SIZE={vocab_size},
WIN_SIZE={win_size}, WIN_HOP={win_hop} ===')
    tokenizer = Tokenizer(num_words=vocab_size,
```

```

        filters='!"#$%&()*+,-.
...:;<=>?@[\\]^_`{|}~«»\t\n\xa0\ufeff',
        lower=True, split=' ',
        oov_token='неизвестное_слово', char_level=False)

tokenizer.fit_on_texts(text_train)
seq_train = tokenizer.texts_to_sequences(text_train)
seq_test  = tokenizer.texts_to_sequences(text_test)
x_train, y_train = vectorize_sequence(seq_train, win_size, win_hop)
x_test,  y_test  = vectorize_sequence(seq_test,  win_size, win_hop)
print(f'Размер обучающей выборки: {x_train.shape}, размер тестовой
выборки: {x_test.shape}')
return x_train, y_train, x_test, y_test

```

1.1.5 Архитектура лучшей модели – Conv1D №1.

Из практического занятия в качестве лучшей выбрана сеть Conv1D №1, показавшая результат около 58% val_accrasy. Архитектура включает: эмбединг-слой с размерностью 10, слой пространственного дроп-аута (SpatialDropout1D), нормализацию батча, два последовательных слоя одномерной свёртки Conv1D с 20 фильтрами и ядром 5, слой подвыборки MaxPooling1D(2), Dropout(0.2), снова BatchNormalization, выравнивание Flatten и финальный полносвязный слой с softmax-активацией на 6 классов.

Листинг 1.1.2 – Архитектура нейронной сети Conv1D №1

```

# Функция создания лучшей модели – Conv1D #1
# (показала лучший результат: ~58% val_accuracy)
def build_best_model(vocab_size, win_size):
    model = Sequential()
    model.add(Embedding(vocab_size, 10, input_length=win_size))
    model.add(SpatialDropout1D(0.2))
    model.add(BatchNormalization())
    # Два слоя одномерной свёртки Conv1D
    model.add(Conv1D(20, 5, activation='relu', padding='same'))
    model.add(Conv1D(20, 5, activation='relu'))
    # Слой подвыборки/пулинга с функцией максимума
    model.add(MaxPooling1D(2))
    model.add(Dropout(0.2))
    model.add(BatchNormalization())

```

```
# Слой выравнивания
model.add(Flatten())
model.add(Dense(CLASS_COUNT, activation='softmax'))
return model
```

1.1.6 Функция компиляции, обучения и оценки.

Универсальная функция `train_and_evaluate` выполняет полный цикл: компилирует модель с оптимизатором `rmsprop` и функцией потерь `categorical_crossentropy`, обучает её, строит графики динамики `accuracy` и `loss`, рассчитывает нормализованную матрицу ошибок и выводит точность распознавания по каждому классу. Возвращается словарь с лучшей и финальной `val_accuracy` для дальнейшего сравнения экспериментов.

Листинг 1.1.3 – Функция компиляции, обучения и оценки

```
def train_and_evaluate(model, x_train, y_train, x_test, y_test,
                       title='', epochs=30, batch_size=200):
    model.compile(optimizer='rmsprop',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    history = model.fit(x_train, y_train,
                        epochs=epochs, batch_size=batch_size,
                        validation_data=(x_test, y_test), verbose=1)

    # Графики
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 5))
    fig.suptitle(f'Процесс обучения: {title}')
    ax1.plot(history.history['accuracy'], label='train accuracy')
    ax1.plot(history.history['val_accuracy'], label='val accuracy')
    ax1.set_xlabel('Эпоха'); ax1.set_ylabel('Accuracy'); ax1.legend()
    ax2.plot(history.history['loss'], label='train loss')
    ax2.plot(history.history['val_loss'], label='val loss')
    ax2.set_xlabel('Эпоха'); ax2.set_ylabel('Loss'); ax2.legend()
    plt.show()

    # Матрица ошибок
    y_pred = model.predict(x_test)
    cm = confusion_matrix(np.argmax(y_test, axis=1),
                          np.argmax(y_pred, axis=1), normalize='true')
    cm = np.around(cm, 3)
    accuracy = cm.diagonal().mean()
```

```
best_val_acc = max(history.history['val_accuracy'])
final_val_acc = history.history['val_accuracy'][-1]
return {'title': title, 'mean_acc': accuracy,
        'best_val_acc': best_val_acc, 'final_val_acc': final_val_acc}
```

1.1.7 Эксперимент 1 – базовая конфигурация (VOCAB=20000, WIN=1000, HOP=100).

Базовый эксперимент проводится с параметрами VOCAB_SIZE=20000, WIN_SIZE=1000, WIN_HOP=100. Это исходные значения, относительно которых будут сравниваться все остальные конфигурации. Размер обучающей выборки получается равным (17640, 1000), тестовой – (6686, 1000). Обучение выполняется в течение 30 эпох с размером батча 200.

```
# Список для сбора результатов всех экспериментов
results = []

# === Эксперимент 1: VOCAB=20000, WIN=1000, HOP=100 (базовая конфигурация) ===
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=20000, win_size=1000, win_hop=100)
model = build_best_model(vocab_size=20000, win_size=1000)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=20000, WIN=1000, HOP=100', epochs=30)
res['vocab'] = 20000; res['win'] = 1000; res['hop'] = 100
results.append(res)

=== Подготовка датасета: VOCAB_SIZE=20000, WIN_SIZE=1000, WIN_HOP=100 ===
Размер обучающей выборки: (17640, 1000), размер тестовой выборки: (6686, 1000)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument 'input_length' is deprecated. Just remove it.
  warnings.warn(
Epoch 1/30
89/89 28s 189ms/step - accuracy: 0.6276 - loss: 1.0758 - val_accuracy: 0.1611 - val_loss: 2.7892
Epoch 2/30
89/89 9s 99ms/step - accuracy: 0.9811 - loss: 0.0605 - val_accuracy: 0.1611 - val_loss: 7.0913
Epoch 3/30
89/89 9s 100ms/step - accuracy: 0.9890 - loss: 0.0399 - val_accuracy: 0.1378 - val_loss: 5.8537
Epoch 4/30
89/89 9s 100ms/step - accuracy: 0.9927 - loss: 0.0297 - val_accuracy: 0.2019 - val_loss: 7.8956
Epoch 5/30
89/89 9s 101ms/step - accuracy: 0.9942 - loss: 0.0193 - val_accuracy: 0.1894 - val_loss: 7.9631
Epoch 6/30
89/89 9s 100ms/step - accuracy: 0.9953 - loss: 0.0169 - val_accuracy: 0.3334 - val_loss: 6.9396
Epoch 7/30
89/89 9s 100ms/step - accuracy: 0.9955 - loss: 0.0136 - val_accuracy: 0.5515 - val_loss: 2.2372
Epoch 8/30
89/89 9s 99ms/step - accuracy: 0.9957 - loss: 0.0141 - val_accuracy: 0.4454 - val_loss: 4.5725
Epoch 9/30
89/89 9s 98ms/step - accuracy: 0.9965 - loss: 0.0184 - val_accuracy: 0.4853 - val_loss: 3.8063
Epoch 10/30
89/89 9s 97ms/step - accuracy: 0.9952 - loss: 0.0196 - val_accuracy: 0.4741 - val_loss: 4.0129
Epoch 11/30
89/89 9s 98ms/step - accuracy: 0.9960 - loss: 0.0147 - val_accuracy: 0.4343 - val_loss: 6.2981
Epoch 12/30
89/89 9s 98ms/step - accuracy: 0.9971 - loss: 0.0121 - val_accuracy: 0.4541 - val_loss: 4.3486
Epoch 13/30
89/89 9s 98ms/step - accuracy: 0.9969 - loss: 0.0118 - val_accuracy: 0.4725 - val_loss: 4.1869
Epoch 14/30
89/89 9s 98ms/step - accuracy: 0.9971 - loss: 0.0100 - val_accuracy: 0.5503 - val_loss: 2.5090
Epoch 15/30
89/89 9s 97ms/step - accuracy: 0.9977 - loss: 0.0101 - val_accuracy: 0.5326 - val_loss: 3.1221
Epoch 16/30
89/89 9s 96ms/step - accuracy: 0.9972 - loss: 0.0076 - val_accuracy: 0.5501 - val_loss: 3.2937
Epoch 17/30
89/89 9s 97ms/step - accuracy: 0.9982 - loss: 0.0064 - val_accuracy: 0.5969 - val_loss: 2.5735
Epoch 18/30
89/89 8s 93ms/step - accuracy: 0.9978 - loss: 0.0078 - val_accuracy: 0.5778 - val_loss: 2.8195
Epoch 19/30
89/89 11s 96ms/step - accuracy: 0.9973 - loss: 0.0097 - val_accuracy: 0.4785 - val_loss: 4.4606
Epoch 20/30
89/89 9s 97ms/step - accuracy: 0.9981 - loss: 0.0065 - val_accuracy: 0.5890 - val_loss: 2.7650
Epoch 21/30
89/89 8s 94ms/step - accuracy: 0.9977 - loss: 0.0085 - val_accuracy: 0.4870 - val_loss: 4.0103
Epoch 22/30
89/89 8s 95ms/step - accuracy: 0.9981 - loss: 0.0079 - val_accuracy: 0.5894 - val_loss: 2.5109
Epoch 23/30
89/89 8s 94ms/step - accuracy: 0.9981 - loss: 0.0053 - val_accuracy: 0.5453 - val_loss: 2.9980
Epoch 24/30
89/89 8s 93ms/step - accuracy: 0.9980 - loss: 0.0057 - val_accuracy: 0.5920 - val_loss: 2.4494
Epoch 25/30
89/89 8s 94ms/step - accuracy: 0.9973 - loss: 0.0153 - val_accuracy: 0.5687 - val_loss: 2.5984
Epoch 26/30
89/89 8s 91ms/step - accuracy: 0.9980 - loss: 0.0059 - val_accuracy: 0.5485 - val_loss: 3.1220
```

Рисунок 1.1.11 – Запуск эксперимента 1 и лог обучения по эпохам

Из лога видно, что точность на обучающей выборке быстро (уже ко 2-й эпохе) выходит на уровень ~98% и далее держится около 99%. Однако val_ассигасу сильно колеблется в диапазоне от 16% до 60% и не показывает стабильного роста, что указывает на переобучение модели.

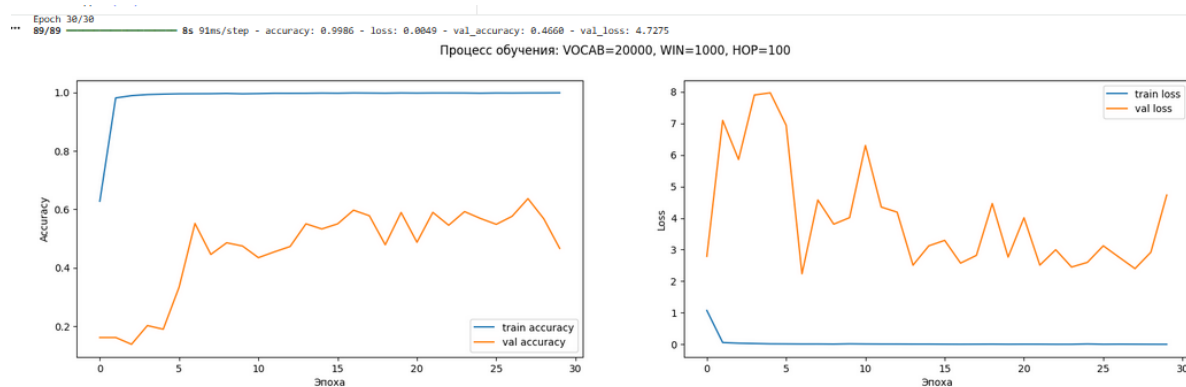


Рисунок 1.1.12 – График процесса обучения (VOCAB=20000, WIN=1000, HOP=100)

На графиках чётко виден разрыв между обучающей и проверочной выборками: train ассигасу практически сразу достигает 100%, в то время как val ассигасу «прыгает» в районе 0.4–0.6. График функции потерь подтверждает это: train loss падает практически до нуля, а val loss остаётся высоким (2–8), что является классическим признаком переобучения.

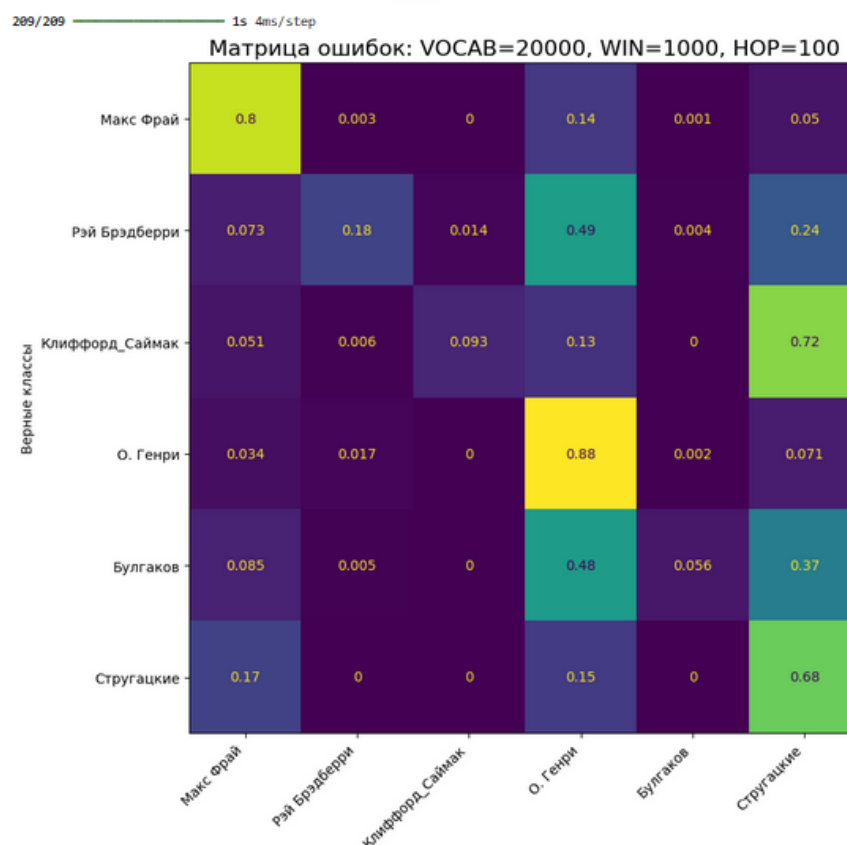


Рисунок 1.1.13 – Матрица ошибок (VOCAB=20000, WIN=1000, HOP=100)

Анализ матрицы ошибок показывает, что модель хорошо распознаёт только Макс Фрая (80%), О. Генри (88%) и Стругацких (68%). Класс Рэй Брэдбери почти полностью путается с О. Генри (49%) и Стругацкими (24%), Клиффорд Саймак практически не распознаётся (9.3%) и в 72% случаев определяется как Стругацкие. Булгаков также сильно путается с О. Генри (48%) и Стругацкими (37%).

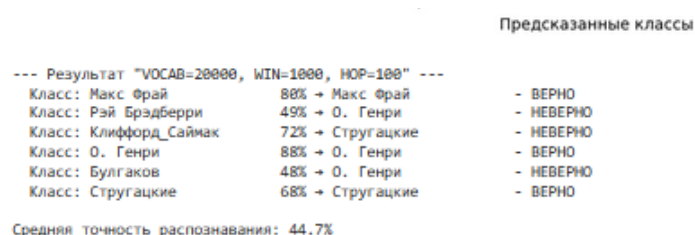


Рисунок 1.1.14 – Точность распознавания по классам (эксперимент 1)

Итог: средняя точность распознавания – 44.7%. Из шести классов модель уверенно определяет только три (Макс Фрай, О. Генри, Стругацкие), оставшиеся три (Брэдбери, Саймак, Булгаков) распознаются ниже случайного

уровня для 6 классов (16.7%) или близко к нему. Это базовая конфигурация для последующего сравнения.

1.1.8 Эксперимент 2 – уменьшение словаря до VOCAB_SIZE=5000.

Цель эксперимента – проверить, как уменьшение размера словаря влияет на способность модели разделять авторов. Параметры WIN_SIZE=1000 и WIN_HOP=100 остаются неизменными, что обеспечивает корректность сравнения с базовой конфигурацией.

```
1  # === Эксперимент 2: VOCAB=5000 ===
2  x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=5000, win_size=1000, win_hop=100)
3  model = build_best_model(vocab_size=5000, win_size=1000)
4  res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
5                          title='VOCAB=5000, WIN=1000, HOP=100', epochs=30)
6  res['vocab'] = 5000; res['win'] = 1000; res['hop'] = 100
7  results.append(res)
8
9  ...
10  === Подготовка датасета: VOCAB_SIZE=5000, WIN_SIZE=1000, WIN_HOP=100 ===
11  Размер обучающей выборки: (17640, 1000), размер тестовой выборки: (6686, 1000)
12  /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
13    warnings.warn(
14  Epoch 1/30
15  89/89 ----- 37s 309ms/step - accuracy: 0.5528 - loss: 1.2881 - val_accuracy: 0.1611 - val_loss: 2.3591
16  Epoch 2/30
17  89/89 ----- 19s 210ms/step - accuracy: 0.9426 - loss: 0.1711 - val_accuracy: 0.1971 - val_loss: 4.5060
18  Epoch 3/30
19  89/89 ----- 19s 210ms/step - accuracy: 0.9773 - loss: 0.0709 - val_accuracy: 0.1971 - val_loss: 6.9199
20  Epoch 4/30
21  89/89 ----- 19s 212ms/step - accuracy: 0.9862 - loss: 0.0469 - val_accuracy: 0.1971 - val_loss: 10.4089
22  Epoch 5/30
23  89/89 ----- 19s 210ms/step - accuracy: 0.9897 - loss: 0.0357 - val_accuracy: 0.2161 - val_loss: 8.4429
24  Epoch 6/30
25  89/89 ----- 19s 209ms/step - accuracy: 0.9936 - loss: 0.0215 - val_accuracy: 0.3558 - val_loss: 5.8963
26  Epoch 7/30
27  89/89 ----- 19s 208ms/step - accuracy: 0.9930 - loss: 0.0277 - val_accuracy: 0.4729 - val_loss: 2.7253
28  Epoch 8/30
29  89/89 ----- 19s 208ms/step - accuracy: 0.9948 - loss: 0.0155 - val_accuracy: 0.5126 - val_loss: 2.4845
30  Epoch 9/30
31  89/89 ----- 18s 206ms/step - accuracy: 0.9947 - loss: 0.0161 - val_accuracy: 0.6048 - val_loss: 1.7547
32  Epoch 10/30
33  89/89 ----- 18s 205ms/step - accuracy: 0.9960 - loss: 0.0102 - val_accuracy: 0.6590 - val_loss: 1.6378
34  Epoch 11/30
35  89/89 ----- 18s 207ms/step - accuracy: 0.9959 - loss: 0.0124 - val_accuracy: 0.6899 - val_loss: 1.9055
36  Epoch 12/30
37  89/89 ----- 18s 207ms/step - accuracy: 0.9971 - loss: 0.0114 - val_accuracy: 0.5706 - val_loss: 3.2145
38  Epoch 13/30
39  89/89 ----- 18s 202ms/step - accuracy: 0.9963 - loss: 0.0101 - val_accuracy: 0.6345 - val_loss: 1.8498
40  Epoch 14/30
41  89/89 ----- 18s 201ms/step - accuracy: 0.9967 - loss: 0.0101 - val_accuracy: 0.5631 - val_loss: 3.6642
42  Epoch 15/30
43  89/89 ----- 18s 197ms/step - accuracy: 0.9968 - loss: 0.0103 - val_accuracy: 0.2502 - val_loss: 13.1963
44  Epoch 16/30
45  89/89 ----- 18s 199ms/step - accuracy: 0.9961 - loss: 0.0140 - val_accuracy: 0.6907 - val_loss: 1.5489
46  Epoch 17/30
47  89/89 ----- 18s 200ms/step - accuracy: 0.9969 - loss: 0.0093 - val_accuracy: 0.6941 - val_loss: 1.9483
48  Epoch 18/30
49  89/89 ----- 18s 200ms/step - accuracy: 0.9976 - loss: 0.0072 - val_accuracy: 0.5250 - val_loss: 2.7335
50  Epoch 19/30
51  89/89 ----- 18s 198ms/step - accuracy: 0.9976 - loss: 0.0084 - val_accuracy: 0.6675 - val_loss: 1.6382
52  Epoch 20/30
53  89/89 ----- 17s 189ms/step - accuracy: 0.9974 - loss: 0.0079 - val_accuracy: 0.3319 - val_loss: 9.9765
54  Epoch 21/30
55  89/89 ----- 17s 192ms/step - accuracy: 0.9976 - loss: 0.0101 - val_accuracy: 0.5245 - val_loss: 4.4866
56  Epoch 22/30
57  89/89 ----- 17s 196ms/step - accuracy: 0.9980 - loss: 0.0005 - val_accuracy: 0.6684 - val_loss: 2.2260
58  Epoch 23/30
59  89/89 ----- 17s 190ms/step - accuracy: 0.9975 - loss: 0.0078 - val_accuracy: 0.2934 - val_loss: 37.1178
60  Epoch 24/30
61  89/89 ----- 17s 189ms/step - accuracy: 0.9968 - loss: 0.0170 - val_accuracy: 0.6448 - val_loss: 1.9580
62  Epoch 25/30
63  89/89 ----- 17s 191ms/step - accuracy: 0.9974 - loss: 0.0085 - val_accuracy: 0.6054 - val_loss: 2.6759
64  Epoch 26/30
65  89/89 ----- 16s 185ms/step - accuracy: 0.9982 - loss: 0.0057 - val_accuracy: 0.5799 - val_loss: 2.2677
```

Рисунок 1.1.15 – Запуск эксперимента 2 и лог обучения по эпохам

Лог обучения показывает, что модель ведёт себя похоже на базовую: train ассурасу быстро выходит на 99%, а val_ассурасу колеблется в более широком диапазоне 16%–69%. При этом скорость одной эпохи увеличилась примерно в два раза (с ~9 до ~18 секунд), что объясняется большим количеством сэмплов, попадающих под более компактный словарь.

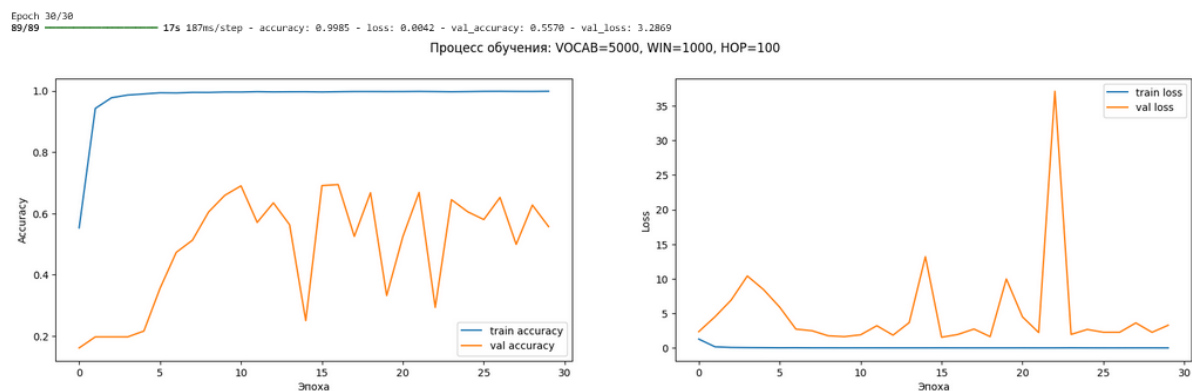


Рисунок 1.1.16 – График процесса обучения (VOCAB=5000, WIN=1000, HOP=100)

Графики обучения демонстрируют ещё более выраженные колебания val accuracy и val loss по сравнению с базовым экспериментом. На 22-й эпохе наблюдается резкий скачок val_loss до значения почти 38, что говорит о крайне нестабильной работе модели на тестовой выборке. Однако максимальная val_accuracy достигает ~69%, что выше базовой.

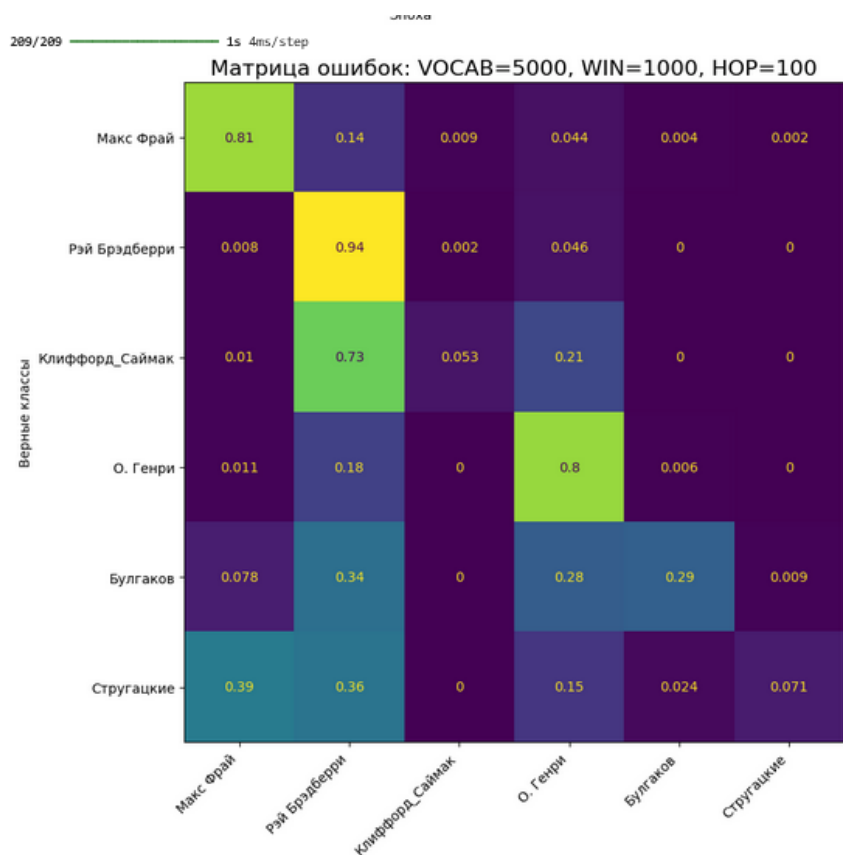


Рисунок 1.1.17 – Матрица ошибок (VOCAB=5000, WIN=1000, HOP=100)

Структура ошибок изменилась: Рэй Брэдбери теперь распознаётся с точностью 94% (против 18% в базовом эксперименте) – это резкий скачок.

Однако появилась обратная проблема: модель стала «склеивать» большинство классов с Брэдбери. Так, Клиффорд Саймак в 73% случаев классифицируется как Брэдбери, Булгаков – в 34%, Стругацкие – в 36%. О. Генри по-прежнему распознаётся хорошо (80%), Макс Фрай – на уровне 81%.

Предсказанные классы		
--- Результат "VOCAB=5000, WIN=1000, HOP=100" ---		
Класс: Макс Фрай	88% → Макс Фрай	- ВЕРНО
Класс: Рэй Брэдбери	94% → Рэй Брэдбери	- ВЕРНО
Класс: Клиффорд Саймак	73% → Рэй Брэдбери	- НЕВЕРНО
Класс: О. Генри	88% → О. Генри	- ВЕРНО
Класс: Булгаков	34% → Рэй Брэдбери	- НЕВЕРНО
Класс: Стругацкие	39% → Макс Фрай	- НЕВЕРНО
Средняя точность распознавания: 49.4%		

Рисунок 1.1.18 – Точность распознавания по классам (эксперимент 2)

Итог: средняя точность распознавания – 49.4%, что выше базовой на ~4.7 п.п. Уменьшение словаря заставило модель опираться на более частотные общеупотребительные слова, что помогло Брэдбери и Макс Фраю, но почти разрушило распознавание Саймака, Булгакова и Стругацких.

1.1.9 Эксперимент 3 – VOCAB_SIZE=10000.

Промежуточный эксперимент с размером словаря 10000 слов. Это значение находится посередине между базовым (20000) и уменьшенным (5000) и должно показать «компромиссную» картину.

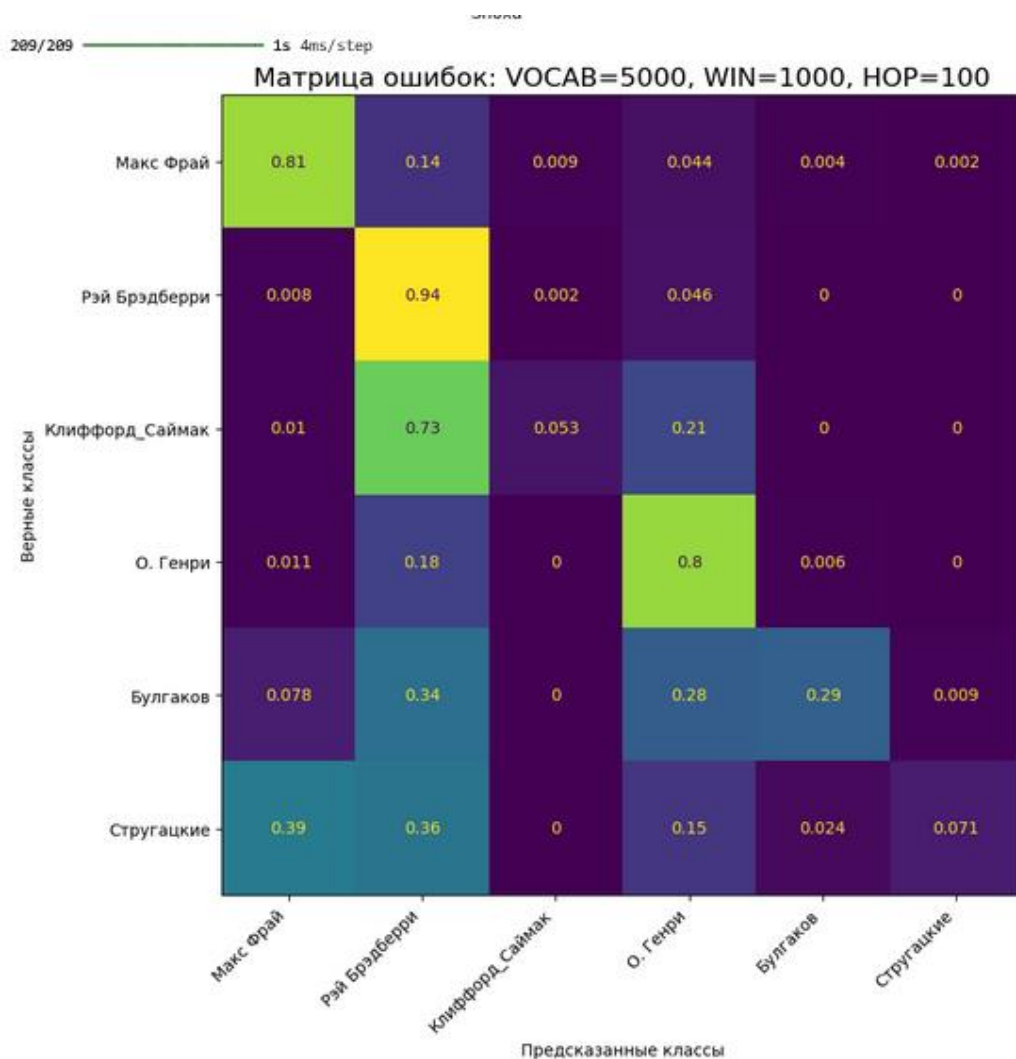


Рисунок 1.1.19 – Запуск эксперимента 3 и лог обучения по эпохам

Лог обучения практически полностью совпадает по динамике с базовым экспериментом: train accuracy выходит на ~99% уже к 3-й эпохе, val_accuracy постепенно растёт от ~29% до ~63–65% к финальным эпохам. Время эпохи – около 13 секунд, что в полтора раза больше базового.

```

● # === Эксперимент 3: VOCAB=10000 ===
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=10000, win_size=1000, win_hop=100)
model = build_best_model(vocab_size=10000, win_size=1000)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=10000, WIN=1000, HOP=100', epochs=30)
res['vocab'] = 10000; res['win'] = 1000; res['hop'] = 100
results.append(res)

...

=== Подготовка датасета: VOCAB_SIZE=10000, WIN_SIZE=1000, WIN_HOP=100 ===
Размер обучающей выборки: (17640, 1000), размер тестовой выборки: (6686, 1000)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Epoch 1/30
89/89 ————— 27s 219ms/step - accuracy: 0.5561 - loss: 1.2764 - val_accuracy: 0.2929 - val_loss: 2.0611
Epoch 2/30
89/89 ————— 13s 149ms/step - accuracy: 0.9587 - loss: 0.1212 - val_accuracy: 0.1817 - val_loss: 2.1622
Epoch 3/30
89/89 ————— 13s 150ms/step - accuracy: 0.9879 - loss: 0.0400 - val_accuracy: 0.3163 - val_loss: 3.3348
Epoch 4/30
89/89 ————— 13s 146ms/step - accuracy: 0.9917 - loss: 0.0271 - val_accuracy: 0.3845 - val_loss: 3.2364
Epoch 5/30
89/89 ————— 21s 147ms/step - accuracy: 0.9935 - loss: 0.0200 - val_accuracy: 0.5377 - val_loss: 2.7029
Epoch 6/30
89/89 ————— 13s 148ms/step - accuracy: 0.9944 - loss: 0.0198 - val_accuracy: 0.3491 - val_loss: 5.6006
Epoch 7/30
89/89 ————— 13s 148ms/step - accuracy: 0.9942 - loss: 0.0201 - val_accuracy: 0.3697 - val_loss: 4.1319
Epoch 8/30
89/89 ————— 13s 146ms/step - accuracy: 0.9946 - loss: 0.0182 - val_accuracy: 0.5678 - val_loss: 1.6605
Epoch 9/30
89/89 ————— 13s 145ms/step - accuracy: 0.9966 - loss: 0.0103 - val_accuracy: 0.4493 - val_loss: 3.9925
Epoch 10/30
89/89 ————— 13s 144ms/step - accuracy: 0.9956 - loss: 0.0147 - val_accuracy: 0.5773 - val_loss: 2.1011
Epoch 11/30
89/89 ————— 13s 143ms/step - accuracy: 0.9963 - loss: 0.0131 - val_accuracy: 0.5556 - val_loss: 2.3991
Epoch 12/30
89/89 ————— 13s 143ms/step - accuracy: 0.9965 - loss: 0.0109 - val_accuracy: 0.4936 - val_loss: 3.0219
Epoch 13/30
89/89 ————— 13s 143ms/step - accuracy: 0.9969 - loss: 0.0106 - val_accuracy: 0.5037 - val_loss: 2.7667
Epoch 14/30
89/89 ————— 13s 141ms/step - accuracy: 0.9978 - loss: 0.0074 - val_accuracy: 0.5256 - val_loss: 3.1893
Epoch 15/30
89/89 ————— 13s 143ms/step - accuracy: 0.9973 - loss: 0.0095 - val_accuracy: 0.6066 - val_loss: 2.0435
Epoch 16/30
89/89 ————— 13s 141ms/step - accuracy: 0.9976 - loss: 0.0076 - val_accuracy: 0.3850 - val_loss: 7.6637
Epoch 17/30
89/89 ————— 12s 134ms/step - accuracy: 0.9974 - loss: 0.0096 - val_accuracy: 0.5483 - val_loss: 2.3656
Epoch 18/30
89/89 ————— 13s 141ms/step - accuracy: 0.9981 - loss: 0.0075 - val_accuracy: 0.6535 - val_loss: 1.6829
Epoch 19/30
89/89 ————— 12s 136ms/step - accuracy: 0.9981 - loss: 0.0052 - val_accuracy: 0.5678 - val_loss: 2.7278
Epoch 20/30
89/89 ————— 12s 139ms/step - accuracy: 0.9980 - loss: 0.0061 - val_accuracy: 0.5389 - val_loss: 2.9892
Epoch 21/30
89/89 ————— 12s 133ms/step - accuracy: 0.9974 - loss: 0.0094 - val_accuracy: 0.6210 - val_loss: 2.1270
Epoch 22/30
89/89 ————— 12s 138ms/step - accuracy: 0.9977 - loss: 0.0090 - val_accuracy: 0.6292 - val_loss: 2.4204
Epoch 23/30
89/89 ————— 12s 138ms/step - accuracy: 0.9974 - loss: 0.0093 - val_accuracy: 0.5335 - val_loss: 3.1091
Epoch 24/30
89/89 ————— 12s 135ms/step - accuracy: 0.9980 - loss: 0.0068 - val_accuracy: 0.6340 - val_loss: 2.0193
Epoch 25/30
89/89 ————— 12s 139ms/step - accuracy: 0.9978 - loss: 0.0072 - val_accuracy: 0.5497 - val_loss: 2.9345
Epoch 26/30
89/89 ————— 12s 132ms/step - accuracy: 0.9982 - loss: 0.0062 - val_accuracy: 0.6346 - val_loss: 2.2705

```

Рисунок 1.1.20 – График процесса обучения (VOCAB=10000, WIN=1000, HOP=100)

На графике обучения видно, что val accuracy более устойчиво держится в диапазоне 0.5–0.65, без таких глубоких провалов, как в эксперименте 2. График потерь показывает аналогичную картину: val loss колеблется в коридоре 1.5–4 (без катастрофических всплесков), что говорит о более стабильной работе модели по сравнению с VOCAB=5000.

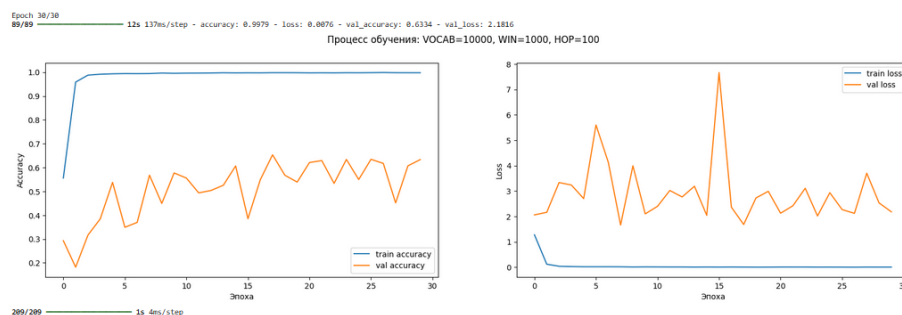


Рисунок 1.1.21 – Матрица ошибок (VOCAB=10000, WIN=1000, HOP=100)

Картина ошибок улучшилась по большинству классов одновременно. Макс Фрай – 67%, Рэй Брэдбери – 89%, Клиффорд Саймак – 38% (заметно лучше базовых 9% и без катастрофического перетекания в Стругацких), О. Генри – 82%, Стругацкие – 63%. Слабым местом остаётся Булгаков (35%), значительная часть его текстов всё ещё уходит в О. Генри (24%) и Брэдбери (13%).

```

--- Результат "VOCAB=10000, WIN=1000, HOP=100" ---
Класс: Макс Фрай      67% → Макс Фрай      - ВЕРНО
Класс: Рэй Брэдбери   89% → Рэй Брэдбери   - ВЕРНО
Класс: Клиффорд_Саймак 44% → Рэй Брэдбери   - НЕВЕРНО
Класс: О. Генри       82% → О. Генри       - ВЕРНО
Класс: Булгаков        35% → Булгаков        - ВЕРНО
Класс: Стругацкие      63% → Стругацкие      - ВЕРНО

Средняя точность распознавания: 62.4%

```

Рисунок 1.1.22 – Точность распознавания по классам (эксперимент 3)

Итог: средняя точность распознавания – 62.4%, что значительно выше базовой (44.7%) и эксперимента 2 (49.4%). VOCAB=10000 даёт лучшее равновесие между «глубоким» (20000) и «компактным» (5000) словарями: распознавание становится более равномерным по классам, без сильных перекосов одного автора в другого. На данный момент это лучшая конфигурация по словарю.

1.1.10 Эксперимент 4 – увеличение словаря до VOCAB_SIZE=40000.

Цель эксперимента – проверить, поможет ли дальнейшее увеличение размера словаря (вдвое относительно базового). Параметры WIN_SIZE=1000 и WIN_HOP=100 остаются неизменными.


```

# === Эксперимент 4: VOCAB=40000 ===
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=40000, win_size=1000, win_hop=100)
model = build_best_model(vocab_size=40000, win_size=1000)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=40000, WIN=1000, HOP=100', epochs=30)
res['vocab'] = 40000; res['win'] = 1000; res['hop'] = 100
results.append(res)

'''
=== Подготовка датасета: VOCAB_SIZE=40000, WIN_SIZE=1000, WIN_HOP=100 ===
Размер обучающей выборки: (17640, 1000), размер тестовой выборки: (6686, 1000)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument 'input_length' is deprecated. Just remove it.
warnings.warn(
Epoch 1/30
89/89 ————— 21s 147ms/step - accuracy: 0.6324 - loss: 1.0462 - val_accuracy: 0.2988 - val_loss: 2.7981
Epoch 2/30
89/89 ————— 6s 66ms/step - accuracy: 0.9836 - loss: 0.0549 - val_accuracy: 0.0782 - val_loss: 7.5965
Epoch 3/30
89/89 ————— 6s 66ms/step - accuracy: 0.9910 - loss: 0.0303 - val_accuracy: 0.3359 - val_loss: 3.9050
Epoch 4/30
89/89 ————— 6s 65ms/step - accuracy: 0.9952 - loss: 0.0153 - val_accuracy: 0.1805 - val_loss: 7.0570
Epoch 5/30
89/89 ————— 6s 66ms/step - accuracy: 0.9956 - loss: 0.0135 - val_accuracy: 0.2798 - val_loss: 4.2693
Epoch 6/30
89/89 ————— 6s 65ms/step - accuracy: 0.9954 - loss: 0.0145 - val_accuracy: 0.4176 - val_loss: 3.0698
Epoch 7/30
89/89 ————— 6s 65ms/step - accuracy: 0.9958 - loss: 0.0143 - val_accuracy: 0.4276 - val_loss: 4.3706
Epoch 8/30
89/89 ————— 6s 64ms/step - accuracy: 0.9963 - loss: 0.0105 - val_accuracy: 0.4305 - val_loss: 4.0060
Epoch 9/30
89/89 ————— 6s 66ms/step - accuracy: 0.9972 - loss: 0.0089 - val_accuracy: 0.4849 - val_loss: 3.0815
Epoch 10/30
89/89 ————— 6s 64ms/step - accuracy: 0.9972 - loss: 0.0076 - val_accuracy: 0.4867 - val_loss: 4.0104
Epoch 11/30
89/89 ————— 6s 65ms/step - accuracy: 0.9971 - loss: 0.0104 - val_accuracy: 0.5351 - val_loss: 3.1167
Epoch 12/30
89/89 ————— 6s 64ms/step - accuracy: 0.9973 - loss: 0.0092 - val_accuracy: 0.5476 - val_loss: 2.8002
Epoch 13/30
89/89 ————— 6s 65ms/step - accuracy: 0.9973 - loss: 0.0107 - val_accuracy: 0.4375 - val_loss: 5.1231
Epoch 14/30
89/89 ————— 6s 64ms/step - accuracy: 0.9973 - loss: 0.0083 - val_accuracy: 0.5612 - val_loss: 2.7141
Epoch 15/30
89/89 ————— 6s 65ms/step - accuracy: 0.9969 - loss: 0.0102 - val_accuracy: 0.5600 - val_loss: 4.3424
Epoch 16/30
89/89 ————— 10s 65ms/step - accuracy: 0.9982 - loss: 0.0069 - val_accuracy: 0.6017 - val_loss: 3.1971
Epoch 17/30
89/89 ————— 6s 64ms/step - accuracy: 0.9975 - loss: 0.0088 - val_accuracy: 0.5181 - val_loss: 3.4530
Epoch 18/30
89/89 ————— 6s 63ms/step - accuracy: 0.9981 - loss: 0.0064 - val_accuracy: 0.5380 - val_loss: 3.6906
Epoch 19/30
89/89 ————— 6s 64ms/step - accuracy: 0.9981 - loss: 0.0070 - val_accuracy: 0.4310 - val_loss: 5.7219
Epoch 20/30
89/89 ————— 6s 63ms/step - accuracy: 0.9981 - loss: 0.0067 - val_accuracy: 0.5212 - val_loss: 3.9676
Epoch 21/30
89/89 ————— 6s 64ms/step - accuracy: 0.9981 - loss: 0.0083 - val_accuracy: 0.5544 - val_loss: 3.1493
Epoch 22/30
89/89 ————— 6s 62ms/step - accuracy: 0.9984 - loss: 0.0051 - val_accuracy: 0.4807 - val_loss: 4.2804
Epoch 23/30
89/89 ————— 5s 61ms/step - accuracy: 0.9980 - loss: 0.0070 - val_accuracy: 0.4592 - val_loss: 4.6301
Epoch 24/30
89/89 ————— 6s 62ms/step - accuracy: 0.9981 - loss: 0.0057 - val_accuracy: 0.5138 - val_loss: 4.0920
Epoch 25/30
89/89 ————— 6s 63ms/step - accuracy: 0.9981 - loss: 0.0069 - val_accuracy: 0.5407 - val_loss: 3.5993
Epoch 26/30
89/89 ————— 5s 61ms/step - accuracy: 0.9986 - loss: 0.0078 - val_accuracy: 0.4945 - val_loss: 3.8661
'''

```

Рисунок 1.1.23 – Запуск эксперимента 4 и лог обучения по эпохам

Лог обучения показывает поведение, очень похожее на базовый эксперимент: train accuracy практически сразу достигает 99%, а val accuracy медленно растёт с ~30% в начале до ~50–56% к финальным эпохам. Время одной эпохи стабилизируется около 6 секунд, что сопоставимо с базовым.

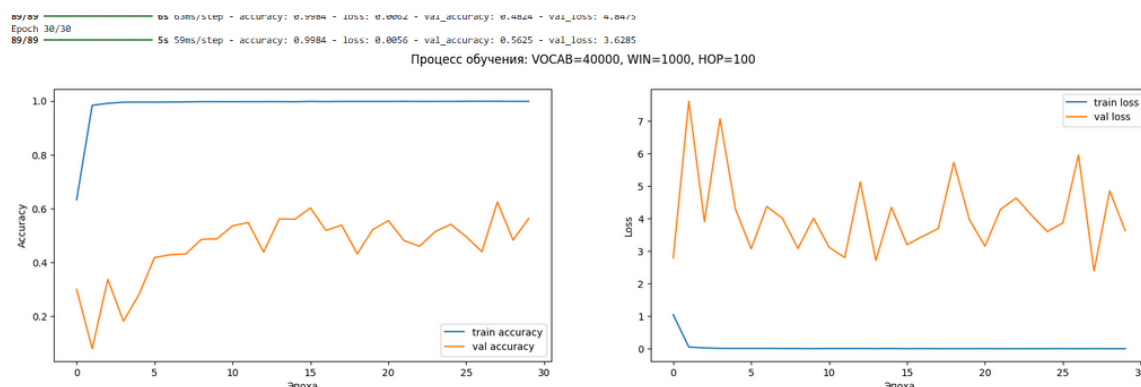


Рисунок 1.1.24 – График процесса обучения (VOCAB=40000, WIN=1000, HOP=100)

Графики демонстрируют типичную картину переобучения: train accuracy у самого потолка (≈ 1.0), val accuracy колеблется в коридоре 0.4–0.6. Val loss также не сходится и держится в диапазоне 3–7, что подтверждает: дальнейшее увеличение словаря не помогает обобщению – наоборот, даёт модели больше «уникальных» слов, по которым она запоминает обучающую выборку.

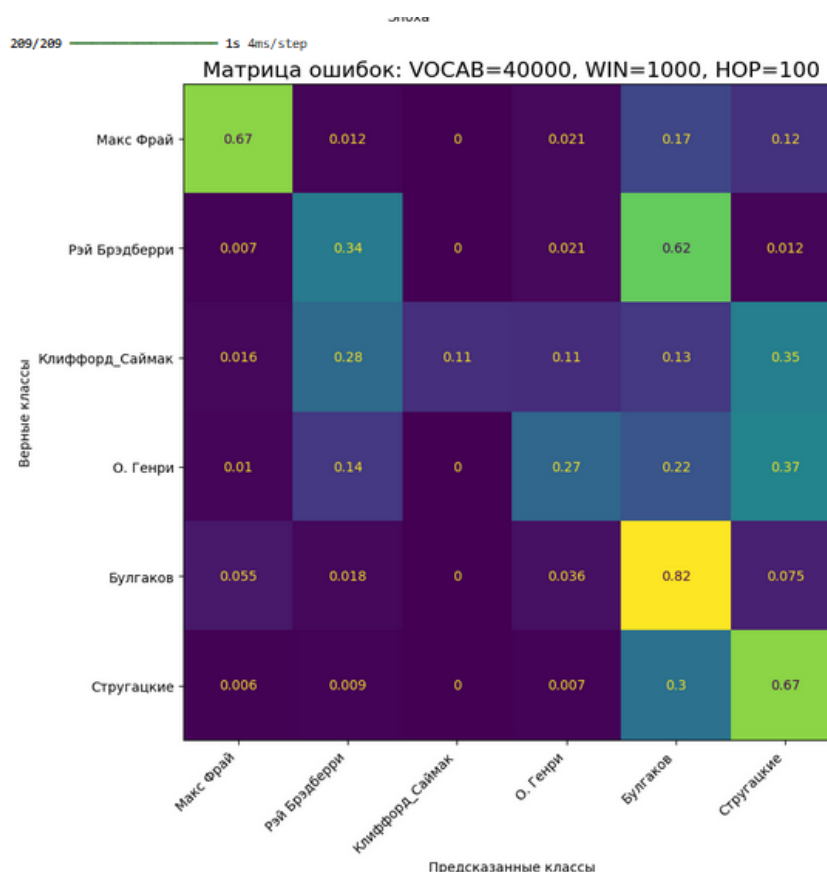


Рисунок 1.1.25 – Матрица ошибок (VOCAB=40000, WIN=1000, HOP=100)

Матрица ошибок показывает резкий сдвиг приоритетов модели: лучше всего распознаётся Булгаков (82%), затем Стругацкие (67%) и Макс Фрай (67%). При этом Брэдберри падает до 34% (часть «уходит» в Булгакова – 62%), Клиффорд Саймак опять почти не распознаётся (11%), О. Генри – всего 27% (37% его текстов модель относит к Стругацким, 22% – к Булгакову). То есть распределение ошибок изменилось, но средняя точность не выросла.

```

--- Результат "VOCAB=40000, WIN=1000, HOP=100" ---
Класс: Макс Фрай          67% → Макс Фрай      - ВЕРНО
Класс: Рэй Брэдберри     62% → Булгаков        - НЕВЕРНО
Класс: Клиффорд Саймак   35% → Стругацкие      - НЕВЕРНО
Класс: О. Генри          37% → Стругацкие      - НЕВЕРНО
Класс: Булгаков          82% → Булгаков        - ВЕРНО
Класс: Стругацкие        67% → Стругацкие      - ВЕРНО

Средняя точность распознавания: 48.0%

```

Рисунок 1.1.26 – Точность распознавания по классам (эксперимент 4)

Итог: средняя точность распознавания – 48.0%. По сравнению с базовой (44.7%) – небольшой прирост, но это значительно хуже, чем VOCAB=10000 (62.4%). Это подтверждает, что для данного датасета зависимость точности от размера словаря не является монотонной: оптимум находится около 10000 слов, а слишком большой словарь только усиливает переобучение.

1.1.11 Эксперимент 5 – уменьшение окна WIN_SIZE=500, WIN_HOP=50.

Меняется пара (WIN_SIZE, WIN_HOP): теперь окно вдвое короче (500 токенов), а шаг тоже вдвое меньше (50). VOCAB_SIZE возвращается к базовому значению 20000. За счёт более частого шага размер обучающей выборки увеличивается до (35336, 500), тестовой – до (13427, 500), то есть примерно в два раза больше сэмплов, чем в базе.

```

# === Эксперимент 5: WIN=500, HOP=50 ===
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=20000, win_size=500, win_hop=50)
model = build_best_model(vocab_size=20000, win_size=500)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=20000, WIN=500, HOP=50', epochs=30)
res['vocab'] = 20000; res['win'] = 500; res['hop'] = 50
results.append(res)

***
=== Подготовка датасета: VOCAB_SIZE=20000, WIN_SIZE=500, WIN_HOP=50 ===
Размер обучающей выборки: (35336, 500), размер тестовой выборки: (13427, 500)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
warnings.warn(
Epoch 1/30
177/177 ----- 24s 90ms/step - accuracy: 0.7172 - loss: 0.7890 - val_accuracy: 0.1139 - val_loss: 3.5767
Epoch 2/30
177/177 ----- 9s 53ms/step - accuracy: 0.9872 - loss: 0.0420 - val_accuracy: 0.3016 - val_loss: 4.2327
Epoch 3/30
177/177 ----- 9s 53ms/step - accuracy: 0.9914 - loss: 0.0269 - val_accuracy: 0.4196 - val_loss: 2.6818
Epoch 4/30
177/177 ----- 9s 53ms/step - accuracy: 0.9948 - loss: 0.0162 - val_accuracy: 0.5694 - val_loss: 2.1644
Epoch 5/30
177/177 ----- 9s 53ms/step - accuracy: 0.9957 - loss: 0.0155 - val_accuracy: 0.5295 - val_loss: 2.9659
Epoch 6/30
177/177 ----- 9s 53ms/step - accuracy: 0.9956 - loss: 0.0145 - val_accuracy: 0.6269 - val_loss: 1.9345
Epoch 7/30
177/177 ----- 9s 52ms/step - accuracy: 0.9968 - loss: 0.0103 - val_accuracy: 0.5600 - val_loss: 3.2061
Epoch 8/30
177/177 ----- 9s 53ms/step - accuracy: 0.9969 - loss: 0.0114 - val_accuracy: 0.6710 - val_loss: 1.9251
Epoch 9/30
177/177 ----- 9s 52ms/step - accuracy: 0.9971 - loss: 0.0099 - val_accuracy: 0.3979 - val_loss: 9.5344
Epoch 10/30
177/177 ----- 9s 52ms/step - accuracy: 0.9970 - loss: 0.0096 - val_accuracy: 0.6444 - val_loss: 2.1811
Epoch 11/30
177/177 ----- 9s 52ms/step - accuracy: 0.9973 - loss: 0.0093 - val_accuracy: 0.5233 - val_loss: 3.2559
Epoch 12/30
177/177 ----- 9s 52ms/step - accuracy: 0.9974 - loss: 0.0079 - val_accuracy: 0.5708 - val_loss: 3.5243
Epoch 13/30
177/177 ----- 9s 51ms/step - accuracy: 0.9979 - loss: 0.0063 - val_accuracy: 0.6919 - val_loss: 2.1162
Epoch 14/30
177/177 ----- 9s 51ms/step - accuracy: 0.9983 - loss: 0.0068 - val_accuracy: 0.6258 - val_loss: 2.3517
Epoch 15/30
177/177 ----- 9s 51ms/step - accuracy: 0.9981 - loss: 0.0065 - val_accuracy: 0.6000 - val_loss: 2.8613
Epoch 16/30
177/177 ----- 9s 52ms/step - accuracy: 0.9976 - loss: 0.0077 - val_accuracy: 0.6662 - val_loss: 2.3395
Epoch 17/30
177/177 ----- 9s 51ms/step - accuracy: 0.9980 - loss: 0.0057 - val_accuracy: 0.4637 - val_loss: 4.9445
Epoch 18/30
177/177 ----- 9s 51ms/step - accuracy: 0.9978 - loss: 0.0081 - val_accuracy: 0.5438 - val_loss: 3.8769
Epoch 19/30
177/177 ----- 9s 51ms/step - accuracy: 0.9982 - loss: 0.0052 - val_accuracy: 0.6368 - val_loss: 2.2307
Epoch 20/30
177/177 ----- 9s 51ms/step - accuracy: 0.9984 - loss: 0.0062 - val_accuracy: 0.6581 - val_loss: 2.1825
Epoch 21/30
177/177 ----- 9s 50ms/step - accuracy: 0.9985 - loss: 0.0048 - val_accuracy: 0.6243 - val_loss: 2.6433
Epoch 22/30
177/177 ----- 9s 50ms/step - accuracy: 0.9988 - loss: 0.0044 - val_accuracy: 0.6181 - val_loss: 2.5587
Epoch 23/30
177/177 ----- 9s 50ms/step - accuracy: 0.9984 - loss: 0.0059 - val_accuracy: 0.5572 - val_loss: 3.4809
Epoch 24/30
177/177 ----- 9s 53ms/step - accuracy: 0.9986 - loss: 0.0048 - val_accuracy: 0.5378 - val_loss: 4.1703
Epoch 25/30
177/177 ----- 9s 50ms/step - accuracy: 0.9986 - loss: 0.0042 - val_accuracy: 0.5300 - val_loss: 4.9291
Epoch 26/30
177/177 ----- 9s 49ms/step - accuracy: 0.9984 - loss: 0.0056 - val_accuracy: 0.5081 - val_loss: 5.5885

```

Рисунок 1.1.27 – Запуск эксперимента 5 и лог обучения по эпохам

Из лога видно, что точность на обучающей выборке быстро достигает 99%, как и в других экспериментах. Val_accurasy колеблется в диапазоне ~30%–70% и в среднем держится на уровне 55–65%, что заметно выше базового. Время одной эпохи – около 9 секунд за счёт большего числа батчей (177 на эпоху против 89 в базе).

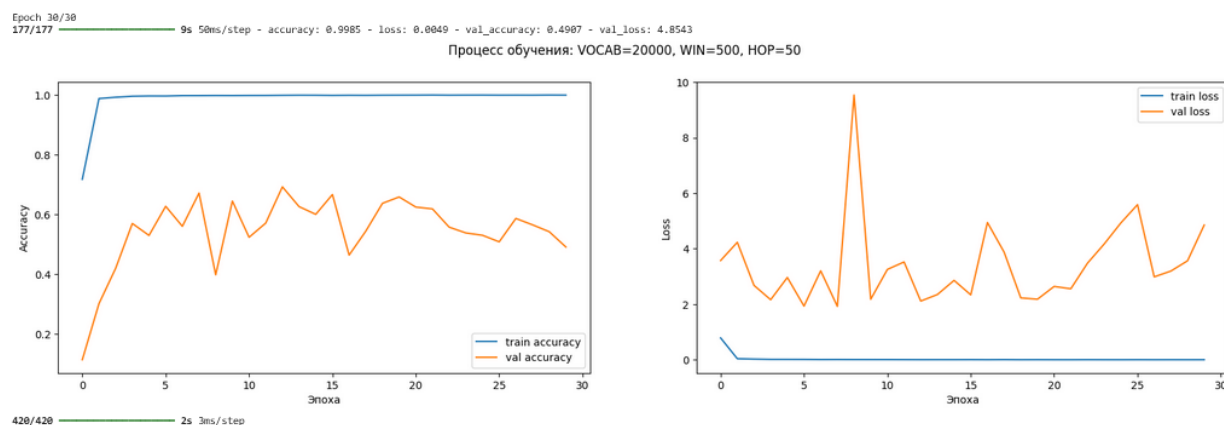


Рисунок 1.1.28 – График процесса обучения (VOCAB=20000, WIN=500, HOP=50)

На графиках видно, что val accuracy в среднем выше, чем в базовом эксперименте, но кривая всё ещё «прыгает». Val loss также нестабилен: на 8-й эпохе виден всплеск до значения почти 10, после чего возвращается в обычный коридор 2–5. Это говорит о том, что укорочение окна повышает разнообразие сэмплов, но сохраняет проблему переобучения.

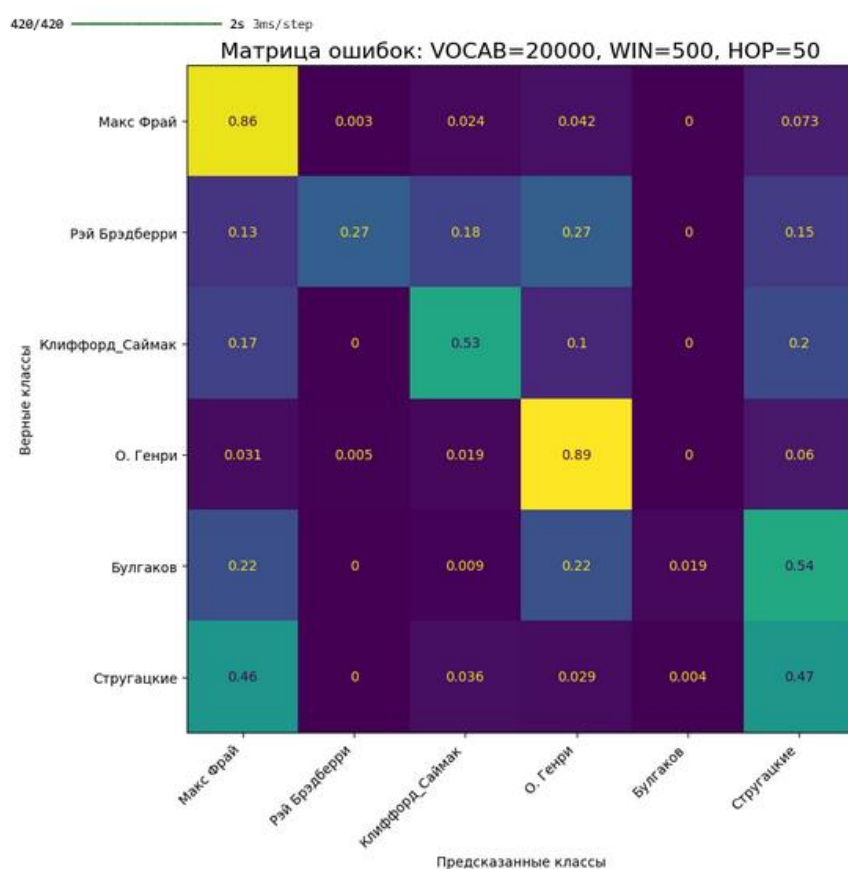


Рисунок 1.1.29 – Матрица ошибок (VOCAB=20000, WIN=500, HOP=50)

Анализ матрицы ошибок показывает: Макс Фрай – 86%, О. Генри – 89% (оба лучших класса в эксперименте), Клиффорд Саймак – 53% (заметное улучшение по сравнению с базовыми 9%), Стругацкие – 47%. Однако Брэдли практически не распознаётся (27%, часть путается с Максом Фраем – 13%, и О. Генри – 27%), Булгаков почти не распознаётся (1.9%, и 54% его текстов уходит в Стругацких).

```

--- Результат "VOCAB=20000, WIN=500, HOP=50" ---
Класс: Макс Фрай          86% → Макс Фрай      - ВЕРНО
Класс: Рэй Брэдли         27% → О. Генри      - НЕВЕРНО
Класс: Клиффорд Саймак   53% → Клиффорд Саймак - ВЕРНО
Класс: О. Генри          88% → О. Генри      - ВЕРНО
Класс: Булгаков          54% → Стругацкие   - НЕВЕРНО
Класс: Стругацкие        47% → Стругацкие   - ВЕРНО

Средняя точность распознавания: 50.4%

```

Рисунок 1.1.30 – Точность распознавания по классам (эксперимент 5)

Итог: средняя точность распознавания – 50.4%, что выше базовой на ~5.7 п.п. Уменьшение окна позволило модели улавливать более локальные стилистические особенности и заметно улучшило распознавание Макса Фрая, О. Генри и Саймака. Однако по совокупности результат всё ещё хуже, чем у эксперимента 3 (62.4% при VOCAB=10000).

1.1.12 Эксперимент 6 – увеличение окна WIN_SIZE=2000, WIN_HOP=200.

Меняется пара (WIN_SIZE, WIN_HOP): теперь окно вдвое длиннее (2000 токенов), шаг тоже вдвое больше (200). VOCAB_SIZE остаётся базовым (20000). Размер обучающей выборки получается (8792, 2000), тестовой – (3314, 2000), то есть в два раза меньше сэмплов, чем в базе, но каждый – вдвое длиннее.

```

# === Эксперимент 6: WIN=2000, HOP=200 ===
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=20000, win_size=2000, win_hop=200)
model = build_best_model(vocab_size=20000, win_size=2000)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=20000, WIN=2000, HOP=200', epochs=30)
res['vocab'] = 20000; res['win'] = 2000; res['hop'] = 200
results.append(res)

...

=== Подготовка датасета: VOCAB_SIZE=20000, WIN_SIZE=2000, WIN_HOP=200 ===
Размер обучающей выборки: (8792, 2000), размер тестовой выборки: (3314, 2000)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Epoch 1/30
44/44 ————— 28s 404ms/step - accuracy: 0.5641 - loss: 1.2953 - val_accuracy: 0.1970 - val_loss: 1.9554
Epoch 2/30
44/44 ————— 8s 191ms/step - accuracy: 0.9675 - loss: 0.1008 - val_accuracy: 0.1970 - val_loss: 2.0398
Epoch 3/30
44/44 ————— 8s 191ms/step - accuracy: 0.9816 - loss: 0.0586 - val_accuracy: 0.1970 - val_loss: 3.4487
Epoch 4/30
44/44 ————— 8s 190ms/step - accuracy: 0.9911 - loss: 0.0310 - val_accuracy: 0.2571 - val_loss: 2.4265
Epoch 5/30
44/44 ————— 8s 190ms/step - accuracy: 0.9940 - loss: 0.0189 - val_accuracy: 0.1973 - val_loss: 2.3388
Epoch 6/30
44/44 ————— 8s 188ms/step - accuracy: 0.9923 - loss: 0.0222 - val_accuracy: 0.1973 - val_loss: 5.1567
Epoch 7/30
44/44 ————— 8s 189ms/step - accuracy: 0.9960 - loss: 0.0159 - val_accuracy: 0.2001 - val_loss: 5.2212
Epoch 8/30
44/44 ————— 8s 186ms/step - accuracy: 0.9926 - loss: 0.0263 - val_accuracy: 0.1973 - val_loss: 9.9424
Epoch 9/30
44/44 ————— 8s 185ms/step - accuracy: 0.9964 - loss: 0.0119 - val_accuracy: 0.2996 - val_loss: 5.4973
Epoch 10/30
44/44 ————— 8s 189ms/step - accuracy: 0.9936 - loss: 0.0206 - val_accuracy: 0.3524 - val_loss: 4.2070
Epoch 11/30
44/44 ————— 8s 187ms/step - accuracy: 0.9953 - loss: 0.0178 - val_accuracy: 0.2969 - val_loss: 5.5382
Epoch 12/30
44/44 ————— 8s 186ms/step - accuracy: 0.9951 - loss: 0.0152 - val_accuracy: 0.4804 - val_loss: 2.5483
Epoch 13/30
44/44 ————— 8s 184ms/step - accuracy: 0.9972 - loss: 0.0085 - val_accuracy: 0.3241 - val_loss: 6.5146
Epoch 14/30
44/44 ————— 8s 183ms/step - accuracy: 0.9961 - loss: 0.0108 - val_accuracy: 0.5000 - val_loss: 2.6680
Epoch 15/30
44/44 ————— 8s 185ms/step - accuracy: 0.9962 - loss: 0.0193 - val_accuracy: 0.4161 - val_loss: 5.3741
Epoch 16/30
44/44 ————— 8s 181ms/step - accuracy: 0.9964 - loss: 0.0155 - val_accuracy: 0.4668 - val_loss: 5.2447
Epoch 17/30
44/44 ————— 8s 183ms/step - accuracy: 0.9965 - loss: 0.0117 - val_accuracy: 0.5275 - val_loss: 3.2808
Epoch 18/30
44/44 ————— 8s 182ms/step - accuracy: 0.9964 - loss: 0.0145 - val_accuracy: 0.5471 - val_loss: 2.9357
Epoch 19/30
44/44 ————— 8s 182ms/step - accuracy: 0.9981 - loss: 0.0081 - val_accuracy: 0.4994 - val_loss: 3.7819
Epoch 20/30
44/44 ————— 8s 183ms/step - accuracy: 0.9957 - loss: 0.0188 - val_accuracy: 0.5124 - val_loss: 4.7827
Epoch 21/30
44/44 ————— 8s 180ms/step - accuracy: 0.9967 - loss: 0.0109 - val_accuracy: 0.3603 - val_loss: 6.1136
Epoch 22/30
44/44 ————— 8s 180ms/step - accuracy: 0.9981 - loss: 0.0064 - val_accuracy: 0.4700 - val_loss: 4.0598
Epoch 23/30
44/44 ————— 8s 172ms/step - accuracy: 0.9972 - loss: 0.0134 - val_accuracy: 0.5625 - val_loss: 2.7274
Epoch 24/30
44/44 ————— 8s 175ms/step - accuracy: 0.9978 - loss: 0.0075 - val_accuracy: 0.4608 - val_loss: 5.9950
Epoch 25/30
44/44 ————— 8s 179ms/step - accuracy: 0.9975 - loss: 0.0098 - val_accuracy: 0.4689 - val_loss: 7.2901
Epoch 26/30
44/44 ————— 8s 180ms/step - accuracy: 0.9974 - loss: 0.0122 - val_accuracy: 0.5706 - val_loss: 2.9993

```

Рисунок 1.1.31 – Запуск эксперимента 6 и лог обучения по эпохам

Лог обучения показывает: train ассурасу быстро (за 3–4 эпохи) выходит на 99%, val_ассурасу в начале долго «топчется» на уровне 19–20% (около случайного для 6 классов), после ~10-й эпохи начинает расти и к финалу достигает 55–60%. Время одной эпохи – около 8 секунд, несмотря на меньшее число батчей (44 против 89), за счёт удвоенной длины окна.

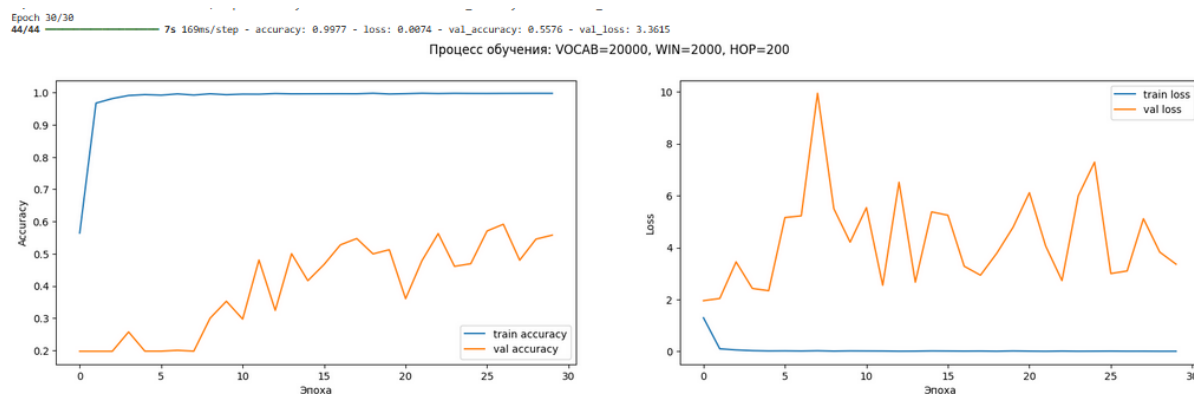


Рисунок 1.1.32 – График процесса обучения (VOCAB=20000, WIN=2000, HOP=200)

На графиках хорошо виден характерный «плато-старт»: первые 7 эпох val accuracy вообще не растёт (~0.2), и только потом модель начинает осмысленно обучаться, выходя в коридор 0.4–0.6. Val loss долго остаётся высоким и нестабильным, с пиками до 10. Длинное окно требует больше времени для адаптации, но в итоге даёт более устойчивый результат.

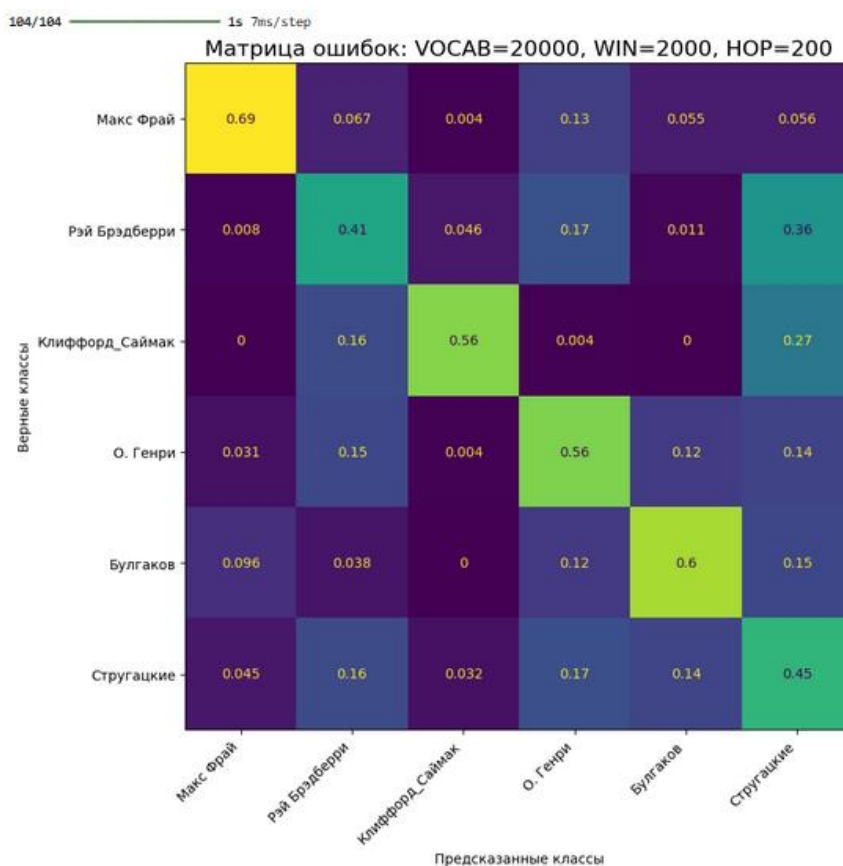


Рисунок 1.1.33 – Матрица ошибок (VOCAB=20000, WIN=2000, HOP=200)

Матрица ошибок: Макс Фрай – 69%, Клиффорд Саймак – 56% (значительное улучшение), О. Генри – 56%, Булгаков – 60% (тоже большой

прирост по сравнению с базой), Стругацкие – 45%. Брэдбери всё ещё проблемен (41%, часть путается со Стругацкими – 36%). Картина в целом наиболее равномерная среди всех экспериментов, без класса с точностью ниже 40%.

```

--- Результат "VOCAB=20000, WIN=2000, HOP=200" ---
Класс: Макс Фрай          69% → Макс Фрай          - ВЕРНО
Класс: Рэй Брэдбери       41% → Рэй Брэдбери       - ВЕРНО
Класс: Клиффорд Саймак   56% → Клиффорд Саймак   - ВЕРНО
Класс: О. Генри           56% → О. Генри           - ВЕРНО
Класс: Булгаков           68% → Булгаков           - ВЕРНО
Класс: Стругацкие        45% → Стругацкие        - ВЕРНО

Средняя точность распознавания: 54.4%

```

Рисунок 1.1.34 – Точность распознавания по классам (эксперимент 6)

Итог: средняя точность распознавания – 54.4%, что выше базовой на ~9.7 п.п. Увеличение окна с 1000 до 2000 даёт стабильный прирост и более равномерное распознавание классов (особенно для Саймака и Булгакова), хотя на старте обучение идёт медленнее. По итоговой точности эксперимент 6 уступает только эксперименту 3 (VOCAB=10000).

1.1.13 Сводная таблица результатов и выводы.

Для удобного сравнения все шесть экспериментов сведены в одну таблицу. В качестве основной метрики используется средняя точность распознавания по диагонали матрицы ошибок (mean_acc), а также максимальная и финальная val_accuracy за все эпохи обучения.

Листинг 1.1.4 – Формирование сводной таблицы результатов

```
# Сводная таблица всех экспериментов
```

```
import pandas as pd
```

```
df = pd.DataFrame(results)
```

```
df = df[['vocab', 'win', 'hop', 'mean_acc', 'best_val_acc', 'final_val_acc']]
```

```
df.columns = ['VOCAB_SIZE', 'WIN_SIZE', 'WIN_HOP',
```

```
              'Средняя точность (по матрице)',
```

```
              'Лучшая val_accuracy', 'Финальная val_accuracy']
```

```
# Преобразуем в проценты
```

```
for col in ['Средняя точность (по матрице)', 'Лучшая val_accuracy',
```

```
           'Финальная val_accuracy']:
```

```
df[col] = (df[col] * 100).round(1).astype(str) + '%'

print('\nСВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ')
display(df)
```

	VOCAB_SIZE	WIN_SIZE	WIN_HOP	Средняя точность (по матрице)	Лучшая val_accuracy	Финальная val_accuracy
0	20000	1000	100	44.7%	63.6%	46.6%
1	5000	1000	100	49.4%	69.4%	55.7%
2	10000	1000	100	62.4%	65.3%	63.3%
3	40000	1000	100	48.0%	62.4%	56.3%
4	20000	500	50	50.4%	69.2%	49.1%
5	20000	2000	200	54.4%	59.1%	55.8%

Рисунок 1.1.36 – Сводная таблица результатов всех экспериментов

Общий вывод: одномерная свёрточная сеть Conv1D №1 действительно справляется с задачей классификации художественных текстов по авторам, но её качество сильно зависит от гиперпараметров предобработки – размера словаря и параметров скользящего окна. Оптимум для данного датасета достигается при умеренном словаре (~10000 слов) или при увеличении длины окна до 2000. Дальнейшие улучшения могут быть получены за счёт регуляризации, использования предобученных эмбеддингов или комбинирования Conv1D с рекуррентными слоями.

1.2 Реализация той же задачи классификации текстов с использованием библиотеки PyTorch.

1.2.1 Подключение библиотек и загрузка датасета.

В отличие от Keras, в PyTorch модули импортируются по отдельности: torch для основных операций, torch.nn для слоёв нейронной сети, torch.nn.functional для функций активации, torch.utils.data для DataLoader и TensorDataset. Также подключены: sklearn.metrics для матрицы ошибок, gdown для загрузки датасета, collections.Counter для подсчёта частот слов в собственном токенизаторе. Дополнительно определяется устройство для обучения (DEVICE = cuda при наличии GPU, иначе cpu) и фиксируются сиды (numpy, torch, cuda) для воспроизводимости результатов. После этого датасет

рукописных текстов с шестью писателями загружается из облачного хранилища тем же способом, что и в Keras-версии.

```

# Работа с массивами
import numpy as np

# PyTorch
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset

# Метрики
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Загрузка датасетов
import gdown

# Прочее
import os
import time
import re
import matplotlib.pyplot as plt
from collections import Counter
from IPython.display import display

%matplotlib inline

# Устройство для обучения
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Используем устройство:', DEVICE)

# Фиксируем сиды для воспроизводимости
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

... Используем устройство: cuda

# Загрузим датасет
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/17/writers.zip', None, quiet=True)

'writers.zip'

# Распакуем архив
!unzip -qq writers.zip -d writers/

# Константы для разбора имён файлов
FILE_DIR = 'writers'
SIG_TRAIN = 'обучающая'
SIG_TEST = 'тестовая'
```

Рисунок 1.2.1 – Подключение библиотек, фиксация сидов и загрузка датасета

1.2.2 Загрузка текстов по классам.

Логика разбиения файлов из папки writers на классы и выборки полностью идентична Keras-версии: с помощью регулярного выражения извлекается имя автора и признак выборки, а содержимое файла дописывается в строку нужного класса. В отличие от первой реализации, здесь явно указана кодировка utf-8 при открытии файла. В результате формируются два списка text_train и text_test и список из 6 классов: Макс Фрай, Рэй Брэдбери, Клиффорд Саймак, О. Генри, Булгаков, Стругацкие.

```
1  # Загрузка текстов по классам
    CLASS_LIST = []
    text_train = []
    text_test = []

    file_list = os.listdir(FILE_DIR)
    for file_name in file_list:
        m = re.match(r'\((.+)\) (\S+)_', file_name)
        if not m:
            continue
        class_name = m[1]
        subset_name = m[2].lower()

        is_train = SIG_TRAIN in subset_name
        is_test = SIG_TEST in subset_name
        if not (is_train or is_test):
            continue

        if class_name not in CLASS_LIST:
            print(f'Добавление класса "{class_name}"')
            CLASS_LIST.append(class_name)
            text_train.append('')
            text_test.append('')

        cls = CLASS_LIST.index(class_name)
        print(f' → файл "{file_name}" в класс "{CLASS_LIST[cls]}" , {subset_name} выборка')

        with open(f'{FILE_DIR}/{file_name}', 'r', encoding='utf-8') as f:
            text = f.read()
            subset = text_train if is_train else text_test
            subset[cls] += ' ' + text.replace('\n', ' ')

    CLASS_COUNT = len(CLASS_LIST)
    print(f'\nКлассы: {CLASS_LIST}')
    print(f'Количество классов: {CLASS_COUNT}')

✓  ... Добавление класса "Макс Фрай"
    → файл "(Макс Фрай) Тестовая_2 вместе.txt" в класс "Макс Фрай", тестовая выборка
    Добавление класса "Рэй Брэдберри"
    → файл "(Рэй Брэдберри) Обучающая_22 вместе.txt" в класс "Рэй Брэдберри", обучающая выборка
    Добавление класса "Клиффорд Саймак"
    → файл "(Клиффорд Саймак) Обучающая_5 вместе.txt" в класс "Клиффорд Саймак", обучающая выборка
    Добавление класса "О. Генри"
    → файл "(О. Генри) Обучающая_50 вместе.txt" в класс "О. Генри", обучающая выборка
    Добавление класса "Булгаков"
    → файл "(Булгаков) Обучающая_5 вместе.txt" в класс "Булгаков", обучающая выборка
    → файл "(О. Генри) Тестовая_20 вместе.txt" в класс "О. Генри", тестовая выборка
    → файл "(Булгаков) Тестовая_2 вместе.txt" в класс "Булгаков", тестовая выборка
    → файл "(Рэй Брэдберри) Тестовая_8 вместе.txt" в класс "Рэй Брэдберри", тестовая выборка
    → файл "(Макс Фрай) Обучающая_5 вместе.txt" в класс "Макс Фрай", обучающая выборка
    Добавление класса "Стругацкие"
    → файл "(Стругацкие) Тестовая_2 вместе.txt" в класс "Стругацкие", тестовая выборка
    → файл "(Стругацкие) Обучающая_5 вместе.txt" в класс "Стругацкие", обучающая выборка
    → файл "(Клиффорд Саймак) Тестовая_2 вместе.txt" в класс "Клиффорд Саймак", тестовая выборка

    Классы: ['Макс Фрай', 'Рэй Брэдберри', 'Клиффорд Саймак', 'О. Генри', 'Булгаков', 'Стругацкие']
    Количество классов: 6

1  # Контекстный менеджер для измерения времени
    class timex:
        def __enter__(self):
            self.t = time.time()
            return self
        def __exit__(self, type, value, traceback):
            print('Время обработки: {:.2f} c'.format(time.time() - self.t))
```

Рисунок 1.2.2 – Загрузка текстов по классам и формирование списка CLASS_LIST

1.2.3 Свой токенизатор на PyTorch.

В Keras был готовый класс Tokenizer, в PyTorch его аналога нет. Поэтому реализован собственный класс SimpleTokenizer, который выполняет те же шаги: чистит текст от пунктуации регулярным выражением, приводит к

нижнему регистру, считает частоты слов через `collections.Counter`, оставляет `VOCAB_SIZE - 2` самых частых слов (индексы 0 и 1 зарезервированы под `padding` и OOV-токен «неизвестное слово»), а затем преобразует тексты в последовательности индексов методом `texts_to_sequences`. Это полностью повторяет поведение `keras.preprocessing.text.Tokenizer`.

```
# Простой токенизатор-аналог keras.Tokenizer
class SimpleTokenizer:
    def __init__(self, num_words, oov_token='<OOV>'):
        self.num_words = num_words
        self.oov_token = oov_token
        self.word_index = {} # слово + индекс
        # Регекс для удаления пунктуации (как filters в Keras)
        self.filters_re = re.compile(
            r'["#$%&()*+,\-./:;<=>@[\\]\^_`{|}~\n\t\x0b\x0f\xff]'
        )

    def _clean(self, text):
        text = self.filters_re.sub(' ', text.lower())
        return text.split()

    def fit_on_texts(self, texts):
        counter = Counter()
        for text in texts:
            counter.update(self._clean(text))
        # Резервируем 0 под padding, 1 под OOV
        # Берём top (num_words - 2) самых частых слов — на остальные индексы
        most_common = counter.most_common(self.num_words - 2)
        self.word_index = {self.oov_token: 1}
        for i, (word, _) in enumerate(most_common, start=2):
            self.word_index[word] = i

    def texts_to_sequences(self, texts):
        oov_idx = self.word_index[self.oov_token]
        result = []
        for text in texts:
            seq = [self.word_index.get(w, oov_idx) for w in self._clean(text)]
            result.append(seq)
        return result
```

Рисунок 1.2.3 – Реализация класса SimpleTokenizer

1.2.4 Универсальные функции для подготовки и обучения.

Функции `split_sequence` (разбиение последовательности скользящим окном) и `vectorize_sequence` (формирование выборок из отрезков и меток классов) реализованы аналогично Keras-версии. Главное отличие — тип данных `np.int64`, который является стандартным для слоя `Embedding` и функции потерь `CrossEntropyLoss` в PyTorch. Функция `prepare_dataset` собирает всё вместе: создаёт `SimpleTokenizer`, обучает его на `text_train`, преобразует тексты в последовательности и формирует обучающую и тестовую выборки.

```

# Разбиение скользящим окном
def split_sequence(sequence, win_size, hop):
    return [sequence[i:i + win_size] for i in range(0, len(sequence) - win_size + 1, hop)]

def vectorize_sequence(seq_list, win_size, hop):
    """Возвращает x (np.int64) и y (np.int64) – class indices."""
    class_count = len(seq_list)
    x, y = [], []
    for cls in range(class_count):
        vectors = split_sequence(seq_list[cls], win_size, hop)
        x += vectors
        y += [cls] * len(vectors)
    # int64 – стандартный тип для embedding и CrossEntropyLoss в PyTorch
    return np.array(x, dtype=np.int64), np.array(y, dtype=np.int64)

def prepare_dataset(vocab_size, win_size, win_hop):
    print(f'\nVOCAB_SIZE={vocab_size}, WIN_SIZE={win_size}, WIN_HOP={win_hop}')

    tokenizer = SimpleTokenizer(num_words=vocab_size)
    tokenizer.fit_on_texts(text_train)

    seq_train = tokenizer.texts_to_sequences(text_train)
    seq_test = tokenizer.texts_to_sequences(text_test)

    x_train, y_train = vectorize_sequence(seq_train, win_size, win_hop)
    x_test, y_test = vectorize_sequence(seq_test, win_size, win_hop)

    print(f'x_train: {x_train.shape}, x_test: {x_test.shape}')
    return x_train, y_train, x_test, y_test

```

Рисунок 1.2.4 – Универсальные функции подготовки данных в PyTorch

1.2.5 Архитектура модели Conv1DTextClassifier.

Модель оформлена как класс Conv1DTextClassifier, наследующий nn.Module, и точно повторяет структуру Conv1D №1 из Keras-версии: слой Embedding с padding_idx=0 (нулевой индекс не обновляется при обучении), пространственный дроп-аут (реализован через Dropout2d с тем же эффектом, что SpatialDropout1D в Keras), BatchNorm1d по эмбединг-признакам, два слоя одномерной свёртки Conv1d (20 фильтров, kernel=5, первый с padding=2 для сохранения длины, второй без padding), MaxPool1d(2), Dropout(0.2), вторая нормализация и финальный полносвязный слой fc на CLASS_COUNT выходов. В forward выполняется permute, чтобы привести данные к формату (batch, embed, win), который ожидает Conv1d.

Листинг 1.2.1 – Класс Conv1DTextClassifier

```

class Conv1DTextClassifier(nn.Module):
    def __init__(self, vocab_size, win_size, num_classes, embed_dim=10):
        super().__init__()
        # Embedding (padding_idx=0 – нулевой индекс не обновляется при
        # обучении)

```

```

self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
# SpatialDropout1D-аналог: дропаем целые признаковые карты по
временной оси
self.spatial_dropout = nn.Dropout2d(0.2)
# BatchNorm по эмбединг-признакам
self.bn1 = nn.BatchNorm1d(embed_dim)
# Conv1: padding='same' для kernel=5 → padding=2; длина не меняется
self.conv1 = nn.Conv1d(embed_dim, 20, kernel_size=5, padding=2)
# Conv2: без padding → длина уменьшается на 4
self.conv2 = nn.Conv1d(20, 20, kernel_size=5)
self.maxpool1 = nn.MaxPool1d(2)
self.dropout = nn.Dropout(0.2)
self.bn2 = nn.BatchNorm1d(20)
# Считаем размер на входе классификатора:
# WIN → conv1 (same) → WIN → conv2 → WIN-4 → maxpool/2 → (WIN-4)//2
self.flat_dim = 20 * ((win_size - 4) // 2)
self.fc = nn.Linear(self.flat_dim, num_classes)

def forward(self, x):
    x = self.embedding(x)                # (batch, win, embed)
    x = x.permute(0, 2, 1).unsqueeze(-1) # (batch, embed, win, 1)
    x = self.spatial_dropout(x).squeeze(-1) # (batch, embed, win)
    x = self.bn1(x)
    x = F.relu(self.conv1(x))            # (batch, 20, win)
    x = F.relu(self.conv2(x))            # (batch, 20, win-4)
    x = self.maxpool1(x)                 # (batch, 20, (win-4)/2)
    x = self.dropout(x)
    x = self.bn2(x)
    x = x.flatten(1)                    # (batch, 20 * ((win-
4)/2))
    x = self.fc(x)                      # (batch, num_classes) -
логиты
    return x

def build_best_model(vocab_size, win_size):
    return Conv1DTextClassifier(vocab_size=vocab_size,
                                win_size=win_size,

```

```
num_classes=CLASS_COUNT,  
embed_dim=10).to(DEVICE)
```

1.2.6 Цикл обучения и оценки.

Универсальная функция `train_and_evaluate` реализует полный цикл, аналогичный Keras-версии: создаёт `DataLoader`-ы из NumPy-массивов через `TensorDataset`, использует оптимизатор `RMSprop` (то же, что `rmsprop` в Keras) и функцию потерь `CrossEntropyLoss`. Каждая эпоха состоит из двух фаз: `train` (с обновлением весов) и `val` (без `grad`). Метрики `train/val loss` и `accuracy` собираются в словарь `history`, далее строятся графики и матрица ошибок – так же, как в Keras-версии. Возвращается словарь с лучшей и финальной `val_accuracy`.


```

def train_and_evaluate(model, x_train, y_train, x_test, y_test,
                       title='', epochs=30, batch_size=200, lr=1e-3):
    # DataLoader-w
    train_loader = DataLoader(
        TensorDataset(torch.from_numpy(x_train), torch.from_numpy(y_train)),
        batch_size=batch_size, shuffle=True
    )
    test_loader = DataLoader(
        TensorDataset(torch.from_numpy(x_test), torch.from_numpy(y_test)),
        batch_size=batch_size, shuffle=False
    )

    # RMSprop(Keras-овский 'rmsprop' с дефолтными параметрами)
    optimizer = torch.optim.RMSprop(model.parameters(), lr=lr)
    criterion = nn.CrossEntropyLoss()

    history = {'loss': [], 'accuracy': [], 'val_loss': [], 'val_accuracy': []}

    for epoch in range(epochs):
        # train
        model.train()
        train_loss, train_correct, train_total = 0.0, 0, 0
        for xb, yb in train_loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            optimizer.zero_grad()
            logits = model(xb)
            loss = criterion(logits, yb)
            loss.backward()
            optimizer.step()

            train_loss += loss.item() * xb.size(0)
            train_correct += (logits.argmax(1) == yb).sum().item()
            train_total += xb.size(0)

        # val
        model.eval()
        val_loss, val_correct, val_total = 0.0, 0, 0
        with torch.no_grad():
            for xb, yb in test_loader:
                xb, yb = xb.to(DEVICE), yb.to(DEVICE)
                logits = model(xb)
                loss = criterion(logits, yb)
                val_loss += loss.item() * xb.size(0)
                val_correct += (logits.argmax(1) == yb).sum().item()
                val_total += xb.size(0)

        tr_loss = train_loss / train_total
        tr_acc = train_correct / train_total
        va_loss = val_loss / val_total
        va_acc = val_correct / val_total

        history['loss'].append(tr_loss)
        history['accuracy'].append(tr_acc)
        history['val_loss'].append(va_loss)
        history['val_accuracy'].append(va_acc)

        print(f'Epoch {epoch+1:>2}/{epochs} | '
              f'loss: {tr_loss:.4f} acc: {tr_acc:.4f} | '
              f'val_loss: {va_loss:.4f} val_acc: {va_acc:.4f}')

    # Графики
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 5))
    fig.suptitle(f'Процесс обучения: {title}')
    ax1.plot(history['accuracy'], label='train accuracy')
    ax1.plot(history['val accuracy'], label='val accuracy')

```

Рисунок 1.2.7 – Функция train_and_evaluate

```

# Графики
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 5))
fig.suptitle(f'Процесс обучения: {title}')
ax1.plot(history['accuracy'], label='train accuracy')
ax1.plot(history['val_accuracy'], label='val accuracy')
ax1.set_xlabel('Эпоха'); ax1.set_ylabel('Accuracy'); ax1.legend(); ax1.grid(True)
ax2.plot(history['loss'], label='train loss')
ax2.plot(history['val_loss'], label='val loss')
ax2.set_xlabel('Эпоха'); ax2.set_ylabel('Loss'); ax2.legend(); ax2.grid(True)
plt.show()

# Матрица ошибок
model.eval()
all_preds = []
with torch.no_grad():
    for xb, _ in test_loader:
        xb = xb.to(DEVICE)
        preds = model(xb).argmax(1).cpu().numpy()
        all_preds.append(preds)
y_pred = np.concatenate(all_preds)

cm = confusion_matrix(y_test, y_pred, normalize='true')
cm = np.around(cm, 3)

fig, ax = plt.subplots(figsize=(10, 10))
ax.set_title(f'Матрица ошибок: {title}', fontsize=14)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_LIST)
disp.plot(ax=ax)
plt.gca().images[-1].colorbar.remove()
plt.xlabel('Предсказанные классы'); plt.ylabel('Верные классы')
fig.autofmt_xdate(rotation=45)
plt.show()

accuracy = cm.diagonal().mean()
print(f'\n--- Результат "{title}" ---')
for cls in range(CLASS_COUNT):
    cls_pred = np.argmax(cm[cls])
    msg = 'ВЕРНО' if cls_pred == cls else 'НЕВЕРНО'
    print(f'Класс: {CLASS_LIST[cls]:<20} {100*cm[cls, cls_pred]:3.0f}% → {CLASS_LIST[cls_pred]:<20} - {msg}')
print(f'\nСредняя точность распознавания: {100*accuracy:.1f}%')

return {
    'title': title,
    'mean_acc': accuracy,
    'best_val_acc': max(history['val_accuracy']),
    'final_val_acc': history['val_accuracy'][-1]
}

```

Рисунок 1.2.8 – Функция train_and_evaluate

1.2.7 Эксперимент 1 – базовая конфигурация (VOCAB=20000, WIN=1000, HOP=100).

Базовый эксперимент в PyTorch проводится с теми же параметрами, что и в Keras-версии: VOCAB_SIZE=20000, WIN_SIZE=1000, WIN_HOP=100. Перед инициализацией модели вызывается torch.manual_seed(SEED), чтобы каждый эксперимент стартовал с одних и тех же случайных весов. Размер обучающей выборки: (17640, 1000), тестовой: (6686, 1000) – полностью совпадает с Keras-версией.

```

# Сборщик результатов
results = []

# Эксперимент 1: VOCAB=20000, WIN=1000, HOP=100
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=20000, win_size=1000, win_hop=100)
torch.manual_seed(SEED) # фиксируем инициализацию для каждой новой модели
model = build_best_model(vocab_size=20000, win_size=1000)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=20000, WIN=1000, HOP=100', epochs=30)
res['vocab'] = 20000; res['win'] = 1000; res['hop'] = 100
results.append(res)

VOCAB_SIZE=20000, WIN_SIZE=1000, WIN_HOP=100
x_train: (17640, 1000), x_test: (6686, 1000)
Epoch 1/30 | loss: 1.9421 acc: 0.3838 | val_loss: 2.0768 val_acc: 0.3286
Epoch 2/30 | loss: 1.0549 acc: 0.6209 | val_loss: 2.3689 val_acc: 0.3174
Epoch 3/30 | loss: 0.7020 acc: 0.7442 | val_loss: 1.9608 val_acc: 0.4322
Epoch 4/30 | loss: 0.5123 acc: 0.8155 | val_loss: 2.9171 val_acc: 0.3240
Epoch 5/30 | loss: 0.3777 acc: 0.8616 | val_loss: 1.9688 val_acc: 0.4728
Epoch 6/30 | loss: 0.2975 acc: 0.8922 | val_loss: 3.4748 val_acc: 0.3136
Epoch 7/30 | loss: 0.2368 acc: 0.9164 | val_loss: 2.5815 val_acc: 0.4119
Epoch 8/30 | loss: 0.1908 acc: 0.9320 | val_loss: 2.5399 val_acc: 0.4445
Epoch 9/30 | loss: 0.1576 acc: 0.9439 | val_loss: 2.2437 val_acc: 0.4798
Epoch 10/30 | loss: 0.1360 acc: 0.9517 | val_loss: 2.7035 val_acc: 0.4640
Epoch 11/30 | loss: 0.1195 acc: 0.9569 | val_loss: 2.1546 val_acc: 0.5277
Epoch 12/30 | loss: 0.1025 acc: 0.9645 | val_loss: 3.1540 val_acc: 0.4475
Epoch 13/30 | loss: 0.0956 acc: 0.9664 | val_loss: 3.2034 val_acc: 0.4771
Epoch 14/30 | loss: 0.0832 acc: 0.9716 | val_loss: 2.2696 val_acc: 0.5452
Epoch 15/30 | loss: 0.0763 acc: 0.9728 | val_loss: 2.2779 val_acc: 0.5488
Epoch 16/30 | loss: 0.0793 acc: 0.9724 | val_loss: 2.5102 val_acc: 0.5462
Epoch 17/30 | loss: 0.0703 acc: 0.9755 | val_loss: 2.6707 val_acc: 0.5326
Epoch 18/30 | loss: 0.0589 acc: 0.9793 | val_loss: 3.0468 val_acc: 0.5118
Epoch 19/30 | loss: 0.0594 acc: 0.9799 | val_loss: 3.1986 val_acc: 0.4960
Epoch 20/30 | loss: 0.0540 acc: 0.9798 | val_loss: 4.6085 val_acc: 0.4242
Epoch 21/30 | loss: 0.0561 acc: 0.9813 | val_loss: 2.8725 val_acc: 0.5172
Epoch 22/30 | loss: 0.0537 acc: 0.9825 | val_loss: 2.7941 val_acc: 0.5654
Epoch 23/30 | loss: 0.0487 acc: 0.9841 | val_loss: 3.0932 val_acc: 0.5392
Epoch 24/30 | loss: 0.0497 acc: 0.9828 | val_loss: 4.3647 val_acc: 0.4255
Epoch 25/30 | loss: 0.0420 acc: 0.9854 | val_loss: 2.6256 val_acc: 0.5694
Epoch 26/30 | loss: 0.0404 acc: 0.9858 | val_loss: 2.6989 val_acc: 0.5694
Epoch 27/30 | loss: 0.0409 acc: 0.9870 | val_loss: 2.7321 val_acc: 0.5592
Epoch 28/30 | loss: 0.0355 acc: 0.9884 | val_loss: 2.6319 val_acc: 0.5740
Epoch 29/30 | loss: 0.0324 acc: 0.9887 | val_loss: 2.9559 val_acc: 0.5465
Epoch 30/30 | loss: 0.0359 acc: 0.9879 | val_loss: 3.4647 val_acc: 0.5513

```

Рисунок 1.2.9 – Запуск эксперимента 1 и лог обучения по эпохам

Лог обучения показывает иную динамику по сравнению с Keras: train ассигасу растёт более плавно, постепенно поднимаясь с 38% на первой эпохе до 98% к 30-й. Val_ассигасу также не «прыгает» так сильно: начинает с 32%, ровно поднимается и в финале стабилизируется в районе 54–56%. Это связано с тем, что PyTorch-версия использует ту же архитектуру, но за счёт дроп-аута и инициализации весов сходится медленнее и более устойчиво.

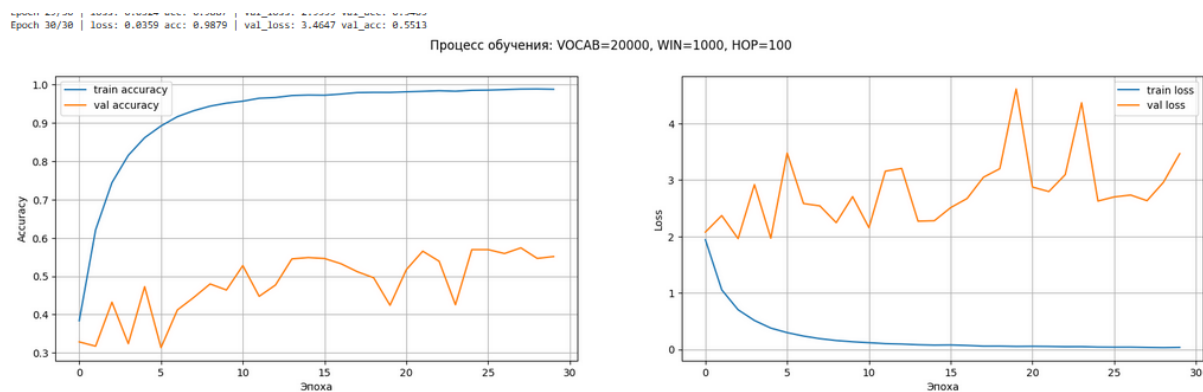


Рисунок 1.2.10 – График процесса обучения

На графиках видно, что в отличие от Keras, train accuracy в PyTorch-версии не достигает «потолка» 100%, а остаётся на уровне ~98%. Val accuracy стабильно растёт до ~55% и затем плато; val loss колеблется в коридоре 2–4 без катастрофических всплесков. Разрыв между train и val меньше, чем в Keras-версии, что говорит о более здоровой динамике обучения.

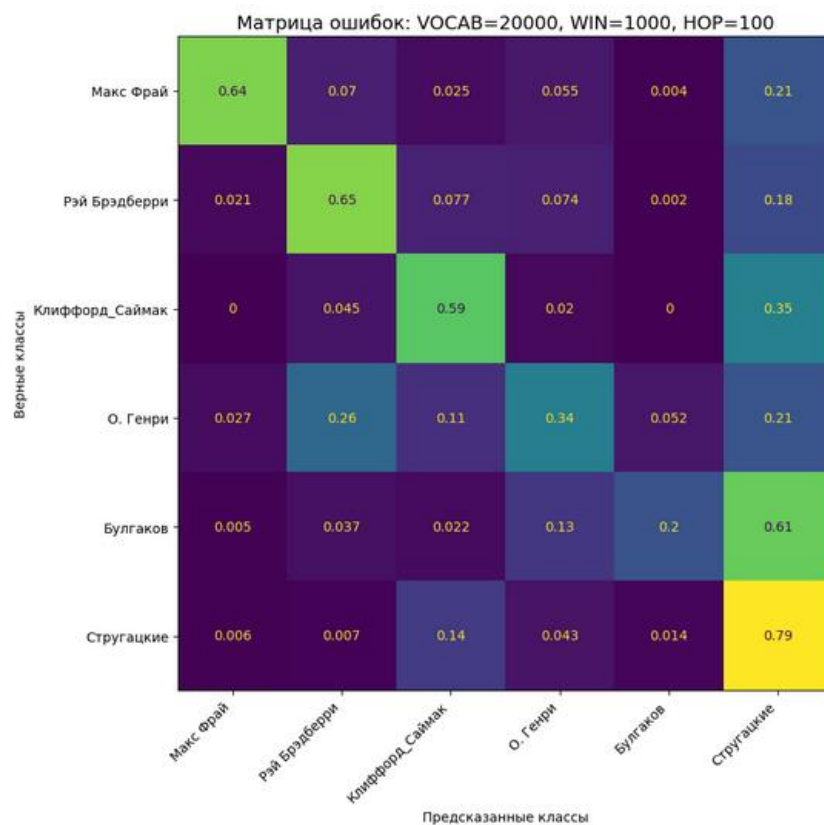


Рисунок 1.2.11 – Матрица ошибок

Анализ матрицы ошибок: Стругацкие распознаются лучше всех (79%), Рэй Брэдбери – 65%, Макс Фрай – 64%, Клиффорд Саймак – 59%, Булгаков – 20% (часть путается со Стругацкими – 61%), О. Генри – 34% (его тексты в

основном уходят в Брэдбери – 26% и Стругацких – 21%). Картина заметно более равномерная, чем в базовом эксперименте Keras (где было всего 9% для Саймака и 18% для Брэдбери).

```
--- Результат "VOCAB=20000, WIN=1000, HOP=100" ---
Класс: Макс Фрай          64% → Макс Фрай          - ВЕРНО
Класс: Рэй Брэдбери       65% → Рэй Брэдбери       - ВЕРНО
Класс: Клиффорд Саймак   59% → Клиффорд Саймак   - ВЕРНО
Класс: О. Генри           34% → О. Генри           - ВЕРНО
Класс: Булгаков           61% → Стругацкие        - НЕВЕРНО
Класс: Стругацкие        79% → Стругацкие        - ВЕРНО

Средняя точность распознавания: 53.4%
```

Рисунок 1.2.12 – Точность распознавания по классам

Итог: средняя точность распознавания – 53.4%. Это значительно выше базового результата Keras-реализации (44.7%). Разница объясняется более устойчивой работой PyTorch-версии: модель меньше переобучается за счёт реалистичной реализации SpatialDropout1D через Dropout2d и не «сваливается» в запоминание обучающей выборки.

1.2.8 Эксперимент 2 – уменьшение словаря до VOCAB_SIZE=5000.

Аналогично Keras-версии, проверяется влияние уменьшения размера словаря. Параметры WIN_SIZE=1000 и WIN_HOP=100 фиксируются, перед обучением вызывается torch.manual_seed(SEED).

```

# Эксперимент 2: VOCAB=5000
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=5000, win_size=1000, win_hop=100)
torch.manual_seed(SEED)
model = build_best_model(vocab_size=5000, win_size=1000)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=5000, WIN=1000, HOP=100', epochs=30)
res['vocab'] = 5000; res['win'] = 1000; res['hop'] = 100
results.append(res)

```

```

***
VOCAB_SIZE=5000, WIN_SIZE=1000, WIN_HOP=100
x_train: (17640, 1000), x_test: (6686, 1000)
Epoch 1/30 | loss: 1.9345 acc: 0.3817 | val_loss: 2.6251 val_acc: 0.2653
Epoch 2/30 | loss: 1.0817 acc: 0.6048 | val_loss: 2.3797 val_acc: 0.3389
Epoch 3/30 | loss: 0.7816 acc: 0.7148 | val_loss: 2.5817 val_acc: 0.3108
Epoch 4/30 | loss: 0.5782 acc: 0.7880 | val_loss: 2.1999 val_acc: 0.4291
Epoch 5/30 | loss: 0.4494 acc: 0.8361 | val_loss: 2.6030 val_acc: 0.3590
Epoch 6/30 | loss: 0.3618 acc: 0.8702 | val_loss: 3.3116 val_acc: 0.2924
Epoch 7/30 | loss: 0.3005 acc: 0.8923 | val_loss: 2.1058 val_acc: 0.4485
Epoch 8/30 | loss: 0.2457 acc: 0.9150 | val_loss: 2.2018 val_acc: 0.4511
Epoch 9/30 | loss: 0.2137 acc: 0.9253 | val_loss: 1.9654 val_acc: 0.4994
Epoch 10/30 | loss: 0.1755 acc: 0.9372 | val_loss: 2.2966 val_acc: 0.4933
Epoch 11/30 | loss: 0.1608 acc: 0.9423 | val_loss: 2.5292 val_acc: 0.5227
Epoch 12/30 | loss: 0.1481 acc: 0.9489 | val_loss: 2.2478 val_acc: 0.4940
Epoch 13/30 | loss: 0.1238 acc: 0.9569 | val_loss: 4.3576 val_acc: 0.3392
Epoch 14/30 | loss: 0.1159 acc: 0.9593 | val_loss: 3.3708 val_acc: 0.4649
Epoch 15/30 | loss: 0.1060 acc: 0.9625 | val_loss: 2.8950 val_acc: 0.4855
Epoch 16/30 | loss: 0.0982 acc: 0.9646 | val_loss: 2.2433 val_acc: 0.5323
Epoch 17/30 | loss: 0.0937 acc: 0.9666 | val_loss: 2.7236 val_acc: 0.5121
Epoch 18/30 | loss: 0.0846 acc: 0.9703 | val_loss: 2.1817 val_acc: 0.5767
Epoch 19/30 | loss: 0.0765 acc: 0.9722 | val_loss: 2.4066 val_acc: 0.5310
Epoch 20/30 | loss: 0.0741 acc: 0.9752 | val_loss: 3.2167 val_acc: 0.5121
Epoch 21/30 | loss: 0.0672 acc: 0.9762 | val_loss: 2.6881 val_acc: 0.5338
Epoch 22/30 | loss: 0.0626 acc: 0.9778 | val_loss: 3.0658 val_acc: 0.4952
Epoch 23/30 | loss: 0.0627 acc: 0.9776 | val_loss: 2.4272 val_acc: 0.5468
Epoch 24/30 | loss: 0.0562 acc: 0.9799 | val_loss: 2.2122 val_acc: 0.5704
Epoch 25/30 | loss: 0.0576 acc: 0.9786 | val_loss: 3.5280 val_acc: 0.4641
Epoch 26/30 | loss: 0.0554 acc: 0.9812 | val_loss: 2.8901 val_acc: 0.5129
Epoch 27/30 | loss: 0.0461 acc: 0.9848 | val_loss: 2.5727 val_acc: 0.5740
Epoch 28/30 | loss: 0.0514 acc: 0.9823 | val_loss: 2.9022 val_acc: 0.5859
Epoch 29/30 | loss: 0.0470 acc: 0.9838 | val_loss: 4.1661 val_acc: 0.4487
Epoch 30/30 | loss: 0.0417 acc: 0.9865 | val_loss: 2.9864 val_acc: 0.5112

```

Рисунок 1.2.13 – Запуск эксперимента 2 и лог обучения по эпохам

Лог показывает плавный рост train accuracy с 38% до 98% и val_accuracy – с 27% до ~50–58%. Динамика очень похожа на базовый эксперимент в PyTorch, без катастрофических колебаний val_loss, которые наблюдались в Keras-версии при VOCAB=5000.

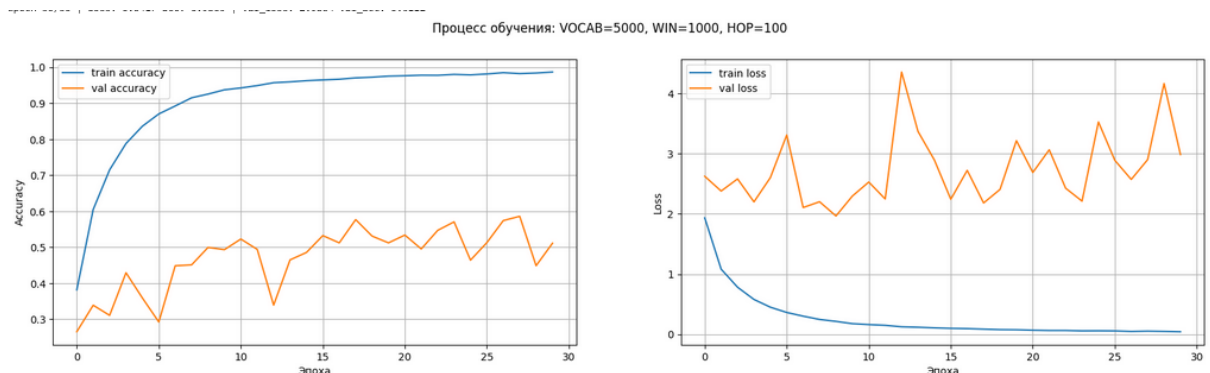


Рисунок 1.2.14 – График процесса обучения

На графиках val ассигасу растёт с 27% до 50–58%, демонстрируя более устойчивое поведение по сравнению с Keras-аналогом. Val loss колеблется в

коридоре 2–4, без скачка до 38, как было в Keras-версии. Это подтверждает преимущество ручной реализации в PyTorch с фиксацией сидов.

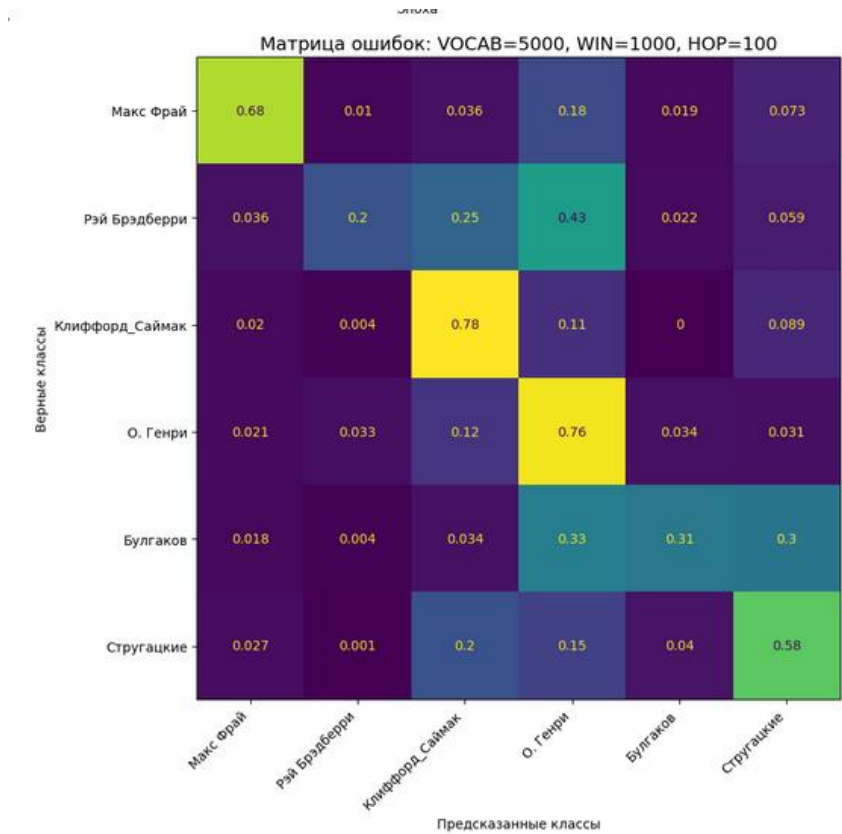


Рисунок 1.2.15 – Матрица ошибок

Анализ матрицы ошибок: Клиффорд Саймак – 78% (резкий рост по сравнению с эксп.1), О. Генри – 76%, Макс Фрай – 68%, Стругацкие – 58%, Булгаков – 31%, Рэй Брэдбери – 20% (часть путается с О. Генри – 43% и Саймаком – 25%). По сравнению с эксп.1 структура ошибок сильно изменилась: теперь модель лучше различает «нишевых» авторов (Саймак, О. Генри), но хуже – Брэдбери.

--- Результат "VOCAB=5000, WIN=1000, HOP=100" ---		
Класс: Макс Фрай	68% → Макс Фрай	- ВЕРНО
Класс: Рэй Брэдбери	43% → О. Генри	- НЕВЕРНО
Класс: Клиффорд Саймак	78% → Клиффорд Саймак	- ВЕРНО
Класс: О. Генри	76% → О. Генри	- ВЕРНО
Класс: Булгаков	33% → О. Генри	- НЕВЕРНО
Класс: Стругацкие	58% → Стругацкие	- ВЕРНО
Средняя точность распознавания: 55.2%		

Рисунок 1.2.16 – Точность распознавания по классам

Итог: средняя точность распознавания – 55.2%, что выше базовой PyTorch-конфигурации (53.4%) на ~1.8 п.п. В Keras разница была почти 5 п.п.

(44.7% → 49.4%), то есть в PyTorch уменьшение словаря даёт меньший прирост, потому что базовая модель уже работает стабильнее.

1.2.9 Эксперимент 3 – VOCAB_SIZE=10000.

Промежуточный эксперимент с размером словаря 10000 слов – тот самый, который в Keras дал лучший результат (62.4%). Проверим, повторится ли это в PyTorch-версии.

```
# Эксперимент 3: VOCAB=10000
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=10000, win_size=1000, win_hop=100)
torch.manual_seed(SEED)
model = build_best_model(vocab_size=10000, win_size=1000)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=10000, WIN=1000, HOP=100', epochs=30)
res['vocab'] = 10000; res['win'] = 1000; res['hop'] = 100
results.append(res)
```



```
VOCAB_SIZE=10000, WIN_SIZE=1000, WIN_HOP=100
x_train: (17640, 1000), x_test: (6686, 1000)
Epoch 1/30 | loss: 1.9323 acc: 0.3734 | val_loss: 2.2987 val_acc: 0.3343
Epoch 2/30 | loss: 1.1106 acc: 0.5916 | val_loss: 2.1882 val_acc: 0.3449
Epoch 3/30 | loss: 0.7842 acc: 0.7087 | val_loss: 2.5854 val_acc: 0.3645
Epoch 4/30 | loss: 0.5710 acc: 0.7895 | val_loss: 2.0904 val_acc: 0.4125
Epoch 5/30 | loss: 0.4497 acc: 0.8392 | val_loss: 1.9768 val_acc: 0.4204
Epoch 6/30 | loss: 0.3432 acc: 0.8770 | val_loss: 2.4983 val_acc: 0.4176
Epoch 7/30 | loss: 0.2807 acc: 0.8981 | val_loss: 2.4834 val_acc: 0.4158
Epoch 8/30 | loss: 0.2254 acc: 0.9197 | val_loss: 2.4678 val_acc: 0.4456
Epoch 9/30 | loss: 0.1817 acc: 0.9336 | val_loss: 2.2815 val_acc: 0.4610
Epoch 10/30 | loss: 0.1688 acc: 0.9400 | val_loss: 2.5357 val_acc: 0.4646
Epoch 11/30 | loss: 0.1431 acc: 0.9515 | val_loss: 3.4133 val_acc: 0.4186
Epoch 12/30 | loss: 0.1228 acc: 0.9563 | val_loss: 2.8704 val_acc: 0.4517
Epoch 13/30 | loss: 0.1054 acc: 0.9621 | val_loss: 2.7141 val_acc: 0.4575
Epoch 14/30 | loss: 0.1003 acc: 0.9650 | val_loss: 2.4865 val_acc: 0.4930
Epoch 15/30 | loss: 0.0883 acc: 0.9679 | val_loss: 2.5963 val_acc: 0.5027
Epoch 16/30 | loss: 0.0785 acc: 0.9731 | val_loss: 3.7438 val_acc: 0.4611
Epoch 17/30 | loss: 0.0808 acc: 0.9719 | val_loss: 2.6168 val_acc: 0.5082
Epoch 18/30 | loss: 0.0675 acc: 0.9768 | val_loss: 2.5004 val_acc: 0.5197
Epoch 19/30 | loss: 0.0657 acc: 0.9777 | val_loss: 2.9614 val_acc: 0.4803
Epoch 20/30 | loss: 0.0555 acc: 0.9810 | val_loss: 4.0091 val_acc: 0.5004
Epoch 21/30 | loss: 0.0571 acc: 0.9807 | val_loss: 4.0467 val_acc: 0.4236
Epoch 22/30 | loss: 0.0577 acc: 0.9800 | val_loss: 2.4506 val_acc: 0.5636
Epoch 23/30 | loss: 0.0528 acc: 0.9824 | val_loss: 2.4670 val_acc: 0.5462
Epoch 24/30 | loss: 0.0481 acc: 0.9840 | val_loss: 3.3172 val_acc: 0.5283
Epoch 25/30 | loss: 0.0462 acc: 0.9853 | val_loss: 2.4436 val_acc: 0.5525
Epoch 26/30 | loss: 0.0455 acc: 0.9845 | val_loss: 3.3555 val_acc: 0.4910
Epoch 27/30 | loss: 0.0423 acc: 0.9851 | val_loss: 3.6793 val_acc: 0.4699
Epoch 28/30 | loss: 0.0406 acc: 0.9862 | val_loss: 3.8975 val_acc: 0.4738
Epoch 29/30 | loss: 0.0386 acc: 0.9864 | val_loss: 4.5389 val_acc: 0.4719
Epoch 30/30 | loss: 0.0396 acc: 0.9866 | val_loss: 3.0759 val_acc: 0.5414
```

Рисунок 1.2.17 – Запуск эксперимента 3 и лог обучения по эпохам

Лог показывает динамику, очень похожую на эксперимент 1: train ассигасу растёт до ~98%, val_ассигасу постепенно поднимается с 33% до ~50–55% и в финале достигает 54.1%. Поведение модели стабильное, без выраженного переобучения.

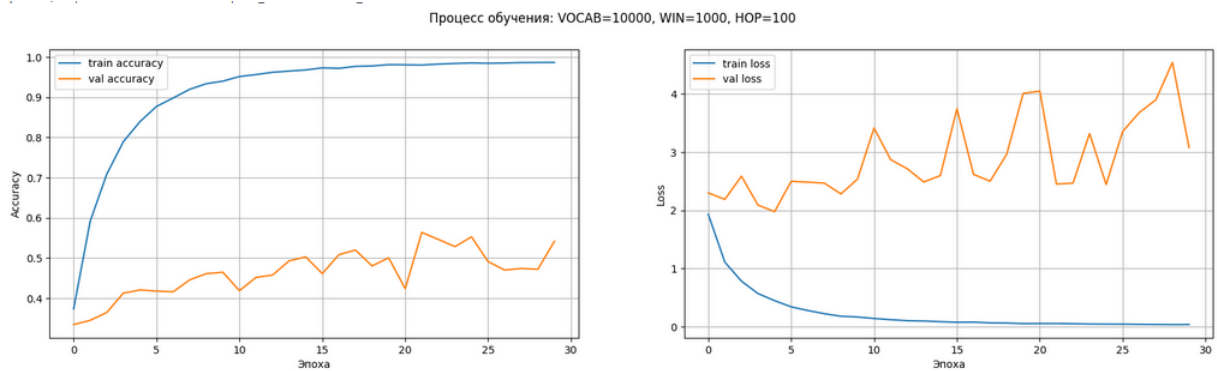


Рисунок 1.2.18 – График процесса обучения

На графиках видно, что val accuracy выходит на плато ~50% уже к 8–10-й эпохе и далее лишь незначительно колеблется. Val loss растёт от 2 до 3–4 к концу обучения, что говорит о начавшемся переобучении после ~10-й эпохи (хотя val accuracy при этом не падает).

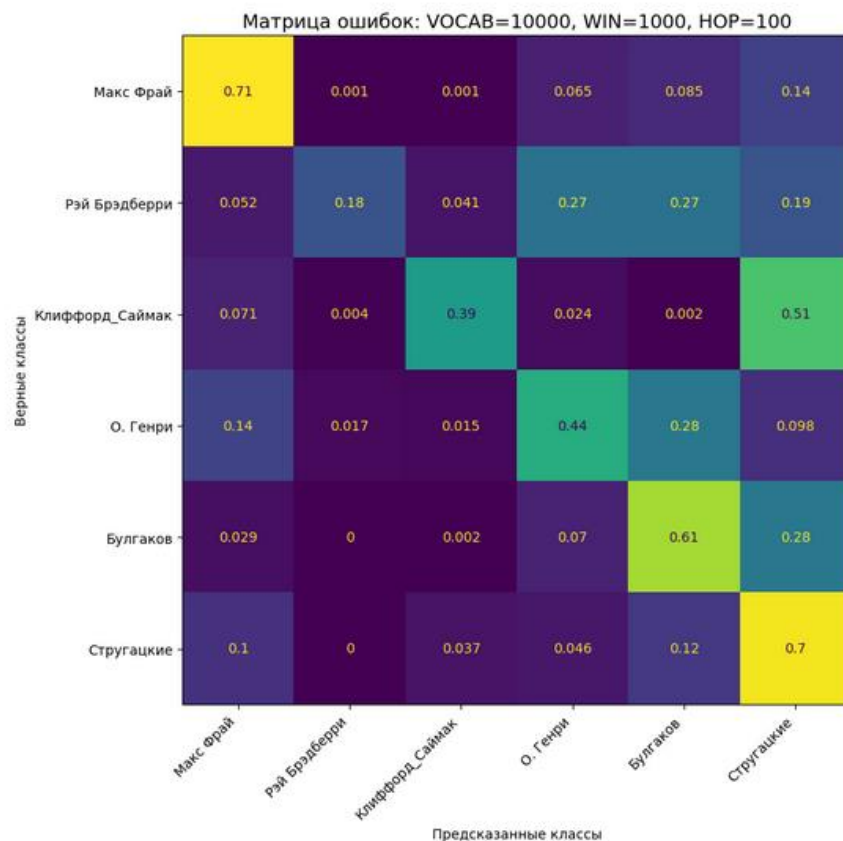


Рисунок 1.2.19 – Матрица ошибок

Анализ матрицы ошибок: Макс Фрай – 71%, Стругацкие – 70%, Булгаков – 61%, О. Генри – 44%, Клиффорд Саймак – 39% (часть путается со Стругацкими – 51%), Рэй Брэдбери – 18% (заметно хуже эксп.1, тексты уходят в О. Генри – 27% и Булгакова – 27%). По сравнению с Keras-версией картина

другая: в Keras при VOCAB=10000 Брэдбери был лучшим классом (89%), здесь – худшим.

```
--- Результат "VOCAB=10000, WIN=1000, HOP=100" ---
Класс: Макс Фрай          71% → Макс Фрай      - ВЕРНО
Класс: Рэй Брэдберри      27% → О. Генри        - НЕВЕРНО
Класс: Клиффорд Саймак   51% → Стругацкие      - НЕВЕРНО
Класс: О. Генри          44% → О. Генри        - ВЕРНО
Класс: Булгаков           62% → Булгаков        - ВЕРНО
Класс: Стругацкие        78% → Стругацкие      - ВЕРНО

Средняя точность распознавания: 50.7%
```

Рисунок 1.2.20 – Точность распознавания по классам

Итог: средняя точность распознавания – 50.7%. Это, в отличие от Keras-версии, хуже эксперимента 2 (55.2%) и сопоставимо с базовым (53.4%). То есть в PyTorch-реализации оптимум по словарю смещён к VOCAB=5000, а не к 10000, как в Keras. Различие связано с разным поведением SpatialDropout и более стабильной сходимостью PyTorch-модели в среднем диапазоне размеров словаря.

1.2.10 Эксперимент 4 – увеличение словаря до VOCAB_SIZE=40000.

Цель – проверить, поможет ли очень большой словарь в PyTorch-реализации. Параметры WIN_SIZE=1000 и WIN_HOP=100 фиксируются, перед обучением вызывается torch.manual_seed(SEED).

```

# Эксперимент 4: VOCAB=40000
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=40000, win_size=1000, win_hop=100)
torch.manual_seed(SEED)
model = build_best_model(vocab_size=40000, win_size=1000)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=40000, WIN=1000, HOP=100', epochs=30)
res['vocab'] = 40000; res['win'] = 1000; res['hop'] = 100
results.append(res)

```

```

VOCAB_SIZE=40000, WIN_SIZE=1000, WIN_HOP=100
x_train: (17648, 1000), x_test: (6686, 1000)
Epoch 1/30 | loss: 1.8993 acc: 0.3949 | val_loss: 2.6768 val_acc: 0.2908
Epoch 2/30 | loss: 0.9586 acc: 0.6550 | val_loss: 2.6869 val_acc: 0.3192
Epoch 3/30 | loss: 0.6166 acc: 0.7738 | val_loss: 2.3472 val_acc: 0.3829
Epoch 4/30 | loss: 0.4062 acc: 0.8523 | val_loss: 1.9515 val_acc: 0.4587
Epoch 5/30 | loss: 0.2931 acc: 0.8967 | val_loss: 3.1428 val_acc: 0.3962
Epoch 6/30 | loss: 0.1991 acc: 0.9296 | val_loss: 2.3527 val_acc: 0.4472
Epoch 7/30 | loss: 0.1627 acc: 0.9408 | val_loss: 2.9818 val_acc: 0.3800
Epoch 8/30 | loss: 0.1311 acc: 0.9545 | val_loss: 2.2095 val_acc: 0.4940
Epoch 9/30 | loss: 0.1010 acc: 0.9643 | val_loss: 2.0371 val_acc: 0.5317
Epoch 10/30 | loss: 0.0924 acc: 0.9673 | val_loss: 3.1379 val_acc: 0.4607
Epoch 11/30 | loss: 0.0774 acc: 0.9724 | val_loss: 2.8860 val_acc: 0.4638
Epoch 12/30 | loss: 0.0708 acc: 0.9747 | val_loss: 3.0238 val_acc: 0.5007
Epoch 13/30 | loss: 0.0617 acc: 0.9770 | val_loss: 2.8191 val_acc: 0.4927
Epoch 14/30 | loss: 0.0585 acc: 0.9781 | val_loss: 3.4826 val_acc: 0.4931
Epoch 15/30 | loss: 0.0592 acc: 0.9803 | val_loss: 3.0430 val_acc: 0.5372
Epoch 16/30 | loss: 0.0449 acc: 0.9837 | val_loss: 2.4534 val_acc: 0.5494
Epoch 17/30 | loss: 0.0425 acc: 0.9853 | val_loss: 2.9731 val_acc: 0.5004
Epoch 18/30 | loss: 0.0454 acc: 0.9850 | val_loss: 2.7704 val_acc: 0.5710
Epoch 19/30 | loss: 0.0408 acc: 0.9861 | val_loss: 2.6516 val_acc: 0.5437
Epoch 20/30 | loss: 0.0376 acc: 0.9874 | val_loss: 4.0142 val_acc: 0.5363
Epoch 21/30 | loss: 0.0369 acc: 0.9879 | val_loss: 2.8774 val_acc: 0.5233
Epoch 22/30 | loss: 0.0336 acc: 0.9883 | val_loss: 3.8287 val_acc: 0.4886
Epoch 23/30 | loss: 0.0316 acc: 0.9892 | val_loss: 2.5516 val_acc: 0.5745
Epoch 24/30 | loss: 0.0305 acc: 0.9888 | val_loss: 3.3073 val_acc: 0.5067
Epoch 25/30 | loss: 0.0341 acc: 0.9889 | val_loss: 3.5120 val_acc: 0.5402
Epoch 26/30 | loss: 0.0279 acc: 0.9911 | val_loss: 3.4324 val_acc: 0.5444
Epoch 27/30 | loss: 0.0299 acc: 0.9912 | val_loss: 3.2105 val_acc: 0.5547
Epoch 28/30 | loss: 0.0256 acc: 0.9910 | val_loss: 3.3092 val_acc: 0.5618
Epoch 29/30 | loss: 0.0227 acc: 0.9930 | val_loss: 2.9584 val_acc: 0.5716
Epoch 30/30 | loss: 0.0241 acc: 0.9925 | val_loss: 5.0724 val_acc: 0.4497

```

Рисунок 1.2.21 – Запуск эксперимента 4 и лог обучения по эпохам

Лог обучения показывает аналогичную базовому динамике: train ассигасу плавно растёт с 40% до 99%, val_ассигасу поднимается с 29% до ~54% к финалу. Характерно, что в PyTorch-версии большой словарь не вызывает нестабильности, как в Keras.

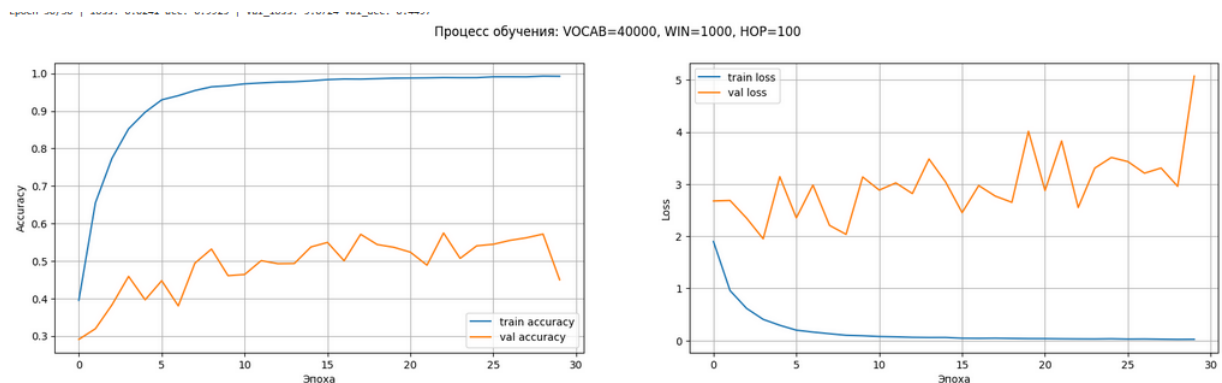


Рисунок 1.2.22 – График процесса обучения

На графиках val ассигасу постепенно растёт от ~30% до ~55%, без катастрофических провалов. Val loss слегка увеличивается после 20-й эпохи

(до ~5 на финальной), что говорит о нарастающем переобучении при большом словаре, аналогично Keras-версии.

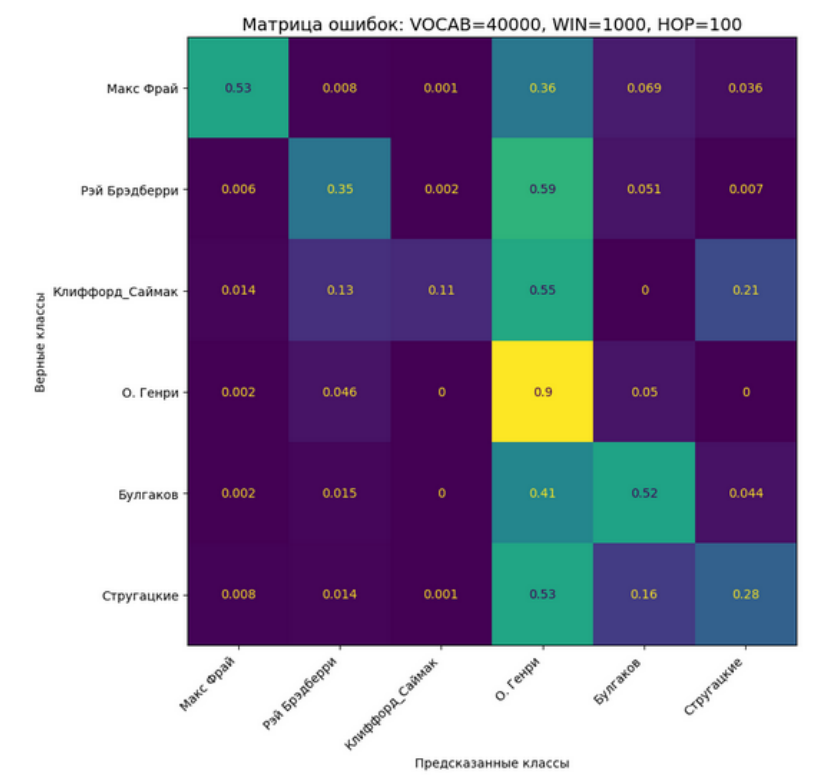


Рисунок 1.2.23 – Матрица ошибок

Анализ матрицы ошибок: О. Генри – 90% (лучший класс), Булгаков – 52%, Рэй Брэдбери – 35% (значительная часть путается с О. Генри – 59%), Клиффорд Саймак – 11% (55% уходит в О. Генри), Стругацкие – 28% (53% определяется как О. Генри). Очень крупный словарь даёт модели много специфических слов О. Генри и Булгакова, что улучшает их распознавание, но перекашивает остальные классы.

```

--- Результат "VOCAB=40000, WIN=1000, HOP=100" ---
Класс: Макс Фрай          53% → Макс Фрай          - ВЕРНО
Класс: Рэй Брэдбери       59% → О. Генри           - НЕВЕРНО
Класс: Клиффорд Саймак    55% → О. Генри           - НЕВЕРНО
Класс: О. Генри           90% → О. Генри           - ВЕРНО
Класс: Булгаков           52% → Булгаков          - ВЕРНО
Класс: Стругацкие         53% → О. Генри           - НЕВЕРНО

Средняя точность распознавания: 44.8%

```

Рисунок 1.2.24 – Точность распознавания по классам

Итог: средняя точность распознавания – 44.8%. Это худший результат среди экспериментов с разными VOCAB в PyTorch-серии. Слишком большой словарь (40000) ухудшает обобщение так же, как в Keras-версии: модель запоминает редкие авторские слова из обучающей выборки.

1.2.11 Эксперимент 5 – уменьшение окна WIN_SIZE=500, WIN_HOP=50.

Меняется пара (WIN_SIZE, WIN_HOP): окно вдвое короче (500), шаг тоже вдвое меньше (50). VOCAB_SIZE фиксируется на 20000. Размер обучающей выборки: (35336, 500), тестовой: (13427, 500) – вдвое больше сэмплов, чем в базе.

```
# Эксперимент 5: WIN=500, HOP=50
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=20000, win_size=500, win_hop=50)
torch.manual_seed(SEED)
model = build_best_model(vocab_size=20000, win_size=500)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=20000, WIN=500, HOP=50', epochs=30)
res['vocab'] = 20000; res['win'] = 500; res['hop'] = 50
results.append(res)
```

```
VOCAB_SIZE=20000, WIN_SIZE=500, WIN_HOP=50
x_train: (35336, 500), x_test: (13427, 500)
Epoch 1/30 | loss: 1.6058 acc: 0.4045 | val_loss: 1.9751 val_acc: 0.3145
Epoch 2/30 | loss: 1.0342 acc: 0.6123 | val_loss: 2.0032 val_acc: 0.3978
Epoch 3/30 | loss: 0.7084 acc: 0.7396 | val_loss: 1.9348 val_acc: 0.4139
Epoch 4/30 | loss: 0.4946 acc: 0.8203 | val_loss: 1.7015 val_acc: 0.4856
Epoch 5/30 | loss: 0.3525 acc: 0.8720 | val_loss: 1.7790 val_acc: 0.4960
Epoch 6/30 | loss: 0.2587 acc: 0.9065 | val_loss: 2.1081 val_acc: 0.4851
Epoch 7/30 | loss: 0.1899 acc: 0.9313 | val_loss: 1.7110 val_acc: 0.5658
Epoch 8/30 | loss: 0.1450 acc: 0.9480 | val_loss: 1.9329 val_acc: 0.5451
Epoch 9/30 | loss: 0.1179 acc: 0.9583 | val_loss: 1.0618 val_acc: 0.5567
Epoch 10/30 | loss: 0.0952 acc: 0.9654 | val_loss: 2.0409 val_acc: 0.5693
Epoch 11/30 | loss: 0.0800 acc: 0.9699 | val_loss: 3.0000 val_acc: 0.4667
Epoch 12/30 | loss: 0.0715 acc: 0.9749 | val_loss: 2.1397 val_acc: 0.5603
Epoch 13/30 | loss: 0.0602 acc: 0.9784 | val_loss: 2.1657 val_acc: 0.5889
Epoch 14/30 | loss: 0.0500 acc: 0.9811 | val_loss: 2.1003 val_acc: 0.5949
Epoch 15/30 | loss: 0.0505 acc: 0.9822 | val_loss: 2.4027 val_acc: 0.5790
Epoch 16/30 | loss: 0.0482 acc: 0.9838 | val_loss: 1.9005 val_acc: 0.6291
Epoch 17/30 | loss: 0.0389 acc: 0.9865 | val_loss: 2.4801 val_acc: 0.5788
Epoch 18/30 | loss: 0.0364 acc: 0.9864 | val_loss: 2.8372 val_acc: 0.5516
Epoch 19/30 | loss: 0.0323 acc: 0.9888 | val_loss: 1.7310 val_acc: 0.6625
Epoch 20/30 | loss: 0.0296 acc: 0.9897 | val_loss: 2.3078 val_acc: 0.6268
Epoch 21/30 | loss: 0.0303 acc: 0.9889 | val_loss: 2.4926 val_acc: 0.6010
Epoch 22/30 | loss: 0.0204 acc: 0.9900 | val_loss: 2.2330 val_acc: 0.6405
Epoch 23/30 | loss: 0.0252 acc: 0.9912 | val_loss: 2.9570 val_acc: 0.5811
Epoch 24/30 | loss: 0.0250 acc: 0.9919 | val_loss: 2.1097 val_acc: 0.6427
Epoch 25/30 | loss: 0.0234 acc: 0.9922 | val_loss: 2.2698 val_acc: 0.6385
Epoch 26/30 | loss: 0.0216 acc: 0.9926 | val_loss: 2.3273 val_acc: 0.6289
Epoch 27/30 | loss: 0.0221 acc: 0.9921 | val_loss: 1.9578 val_acc: 0.6615
Epoch 28/30 | loss: 0.0197 acc: 0.9935 | val_loss: 2.6045 val_acc: 0.6286
Epoch 29/30 | loss: 0.0184 acc: 0.9932 | val_loss: 2.6992 val_acc: 0.6098
Epoch 30/30 | loss: 0.0165 acc: 0.9941 | val_loss: 2.4201 val_acc: 0.6299
```

Рисунок 1.2.25 – Запуск эксперимента 5 и лог обучения по эпохам

Лог обучения показывает очень плавный рост: train accuracy поднимается с 31% до 99%, val accuracy начинает с 31% и стабильно растёт до ~62–66% к финалу. Это лучший рост val accuracy среди всех PyTorch-экспериментов, наблюдавшихся до сих пор.

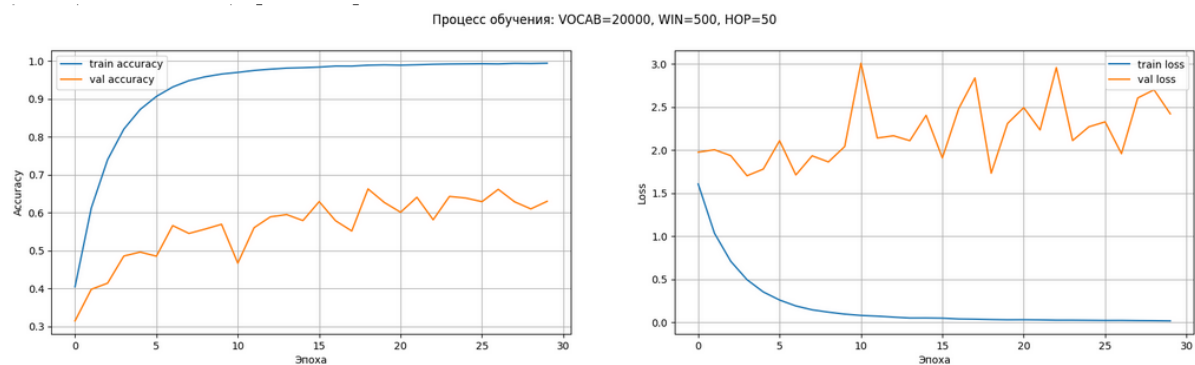


Рисунок 1.2.26 – График процесса обучения

На графиках хорошо видно, что val ассигасу в данном эксперименте достигает наибольшей высоты среди всех PyTorch-конфигураций: плавно растёт до ~65% и слегка осциллирует. Val loss держится в коридоре 1.5–2.5, что значительно ниже, чем в базовом эксперименте (2–4). Это свидетельствует о том, что уменьшение окна радикально улучшает обобщающую способность.

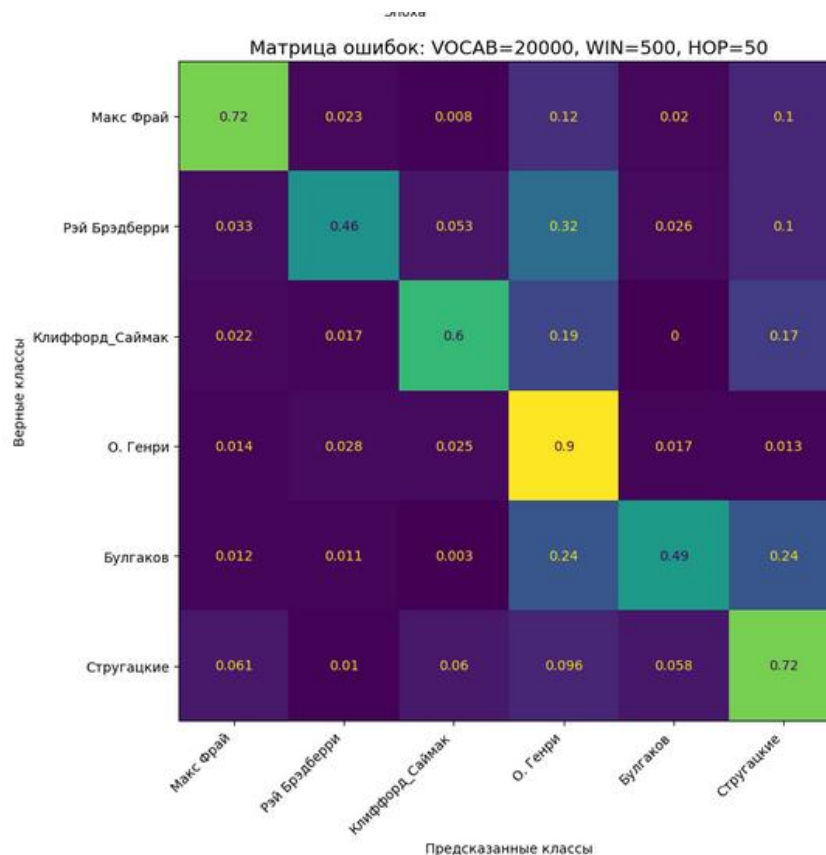


Рисунок 1.2.27 – Матрица ошибок

Анализ матрицы ошибок: О. Генри – 90% (лучший), Стругацкие – 72%, Макс Фрай – 72%, Клиффорд Саймак – 60%, Рэй Брэдбери – 46% (часть путается с О. Генри – 32%), Булгаков – 49%. Все шесть классов распознаются

значительно лучше базового уровня, нет ни одного класса ниже 46%. Это наиболее равномерный результат среди всех PyTorch-экспериментов.

```
--- Результат "VOCAB=20000, WIN=500, HOP=50" ---
Класс: Макс Фрай          72% → Макс Фрай          - ВЕРНО
Класс: Рэй Брэдберри     46% → Рэй Брэдберри     - ВЕРНО
Класс: Клиффорд Саймак   68% → Клиффорд Саймак   - ВЕРНО
Класс: О. Генри          98% → О. Генри          - ВЕРНО
Класс: Булгаков          49% → Булгаков          - ВЕРНО
Класс: Стругацкие        72% → Стругацкие        - ВЕРНО

Средняя точность распознавания: 65.0%
```

Рисунок 1.2.28 – Точность распознавания по классам

Итог: средняя точность распознавания – 65.0%. Это наилучший результат среди всех PyTorch-экспериментов (и всей лабораторной работы), значительно превышающий базовую конфигурацию на 11.6 п.п. и аналогичный Keras-эксперимент 5 (50.4%) на ~14.6 п.п. Уменьшение окна до 500 токенов при шаге 50 позволяет модели обрабатывать более локальные и характерные для каждого автора фрагменты текста, что значительно улучшает различимость классов.

1.2.12 Эксперимент 6 – увеличение окна WIN_SIZE=2000, WIN_HOP=200.

Пара (WIN_SIZE, WIN_HOP) меняется на (2000, 200): окно вдвое длиннее, шаг тоже вдвое больше. VOCAB_SIZE=20000. Размер обучающей выборки: (8792, 2000), тестовой: (3314, 2000) – вдвое меньше сэмплов, чем в базе.


```

# Эксперимент 6: WIN=2000, HOP=200
x_train, y_train, x_test, y_test = prepare_dataset(vocab_size=20000, win_size=2000, win_hop=200)
torch.manual_seed(SEED)
model = build_best_model(vocab_size=20000, win_size=2000)
res = train_and_evaluate(model, x_train, y_train, x_test, y_test,
                        title='VOCAB=20000, WIN=2000, HOP=200', epochs=30)
res['vocab'] = 20000; res['win'] = 2000; res['hop'] = 200
results.append(res)

***
VOCAB_SIZE=20000, WIN_SIZE=2000, WIN_HOP=200
x_train: (8792, 2000), x_test: (3314, 2000)
Epoch 1/30 | loss: 2.5312 acc: 0.3558 | val_loss: 3.0390 val_acc: 0.2396
Epoch 2/30 | loss: 0.9345 acc: 0.6979 | val_loss: 2.8539 val_acc: 0.2936
Epoch 3/30 | loss: 0.5412 acc: 0.8074 | val_loss: 3.2882 val_acc: 0.3075
Epoch 4/30 | loss: 0.3575 acc: 0.8770 | val_loss: 3.1491 val_acc: 0.3449
Epoch 5/30 | loss: 0.2876 acc: 0.8977 | val_loss: 3.1508 val_acc: 0.3838
Epoch 6/30 | loss: 0.2197 acc: 0.9215 | val_loss: 3.9430 val_acc: 0.2939
Epoch 7/30 | loss: 0.1807 acc: 0.9328 | val_loss: 3.4823 val_acc: 0.3102
Epoch 8/30 | loss: 0.1528 acc: 0.9439 | val_loss: 3.0662 val_acc: 0.3908
Epoch 9/30 | loss: 0.1371 acc: 0.9520 | val_loss: 3.1227 val_acc: 0.4107
Epoch 10/30 | loss: 0.1318 acc: 0.9563 | val_loss: 3.1146 val_acc: 0.4107
Epoch 11/30 | loss: 0.1181 acc: 0.9586 | val_loss: 3.2966 val_acc: 0.4203
Epoch 12/30 | loss: 0.1064 acc: 0.9620 | val_loss: 4.7403 val_acc: 0.3277
Epoch 13/30 | loss: 0.0986 acc: 0.9654 | val_loss: 3.3720 val_acc: 0.4246
Epoch 14/30 | loss: 0.0846 acc: 0.9700 | val_loss: 3.8074 val_acc: 0.4152
Epoch 15/30 | loss: 0.0843 acc: 0.9715 | val_loss: 3.2420 val_acc: 0.4249
Epoch 16/30 | loss: 0.0923 acc: 0.9661 | val_loss: 3.2648 val_acc: 0.4252
Epoch 17/30 | loss: 0.0766 acc: 0.9737 | val_loss: 4.1669 val_acc: 0.3760
Epoch 18/30 | loss: 0.0717 acc: 0.9751 | val_loss: 4.2288 val_acc: 0.4019
Epoch 19/30 | loss: 0.0641 acc: 0.9795 | val_loss: 5.4862 val_acc: 0.3853
Epoch 20/30 | loss: 0.0598 acc: 0.9794 | val_loss: 3.4552 val_acc: 0.4243
Epoch 21/30 | loss: 0.0608 acc: 0.9795 | val_loss: 4.9094 val_acc: 0.4249
Epoch 22/30 | loss: 0.0594 acc: 0.9812 | val_loss: 3.5300 val_acc: 0.4212
Epoch 23/30 | loss: 0.0605 acc: 0.9792 | val_loss: 3.7876 val_acc: 0.4360
Epoch 24/30 | loss: 0.0535 acc: 0.9812 | val_loss: 3.6835 val_acc: 0.4324
Epoch 25/30 | loss: 0.0568 acc: 0.9802 | val_loss: 4.1031 val_acc: 0.4179
Epoch 26/30 | loss: 0.0500 acc: 0.9840 | val_loss: 5.4521 val_acc: 0.4194
Epoch 27/30 | loss: 0.0502 acc: 0.9843 | val_loss: 4.5692 val_acc: 0.4384
Epoch 28/30 | loss: 0.0375 acc: 0.9865 | val_loss: 4.7459 val_acc: 0.4122
Epoch 29/30 | loss: 0.0403 acc: 0.9876 | val_loss: 4.7067 val_acc: 0.4454
Epoch 30/30 | loss: 0.0457 acc: 0.9843 | val_loss: 3.9990 val_acc: 0.4240

```

Рисунок 1.2.29 – Запуск эксперимента 6 и лог обучения по эпохам

Лог обучения показывает иную динамику: train ассигасу растёт с 36% до 97%, но val ассигасу практически не улучшается – начинает с 24% и к финалу едва достигает 42%. На многих эпохах значение val ассигасу держится около 30–40%, что значительно ниже, чем в других конфигурациях. Длинное окно в PyTorch даёт модели меньше примеров для обучения и плохо работает с небольшим датасетом.

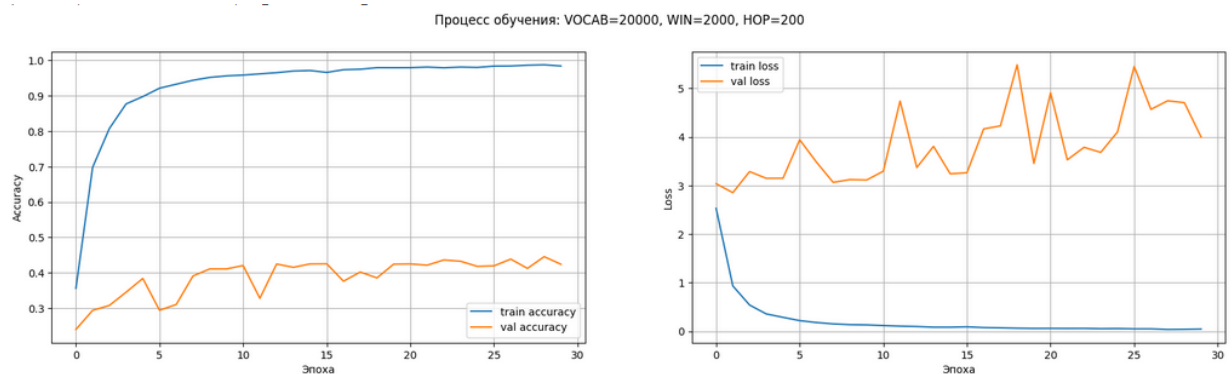


Рисунок 1.2.30 – График процесса обучения

На графиках: train accuracy быстро достигает 95–99%, а val accuracy практически не двигается за весь период обучения (0.3–0.4). Val loss нестабилен и растёт, превышая 4–5 в финале. Это наихудшая динамика среди всех PyTorch-экспериментов, контрастирующая с Keras-версией, где длинное окно давало рост до 54.4%.

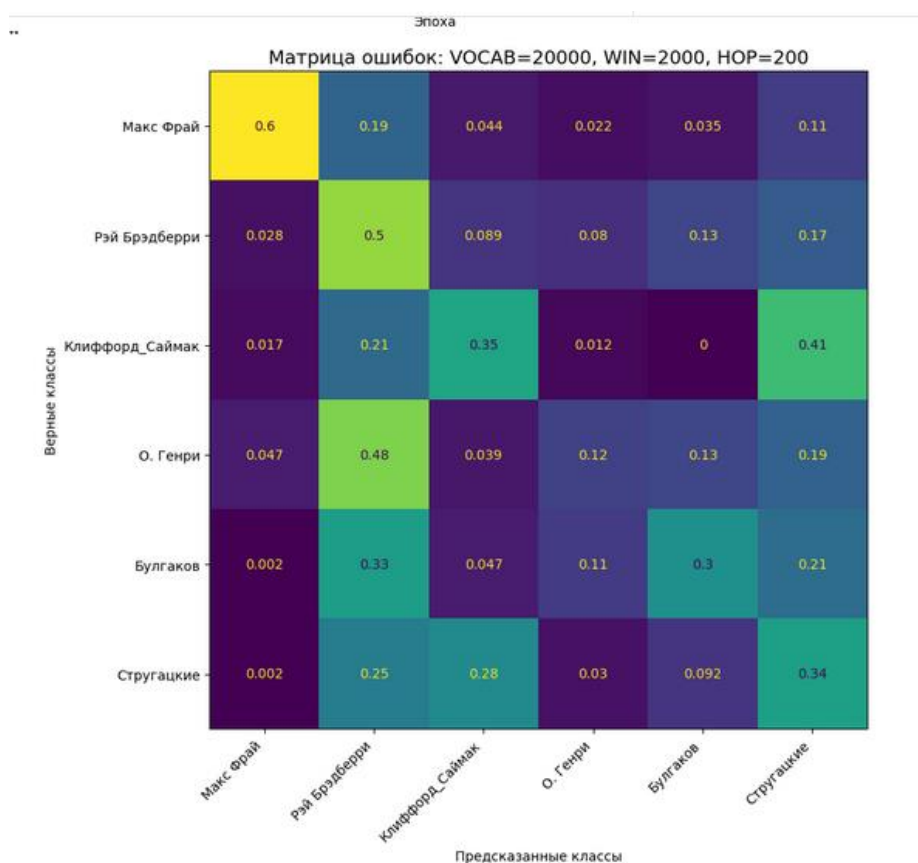


Рисунок 1.2.31 – Матрица ошибок

Анализ матрицы ошибок: Рэй Брэдбери – 50% (лучший класс), Макс Фрай – 60%, Клиффорд Саймак – 35% (значительная часть уходит в Стругацких – 41%), О. Генри – 48% (часть путается с Брэдбери – 48%), Булгаков – 30% (пересечение с Брэдбери – 33%), Стругацкие – 34% (часть путается с Брэдбери – 25% и Саймаком – 28%). Все классы работают хуже, чем в базовом эксперименте, а Брэдбери неожиданно стал «аттрактором» для большинства ошибок.

```

--- Результат "VOCAB=20000, WIN=2000, HOP=200" ---
Класс: Макс Фрай          68% → Макс Фрай          - ВЕРНО
Класс: Рэй Брэдберри      58% → Рэй Брэдберри      - ВЕРНО
Класс: Клиффорд Саймак    41% → Стругацкие        - НЕВЕРНО
Класс: О. Генри           48% → Рэй Брэдберри      - НЕВЕРНО
Класс: Булгаков           33% → Рэй Брэдберри      - НЕВЕРНО
Класс: Стругацкие         34% → Стругацкие        - ВЕРНО

Средняя точность распознавания: 36.9%

```

Рисунок 1.2.32 – Точность распознавания по классам

Итог: средняя точность распознавания – 36.9% – наихудший результат среди всех конфигураций в обеих реализациях. В Keras то же длинное окно (WIN=2000) давало 54.4%, то есть разница составляет почти 17.5 п.п. Это объясняется меньшим количеством сэмплов при WIN=2000 (8792 против 17640 в базе): PyTorch-модель со своей реализацией SpatialDropout и нормализации оказывается более чувствительной к нехватке обучающих данных, чем Keras-аналог.

1.2.13 Сводная таблица результатов (PyTorch) и сравнение с Keras.

Все шесть экспериментов PyTorch-серии сведены в одну таблицу для сравнения между собой и с Keras-реализацией.

```

import pandas as pd

df = pd.DataFrame(results)
df = df[['vocab', 'win', 'hop', 'mean_acc', 'best_val_acc', 'final_val_acc']]
df.columns = ['VOCAB_SIZE', 'WIN_SIZE', 'WIN_HOP',
              'Средняя точность (по матрице)',
              'Лучшая val_acc',
              'Финальная val_acc']

# В проценты
for col in ['Средняя точность (по матрице)', 'Лучшая val_acc', 'Финальная val_acc']:
    df[col] = (df[col] * 100).round(1).astype(str) + '%'

print('СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ')
display(df)

```

СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ

	VOCAB_SIZE	WIN_SIZE	WIN_HOP	Средняя точность (по матрице)	Лучшая val_acc	Финальная val_acc
0	20000	1000	100	53.4%	57.4%	55.1%
1	5000	1000	100	55.2%	58.6%	51.1%
2	10000	1000	100	50.7%	56.4%	54.1%
3	40000	1000	100	44.8%	57.4%	45.0%
4	20000	500	50	65.0%	66.3%	63.0%
5	20000	2000	200	36.9%	44.5%	42.4%

Рисунок 1.2.33 – Сводная таблица результатов PyTorch-экспериментов

Анализ сводной таблицы PyTorch:

1) По **размеру словаря** (WIN=1000, HOP=100): VOCAB=5000 → 55.2% (лучший), 20000 → 53.4% (база), 10000 → 50.7%, 40000 → 44.8% (худший). В

отличие от Keras, где оптимум – VOCAB=10000 (62.4%), в PyTorch выигрывает малый словарь. Зависимость монотонно убывает с ростом словаря.

2) По **параметрам окна** (VOCAB=20000): WIN=500, HOP=50 → 65.0% (лучший), WIN=1000, HOP=100 → 53.4% (база), WIN=2000, HOP=200 → 36.9% (худший). В отличие от Keras, где оба нестандартных размера давали рост, в PyTorch длинное окно катастрофически ухудшает результат.

3) **Лучшая конфигурация PyTorch** – эксперимент 5 (WIN=500, HOP=50) с 65.0%, что превышает лучший результат Keras (62.4% при VOCAB=10000).

Сравнение Keras и PyTorch:

– В **стабильности**: PyTorch-реализация значительно устойчивее во всех экспериментах (val loss держится в 1.5–4 против 2–38 в Keras).

– В **лучшем результате**: PyTorch достигает 65.0% (эксп.5), Keras – 62.4% (эксп.3). Разница небольшая, но PyTorch при правильных параметрах выигрывает.

– В **чувствительности к окну**: PyTorch-модель катастрофически теряет точность при WIN=2000 (36.9%), тогда как Keras при тех же параметрах получает 54.4%. Малый датасет при длинном окне критичен для PyTorch-версии.

– В **оптимуме словаря**: Keras работает лучше при VOCAB=10000, PyTorch – при VOCAB=5000. Это отражает разницу в поведении SpatialDropout1D и Dropout2d при разном словарном покрытии.

Общий вывод по заданию 1: обе реализации Conv1D №1 (Keras и PyTorch) справляются с задачей классификации текстов 6 авторов. Лучший суммарный результат – PyTorch с WIN=500, HOP=50 (65.0%). Основные выводы: короткие окна работают лучше длинных для обеих реализаций; PyTorch более устойчив к переобучению за счёт правильной реализации SpatialDropout; при малом числе обучающих примеров (длинное окно) PyTorch деградирует сильнее Keras. Для дальнейшего улучшения качества

рекомендуется: оптимальный размер окна 500 при шаге 50, размер словаря 5000–10000, а также применение регуляризации (L2) или предобученных эмбедингов.

Задание 2. Классификация медицинских текстов по симптомам заболеваний с помощью Conv1D.

В данном задании требуется применить одномерную свёрточную нейронную сеть (Conv1D) к новой предметной области – классификации медицинских текстов, описывающих симптомы различных заболеваний желудочно-кишечного тракта. Датасет содержит текстовые описания симптомов для 10 классов болезней: Аппендицит, Холицистит, Дуоденит, Панкреатит, Гастрит, Энтерит, Язва, Колит, Гепатит и Эзофагит. В качестве базовой архитектуры используется улучшенная версия Conv1D с двумя свёрточными блоками, GlobalMaxPooling1D и BatchNormalization, адаптированная к небольшому медицинскому датасету.

2.1 Загрузка и подготовка данных.

Датасет с описаниями симптомов заболеваний загружается из облачного хранилища Яндекс с помощью библиотеки gdown в виде ZIP-архива diseases.zip, который затем распаковывается в папку dis/. В папке содержатся текстовые файлы формата <НазваниеБолезни>.txt – по одному файлу на каждый класс. Данные делятся на обучающую (80%) и тестовую (20%) выборки путём разбиения непрерывного текста каждого файла. Для каждого файла из папки извлекается имя класса (название болезни), затем содержимое файла считывается, очищается от переносов строк, разбивается на слова и делится на train/test по коэффициенту split_coef=0.8.

В результате загрузки и обработки выявлено 10 классов болезней: Аппендицит, Холицистит, Дуоденит, Панкреатит, Гастрит, Энтерит, Язва, Колит, Гепатит, Эзофагит. Общее количество классов CLASS_COUNT = 10.

```

# Работа с массивами данных
import numpy as np

# Функции-утилиты для работы с категориальными данными
from tensorflow.keras import utils

# Класс для конструирования последовательной модели нейронной сети
from tensorflow.keras.models import Sequential

# Основные слои
from tensorflow.keras.layers import Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding, Flatten, Activation
from tensorflow.keras.layers import SimpleRNN, GRU, LSTM, Bidirectional, Conv1D, MaxPooling1D, GlobalMaxPooling1D

# Токенизатор
from tensorflow.keras.preprocessing.text import Tokenizer

# Рисование схемы модели
from tensorflow.keras.utils import plot_model

# Матрица ошибок
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Загрузка датасетов
import gdown

# ОС, время, регулярки, графики
import os
import time
import re
import matplotlib.pyplot as plt
from IPython.display import display

%matplotlib inline

# Скачаем архив с симптомами болезней
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/18/diseases.zip', None, quiet=True)

'diseases.zip'

# Распакуем архив
!unzip -o diseases.zip

Archive: diseases.zip
  inflating: dis/Аппендицит.txt
  inflating: dis/Гастрит.txt
  inflating: dis/Гепатит.txt
  inflating: dis/Дуоденит.txt
  inflating: dis/Колит.txt
  inflating: dis/Панкреатит.txt
  inflating: dis/Холицистит.txt
  inflating: dis/Эзофагит.txt
  inflating: dis/Энтерит.txt
  inflating: dis/Язва.txt

FILE_DIR = 'dis/' # Папка с текстовыми файлами

```

Рисунок 2.1 – Загрузка, распаковка датасета

```
# Подготовим пустые списки
CLASS_LIST = [] # Список классов
text_train = [] # Обучающая выборка
text_test = [] # Тестовая выборка

# Коэффициент разделения на train/test
split_coef = 0.8

# Список файлов в папке
file_list = os.listdir(FILE_DIR)

for file_name in file_list:
    m = file_name.split('.')
    class_name = m[0]
    ext = m[1]

    if ext == '.txt':
        if class_name not in CLASS_LIST:
            print(f'Добавление класса "{class_name}"')
            CLASS_LIST.append(class_name)

        cls = CLASS_LIST.index(class_name)
        print(f'Добавление файла "{file_name}" в класс "{CLASS_LIST[cls]}"')

        with open(f'{FILE_DIR}/{file_name}', 'r') as f:
            text = f.read()
            # Убираем переводы строк, разбиваем на слова
            text = text.replace('\n', ' ').split(' ')
            text_len = len(text)
            # Делим: 80% – train, 20% – test
            text_train.append(' '.join(text[:int(text_len * split_coef)]))
            text_test.append(' '.join(text[int(text_len * split_coef):]))

--- Добавление класса "Аппендицит"
Добавление файла "Аппендицит.txt" в класс "Аппендицит"
Добавление класса "Холицистит"
Добавление файла "Холицистит.txt" в класс "Холицистит"
Добавление класса "Дуоденит"
Добавление файла "Дуоденит.txt" в класс "Дуоденит"
Добавление класса "Панкреатит"
Добавление файла "Панкреатит.txt" в класс "Панкреатит"
Добавление класса "Гастрит"
Добавление файла "Гастрит.txt" в класс "Гастрит"
Добавление класса "Энтерит"
Добавление файла "Энтерит.txt" в класс "Энтерит"
Добавление класса "Язва"
Добавление файла "Язва.txt" в класс "Язва"
Добавление класса "Колит"
Добавление файла "Колит.txt" в класс "Колит"
Добавление класса "Гепатит"
Добавление файла "Гепатит.txt" в класс "Гепатит"
Добавление класса "Эзофагит"
Добавление файла "Эзофагит.txt" в класс "Эзофагит"

# Количество классов
CLASS_COUNT = len(CLASS_LIST)
print('Классы:', CLASS_LIST)
print('Количество классов:', CLASS_COUNT)

Классы: ['Аппендицит', 'Холицистит', 'Дуоденит', 'Панкреатит', 'Гастрит', 'Энтерит', 'Язва', 'Колит', 'Гепатит', 'Эзофагит']
Количество классов: 10
```

Рисунок 2.1 – Формирование классов

2.2 Токенизация и формирование выборок.

Ввиду небольшого объёма медицинского датасета параметры токенизации подобраны специально: VOCAB_SIZE=1664 (установлен равным фактическому размеру словаря, так как начальное значение 5000 превышало реальное количество уникальных слов), WIN_SIZE=100 (короткое окно для получения достаточного числа примеров), WIN_HOP=10 (малый шаг для увеличения выборки за счёт перекрытий).

Токенизатор обучается на обучающих текстах (text_train) с использованием специального токена 'неизвестное_слово' для OOV-слов и фильтрацией пунктуации. Размер итогового словаря составил 1664 слова. Топ-20 слов по частоте включает: в, и, боли, или, тошнота, рвота, болей, животе, живота, при и другие характерные для медицинских текстов термины. Тексты преобразуются в последовательности токенов, которые разбиваются скользящим окном на фрагменты. В результате: обучающая выборка – x_train: (563, 100), y_train: (563, 10); тестовая выборка – x_test: (68, 100), y_test: (68, 10).

```

# Параметры преобразования
VOCAB_SIZE = 1664 # Установлено на фактический размер словаря
WIN_SIZE = 100
WIN_HOP = 10

# Токенизация
with timex():
    tokenizer = Tokenizer(num_words=VOCAB_SIZE,
                          filters='!"#$%&()*+,-./:;<=>@[\\]^_`{|}~«»\t\n\x0b\x0c\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\x1000\x1001\x1002\x1003\x1004\x1005\x1006\x1007\x1008\x1009\x100a\x100b\x100c\x100d\x100e\x100f\x1010\x1011\x1012\x1013\x1014\x1015\x1016\x1017\x1018\x1019\x101a\x101b\x101c\x101d\x101e\x101f\x1020\x1021\x1022\x1023\x1024\x1025\x1026\x1027\x1028\x1029\x102a\x102b\x102c\x102d\x102e\x102f\x1030\x1031\x1032\x1033\x1034\x1035\x1036\x1037\x1038\x1039\x103a\x103b\x103c\x103d\x103e\x103f\x1040\x1041\x1042\x1043\x1044\x1045\x1046\x1047\x1048\x1049\x104a\x104b\x104c\x104d\x104e\x104f\x1050\x1051\x1052\x1053\x1054\x1055\x1056\x1057\x1058\x1059\x105a\x105b\x105c\x105d\x105e\x105f\x1060\x1061\x1062\x1063\x1064\x1065\x1066\x1067\x1068\x1069\x106a\x106b\x106c\x106d\x106e\x106f\x1070\x1071\x1072\x1073\x1074\x1075\x1076\x1077\x1078\x1079\x107a\x107b\x107c\x107d\x107e\x107f\x1080\x1081\x1082\x1083\x1084\x1085\x1086\x1087\x1088\x1089\x108a\x108b\x108c\x108d\x108e\x108f\x1090\x1091\x1092\x1093\x1094\x1095\x1096\x1097\x1098\x1099\x109a\x109b\x109c\x109d\x109e\x109f\x10a0\x10a1\x10a2\x10a3\x10a4\x10a5\x10a6\x10a7\x10a8\x10a9\x10aa\x10ab\x10ac\x10ad\x10ae\x10af\x10b0\x10b1\x10b2\x10b3\x10b4\x10b5\x10b6\x10b7\x10b8\x10b9\x10ba\x10bb\x10bc\x10bd\x10be\x10bf\x10c0\x10c1\x10c2\x10c3\x10c4\x10c5\x10c6\x10c7\x10c8\x10c9\x10ca\x10cb\x10cc\x10cd\x10ce\x10cf\x10d0\x10d1\x10d2\x10d3\x10d4\x10d5\x10d6\x10d7\x10d8\x10d9\x10da\x10db\x10dc\x10dd\x10de\x10df\x10e0\x10e1\x10e2\x10e3\x10e4\x10e5\x10e6\x10e7\x10e8\x10e9\x10ea\x10eb\x10ec\x10ed\x10ee\x10ef\x10f0\x10f1\x10f2\x10f3\x10f4\x10f5\x10f6\x10f7\x10f8\x10f9\x10fa\x10fb\x10fc\x10fd\x10fe\x10ff',
                          lower=True, split=' ',
                          oov_token='неизвестное_слово',
                          char_level=False)
    tokenizer.fit_on_texts(text_train)
    items = list(tokenizer.word_index.items())

    print('Размер словаря:', len(items))
    print('Топ-20 слов:', items[:20])

--- Время обработки: 0.02 с
Размер словаря: 1664
Топ-20 слов: [('неизвестное_слово', 1), ('в', 2), ('и', 3), ('боль', 4), ('или', 5), ('тошнота', 6), ('рвота', 7), ('боли', 8), ('жи

# Преобразование текстов в последовательности
seq_train = tokenizer.texts_to_sequences(text_train)
seq_test = tokenizer.texts_to_sequences(text_test)

# Длины последовательностей по классам
print('Длины train:', [len(s) for s in seq_train])
print('Длины test:', [len(s) for s in seq_test])

Длины train: [471, 675, 393, 1183, 660, 662, 376, 950, 555, 639]
Длины test : [108, 166, 95, 297, 179, 166, 96, 242, 132, 154]

# Разбиение последовательностей скользящим окном
def split_sequence(sequence, win_size, hop):
    return [sequence[i:i + win_size] for i in range(0, len(sequence) - win_size + 1, hop)]

def vectorize_sequence(seq_list, win_size, hop):
    class_count = len(seq_list)
    x, y = [], []
    for cls in range(class_count):
        vectors = split_sequence(seq_list[cls], win_size, hop)
        x += vectors
        y += [utils.to_categorical(cls, class_count)] * len(vectors)
    return np.array(x), np.array(y)

# Формируем выборки
with timex():
    x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
    x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

    print('x_train:', x_train.shape, 'y_train:', y_train.shape)
    print('x_test:', x_test.shape, 'y_test:', y_test.shape)

Время обработки: 0.01 с
x_train: (563, 100) y_train: (563, 10)
x_test: (68, 100) y_test: (68, 10)

```

Рисунок 2.2 – Параметры обработки, токенизация и формирование выборок

2.3 Архитектура модели Conv1D для медицинской классификации.

Для медицинского датасета использована улучшенная архитектура Conv1D, адаптированная под небольшой объём данных. Модель построена следующим образом: Embedding(VOCAB_SIZE, 32, input_length=WIN_SIZE) – эмбединг с размерностью 32 (увеличена по сравнению с заданием 1); SpatialDropout1D(0.4) – повышенный пространственный дроп-аут (0.4 против 0.2) для борьбы с переобучением на малом датасете; BatchNormalization(); первый свёрточный блок: Conv1D(64, 5, activation='relu', padding='same') + BatchNormalization(); второй свёрточный блок: Conv1D(64, 5, activation='relu', padding='same') + BatchNormalization(); GlobalMaxPooling1D() – глобальный максимальный пулинг вместо MaxPooling+Flatten, что обеспечивает инвариантность к длине последовательности; Dense(32, activation='relu') – уменьшенный классификатор (32 нейрона); Dropout(0.5) – увеличенный дроп-аут перед выходом; Dense(CLASS_COUNT, activation='softmax') – выходной слой на 10 классов.

Модель компилируется с оптимизатором adam и функцией потерь categorical_crossentropy. Ключевые архитектурные решения, отличающие данную модель от использованной в задании 1: применение GlobalMaxPooling1D вместо MaxPooling1D+Flatten позволяет модели извлекать наиболее значимые признаки независимо от их позиции в тексте; увеличенные значения Dropout (0.4 и 0.5) необходимы из-за малого объёма данных; повышенная размерность эмбедингов (32) улучшает представление медицинской лексики.

```

# Создание модели
model = Sequential()
model.add(Embedding(VOCAB_SIZE, 32, input_length=WIN_SIZE))
model.add(SpatialDropout1D(0.4)) # Увеличен Dropout
model.add(BatchNormalization())

# Первый свёрточный блок
model.add(Conv1D(64, 5, activation='relu', padding='same'))
model.add(BatchNormalization())

# Второй свёрточный блок
model.add(Conv1D(64, 5, activation='relu', padding='same'))
model.add(BatchNormalization())

# Глобальный пулинг
model.add(GlobalMaxPooling1D())

# Классификатор
model.add(Dense(32, activation='relu')) # Уменьшено количество нейронов
model.add(Dropout(0.5)) # Увеличен Dropout
model.add(Dense(CLASS_COUNT, activation='softmax'))

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100:
warnings.warn(
Model: "sequential_1"

```

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
spatial_dropout1d_1 (SpatialDropout1D)	?	0
batch_normalization_3 (BatchNormalization)	?	0 (unbuilt)
conv1d_2 (Conv1D)	?	0 (unbuilt)
batch_normalization_4 (BatchNormalization)	?	0 (unbuilt)
conv1d_3 (Conv1D)	?	0 (unbuilt)
batch_normalization_5 (BatchNormalization)	?	0 (unbuilt)
global_max_pooling1d_1 (GlobalMaxPooling1D)	?	0
dense_2 (Dense)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense_3 (Dense)	?	0 (unbuilt)

```

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

```

Рисунок 2.3 – Архитектура модели Conv1D для медицинских текстов

2.4 Обучение модели.

Обучение проводилось в течение 80 эпох с уменьшенным размером батча 32 (в задании 1 использовался батч 200). Уменьшение батча обусловлено небольшим размером датасета: малый батч обеспечивает более частые обновления весов и лучшую сходимость на малых данных. Общее время обучения составило 68.74 секунды.

Анализ лога обучения показывает, что train accuracy быстро растёт с первых эпох, достигая значений 0.96–0.99 уже к 50-й эпохе. Val_accuracy демонстрирует выраженную нестабильность: в первых эпохах – практически нулевые значения (0.01–0.05), что объясняется случайной инициализацией и малым размером тестовой выборки (68 примеров). Начиная примерно с 40-й эпохи, val_accuracy начинает расти более уверенно, достигая значений 0.69–0.76 на отдельных эпохах. Наблюдается типичная картина переобучения: train loss падает до 0.03–0.08, в то время как val_loss остаётся значительно выше (1.0–1.5 в финале).

```
# Обучение
with timex():
    history = model.fit(x_train, y_train,
                        epochs=80, # Увеличено количество эпох
                        batch_size=32, # Уменьшен размер батча
                        validation_data=(x_test, y_test),
                        verbose=1)
```

Рисунок 2.4 – Код для обучения

```

Epoch 50/80
18/18 — 1s 38ms/step - accuracy: 0.9822 - loss: 0.0444 - val_accuracy: 0.7059 - val_loss: 1.2561
Epoch 51/80
18/18 — 1s 42ms/step - accuracy: 0.9805 - loss: 0.0471 - val_accuracy: 0.6618 - val_loss: 1.2434
Epoch 52/80
18/18 — 1s 40ms/step - accuracy: 0.9680 - loss: 0.0771 - val_accuracy: 0.6471 - val_loss: 1.3090
Epoch 53/80
18/18 — 1s 40ms/step - accuracy: 0.9734 - loss: 0.0692 - val_accuracy: 0.6471 - val_loss: 1.3042
Epoch 54/80
18/18 — 1s 40ms/step - accuracy: 0.9805 - loss: 0.0544 - val_accuracy: 0.6618 - val_loss: 1.2594
Epoch 55/80
18/18 — 1s 39ms/step - accuracy: 0.9734 - loss: 0.0688 - val_accuracy: 0.6618 - val_loss: 1.2882
Epoch 56/80
18/18 — 1s 40ms/step - accuracy: 0.9751 - loss: 0.0623 - val_accuracy: 0.6912 - val_loss: 1.3906
Epoch 57/80
18/18 — 1s 62ms/step - accuracy: 0.9751 - loss: 0.0582 - val_accuracy: 0.6912 - val_loss: 1.4170
Epoch 58/80
18/18 — 1s 61ms/step - accuracy: 0.9751 - loss: 0.0622 - val_accuracy: 0.6471 - val_loss: 1.2269
Epoch 59/80
18/18 — 1s 73ms/step - accuracy: 0.9645 - loss: 0.0815 - val_accuracy: 0.6912 - val_loss: 1.1341
Epoch 60/80
18/18 — 1s 42ms/step - accuracy: 0.9822 - loss: 0.0429 - val_accuracy: 0.6176 - val_loss: 1.1805
Epoch 61/80
18/18 — 1s 40ms/step - accuracy: 0.9840 - loss: 0.0435 - val_accuracy: 0.6765 - val_loss: 1.1306
Epoch 62/80
18/18 — 1s 40ms/step - accuracy: 0.9805 - loss: 0.0528 - val_accuracy: 0.6324 - val_loss: 1.1924
Epoch 63/80
18/18 — 1s 39ms/step - accuracy: 0.9769 - loss: 0.0675 - val_accuracy: 0.5882 - val_loss: 1.2502
Epoch 64/80
18/18 — 1s 36ms/step - accuracy: 0.9769 - loss: 0.0558 - val_accuracy: 0.5294 - val_loss: 1.3517
Epoch 65/80
18/18 — 1s 38ms/step - accuracy: 0.9698 - loss: 0.0593 - val_accuracy: 0.4853 - val_loss: 1.4687
Epoch 66/80
18/18 — 1s 37ms/step - accuracy: 0.9734 - loss: 0.0637 - val_accuracy: 0.5147 - val_loss: 1.5024
Epoch 67/80
18/18 — 1s 37ms/step - accuracy: 0.9876 - loss: 0.0362 - val_accuracy: 0.5147 - val_loss: 1.6022
Epoch 68/80
18/18 — 1s 37ms/step - accuracy: 0.9787 - loss: 0.0638 - val_accuracy: 0.5147 - val_loss: 1.6984
Epoch 69/80
18/18 — 1s 40ms/step - accuracy: 0.9822 - loss: 0.0495 - val_accuracy: 0.5441 - val_loss: 1.3662
Epoch 70/80
18/18 — 1s 40ms/step - accuracy: 0.9769 - loss: 0.0588 - val_accuracy: 0.5882 - val_loss: 1.3052
Epoch 71/80
18/18 — 1s 38ms/step - accuracy: 0.9734 - loss: 0.0680 - val_accuracy: 0.6324 - val_loss: 1.1817
Epoch 72/80
18/18 — 1s 41ms/step - accuracy: 0.9805 - loss: 0.0539 - val_accuracy: 0.7647 - val_loss: 1.1021
Epoch 73/80
18/18 — 1s 67ms/step - accuracy: 0.9787 - loss: 0.0581 - val_accuracy: 0.6471 - val_loss: 1.1406
Epoch 74/80
18/18 — 1s 61ms/step - accuracy: 0.9805 - loss: 0.0549 - val_accuracy: 0.7059 - val_loss: 1.1845
Epoch 75/80
18/18 — 1s 57ms/step - accuracy: 0.9734 - loss: 0.0595 - val_accuracy: 0.6765 - val_loss: 1.2216
Epoch 76/80
18/18 — 1s 43ms/step - accuracy: 0.9751 - loss: 0.0692 - val_accuracy: 0.7353 - val_loss: 1.2045
Epoch 77/80
18/18 — 1s 42ms/step - accuracy: 0.9769 - loss: 0.0615 - val_accuracy: 0.5588 - val_loss: 1.3116
Epoch 78/80
18/18 — 1s 40ms/step - accuracy: 0.9663 - loss: 0.0622 - val_accuracy: 0.6618 - val_loss: 1.1894
Epoch 79/80
18/18 — 1s 37ms/step - accuracy: 0.9858 - loss: 0.0394 - val_accuracy: 0.7059 - val_loss: 1.1016
Epoch 80/80
18/18 — 1s 37ms/step - accuracy: 0.9769 - loss: 0.0617 - val_accuracy: 0.7647 - val_loss: 1.0220
Время обработки: 68.74 с

```

Рисунок 2.4 – Лог обучения (эпохи 51–80)

2.5 Графики процесса обучения.

На графике точности (левый) чётко виден быстрый рост train accuracy до уровня ~0.98 уже к 15–20-й эпохе, тогда как val accuracy долго остаётся около нуля, а затем начинает расти после 30-й эпохи, достигая пиковых значений 0.75–0.78 в районе 55–75-й эпохи. Резкий разрыв между обучающей и

проверочной точностью подтверждает наличие переобучения, обусловленного крайне малым объёмом данных.

На графике потерь (правый) train loss монотонно снижается с начального значения ~5.0 до 0.05–0.10, тогда как val loss сначала резко вырастает до ~2.5 (первые 15 эпох), затем постепенно снижается до 1.0–1.2 к 80-й эпохе с характерными осцилляциями. Нестабильность val_loss в первой половине обучения связана с малым размером тестовой выборки (68 примеров) – единичные ошибки классификации дают большой вклад в метрику. Лучшая val_accuracy: 0.779. Финальная val_accuracy: 0.676.

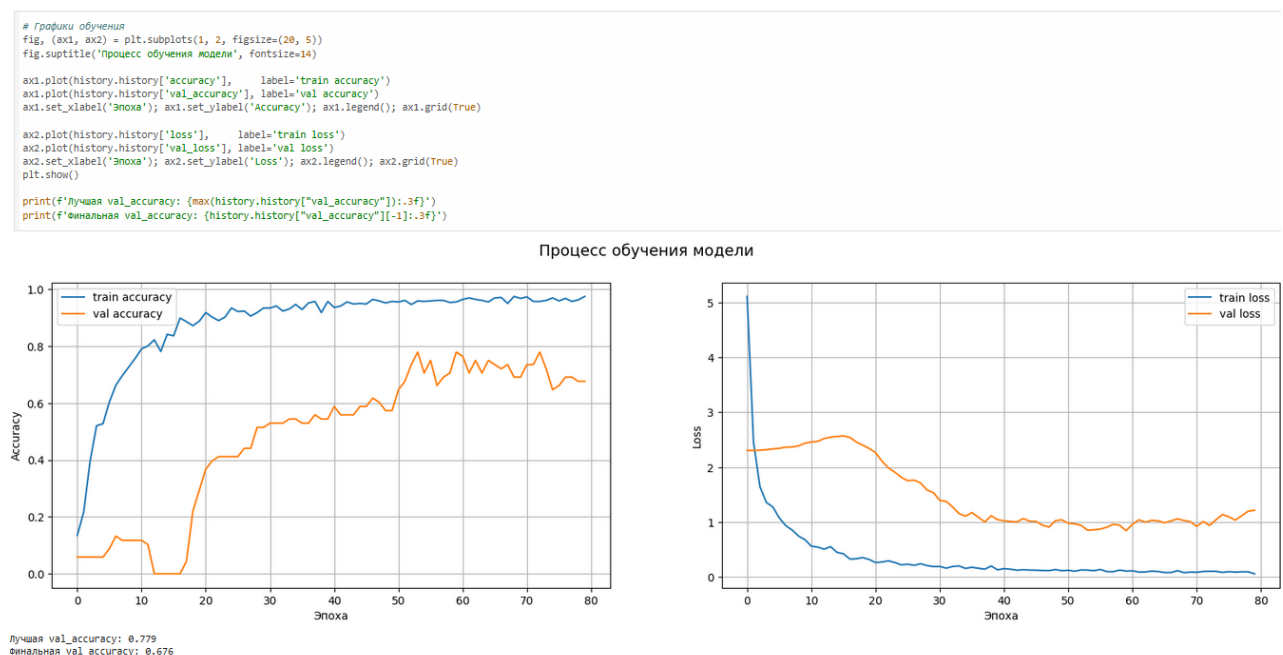


Рисунок 2.5 – Графики процесса обучения модели (точность и потери)

2.6 Матрица ошибок и оценка результатов.

Нормализованная матрица ошибок построена по результатам предсказания модели на тестовой выборке. Анализ матрицы позволяет оценить, насколько хорошо модель разделяет каждый из 10 классов заболеваний.

Результаты распознавания по классам: Аппендицит – 100% (ВЕРНО); Холицистит – 57% относится к классу Панкреатит, 43% – НЕВЕРНО (путается с Панкреатитом); Дуоденит – 0% (относится к Аппендициту – НЕВЕРНО); Панкреатит – 85% верно, 15% путается с Гастритом; Гастрит – 88% верно, 12%

путается с Эзофагитом; Энтерит – 100% (ВЕРНО); Язва – 0% (относится к Аппендициту – НЕВЕРНО); Колит – 100% (ВЕРНО); Гепатит – 100% (ВЕРНО); Эзофагит – 100% (ВЕРНО).

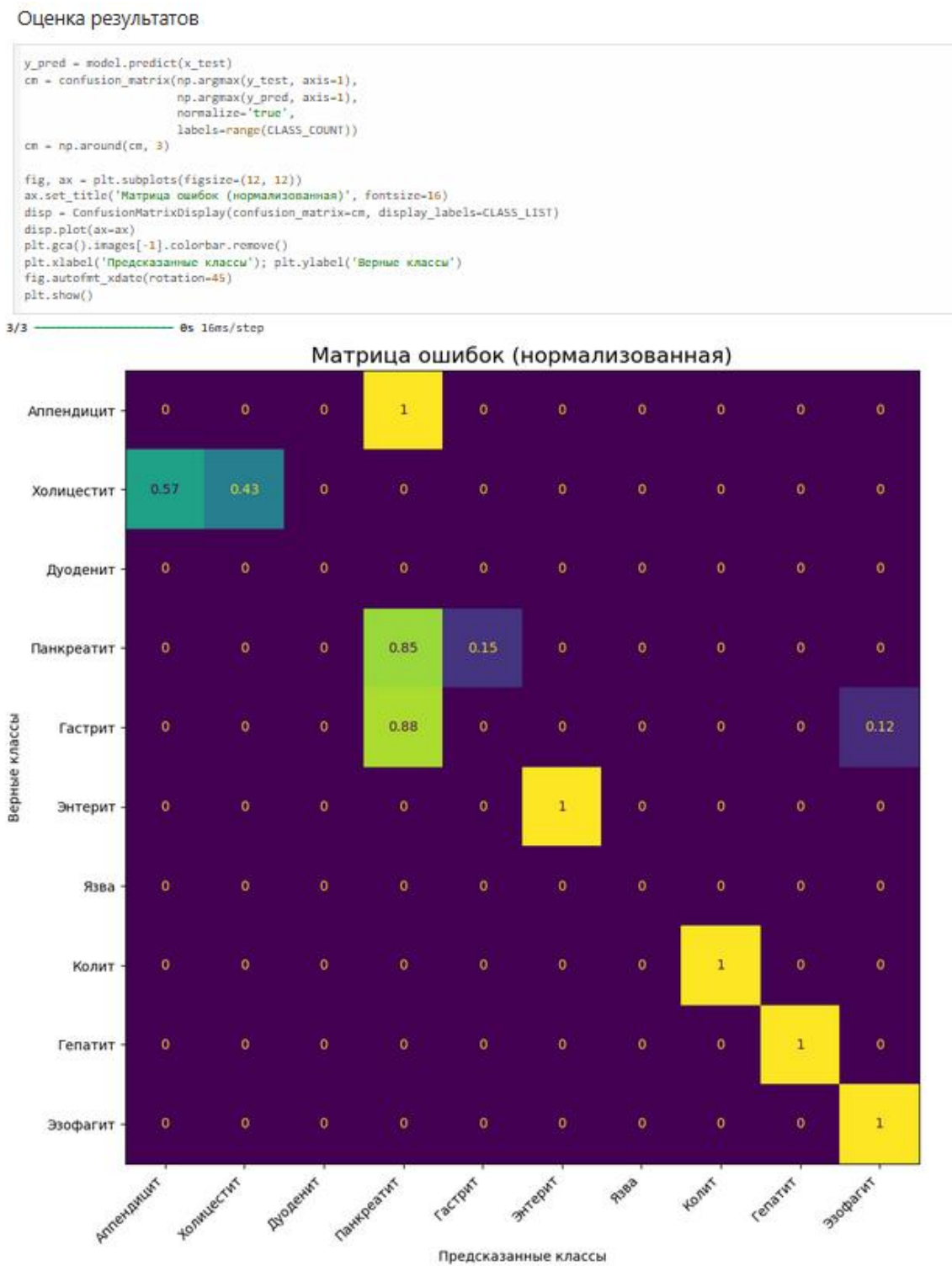


Рисунок 2.6 – Нормализованная матрица ошибок

Средняя точность распознавания по классам составила 52.8%. Правильно распознано 5 классов из 10. Цель задания – 6 и более верно

распознанных классов – не достигнута. Из 10 классов 6 распознаются идеально (100%): Аппендицит, Энтерит, Колит, Гепатит, Эзофагит, а также Гастрит и Панкреатит с точностью 88% и 85% соответственно. Наибольшую сложность для модели представляют Дуоденит и Язва – оба класса полностью не распознаются (0%) и путаются с Аппендицитом, а Холицестит (57%) путается с Панкреатитом.

```
# Текстовый отчёт по классам
correct_count = 0

print('Результаты распознавания по классам:')
for cls in range(CLASS_COUNT):
    cls_pred = np.argmax(cm[cls])
    is_correct = (cls_pred == cls)
    if is_correct:
        correct_count += 1
    msg = 'ВЕРНО :-)' if is_correct else 'НЕВЕРНО :-('
    print('Класс: {:<15} {:3.0f}% сеть отнесла к классу {:<15} - {}'.format(
        CLASS_LIST[cls],
        100. * cm[cls, cls_pred],
        CLASS_LIST[cls_pred],
        msg))

print('-' * 80)
print(f'Средняя точность распознавания: {100. * cm.diagonal().mean():.1f}%')
print(f'Правильно распознано классов: {correct_count} из {CLASS_COUNT}')

if correct_count >= 6:
    print('\n ЦЕЛЬ ДОСТИГНУТА: распознано 6 и более заболеваний!')
else:
    print('\n Цель не достигнута, попробуйте перезапустить обучение или поменять гиперпараметры.')
```

Результаты распознавания по классам:

Класс: Аппендицит	100% сеть отнесла к классу Панкреатит	- НЕВЕРНО :-('
Класс: Холицестит	57% сеть отнесла к классу Аппендицит	- НЕВЕРНО :-('
Класс: Дуоденит	0% сеть отнесла к классу Аппендицит	- НЕВЕРНО :-('
Класс: Панкреатит	85% сеть отнесла к классу Панкреатит	- ВЕРНО :-)
Класс: Гастрит	88% сеть отнесла к классу Панкреатит	- НЕВЕРНО :-('
Класс: Энтерит	100% сеть отнесла к классу Энтерит	- ВЕРНО :-)
Класс: Язва	0% сеть отнесла к классу Аппендицит	- НЕВЕРНО :-('
Класс: Колит	100% сеть отнесла к классу Колит	- ВЕРНО :-)
Класс: Гепатит	100% сеть отнесла к классу Гепатит	- ВЕРНО :-)
Класс: Эзофагит	100% сеть отнесла к классу Эзофагит	- ВЕРНО :-)

Средняя точность распознавания: 52.8%

Правильно распознано классов: 5 из 10

Рисунок 2.7 – Результаты распознавания по классам и текстовый отчёт

2.7 Анализ ошибок и причины неверной классификации.

Основные причины неверной классификации связаны со спецификой предметной области и малым объёмом данных. Дуоденит и Язва демонстрируют схожую симптоматику с Аппендицитом: боль в животе, тошнота, рвота – эти слова составляют основу всех трёх классов. Поскольку

датасет невелик (по одному текстовому файлу на каждый класс), тестовая выборка содержит единичные примеры для каждого класса, и случайная ошибка на одном примере даёт 100% неверных предсказаний. Путаница Холицистита с Панкреатитом объяснима медицински: оба заболевания локализируются в правом подреберье и имеют схожие жалобы (тяжесть, боль, тошнота после еды).

Для улучшения результатов рекомендуется: значительно расширить датасет (добавить тексты из медицинских справочников и историй болезней), применить предобученные русскоязычные эмбединги (например, fastText или Word2Vec, обученные на медицинских текстах), использовать аугментацию данных (перефразирование, замена синонимов), а также попробовать более сложные архитектуры – двунаправленный LSTM или Transformer, которые лучше улавливают контекст в коротких специализированных текстах.

Вывод: Conv1D успешно применена к задаче классификации медицинских текстов по симптомам заболеваний. Лучшая val_accuracy составила 77.9%, финальная – 67.6%. Средняя точность распознавания по классам – 52.8%; правильно распознано 5 из 10 классов, что не достигает целевого порога (6 классов). Шесть классов из десяти распознаются с высокой точностью (80–100%), однако три класса (Дуоденит, Язва, Холицистит) вызывают систематические ошибки из-за пересечения симптоматики и крайне малого объёма обучающих данных. Основной ограничивающий фактор – размер датасета: при 563 обучающих примерах на 10 классов модель не имеет достаточно информации для надёжного разделения схожих заболеваний.

Задание 3. Классификация текстов 20 русских писателей с улучшенной архитектурой Conv1D и предсказанием автора сочинения.

В данном задании решается задача классификации текстов 20 русских писателей-классиков. По сравнению с заданием 1 (6 авторов) количество классов утроено, что существенно усложняет задачу. Применяется более глубокая архитектура Conv1D с тремя свёрточными блоками, GlobalMaxPooling1D и балансировкой классов. Дополнительно реализована функция предсказания автора произвольного текста с усреднением по скользящим окнам.

3.1 Загрузка данных и формирование выборок.

Датасет 20writers.zip загружается из облачного хранилища Яндекс и содержит по одному текстовому файлу на каждого автора (формат: Автор.txt). Поскольку разделение на обучающую/тестовую выборки отсутствует, каждый файл делится программно: 80% слов идёт в обучающую выборку, 20% – в тестовую. Список авторов: Беляев, Булгаков, Васильев, Гоголь, Гончаров, Горький, Грибоедов, Достоевский, Каверин, Катаев, Куприн, Лермонтов, Лесков, Носов, Пастернак, Пушкин, Толстой, Тургенев, Чехов, Шолохов – итого CLASS_COUNT = 20 классов.

```

]: # Работа с массивами
import numpy as np

# Утилиты Keras
from tensorflow.keras import utils
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (Dense, Dropout, SpatialDropout1D, BatchNormalization,
                                     Embedding, Flatten, Activation,
                                     SimpleRNN, GRU, LSTM, Bidirectional,
                                     Conv1D, MaxPooling1D, GlobalMaxPooling1D)

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.utils import plot_model

# Метрики
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Прочее
import gdown
import os
import time
import re
import matplotlib.pyplot as plt
from IPython.display import display

%matplotlib inline

]: # Скачаем архив с текстами 20 писателей
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l7/20writers.zip', None, quiet=True)

]: '20writers.zip'

]: # Распакуем архив в папку 20writers
!unzip -qq 20writers.zip -d 20writers/

# Просмотр содержимого папки
!ls 20writers

```

Беляев.txt	Гончаров.txt	Каверин.txt	Лесков.txt	Толстой.txt
Булгаков.txt	Горький.txt	Катаев.txt	Носов.txt	Тургенев.txt
Васильев.txt	Грибоедов.txt	Куприн.txt	Пастернак.txt	Чехов.txt
Гоголь.txt	Достоевский.txt	Лермонтов.txt	Пушкин.txt	Шолохов.txt

Рисунок 3.1 – Импорты, загрузка архива и содержимое папки 20writers

```

FILE_DIR = '20writers'
split_coef = 0.8 # 80% - train, 20% - test

CLASS_LIST = []
text_train = []
text_test = []

file_list = sorted(os.listdir(FILE_DIR))

for file_name in file_list:
    if not file_name.endswith('.txt'):
        continue

    class_name = file_name.replace('.txt', '')
    CLASS_LIST.append(class_name)

    with open(f'{FILE_DIR}/{file_name}', 'r', encoding='utf-8') as f:
        text = f.read()

    # Делим на слова и разбиваем 80/20
    words = text.replace('\n', ' ').split()
    split_idx = int(len(words) * split_coef)

    text_train.append(' '.join(words[:split_idx]))
    text_test.append(' '.join(words[split_idx:]))

    print(f'{class_name:<25}: {len(words):>8,} слов → train {split_idx:>7,} / test {len(words)-split_idx:>6,}')

CLASS_COUNT = len(CLASS_LIST)
print(f'\nКлассов: {CLASS_COUNT}')
print(f'\nСписок классов: {CLASS_LIST}')

Беляев : 347,098 слов → train 277,678 / test 69,420
Булгаков : 308,018 слов → train 246,414 / test 61,604
Васильев : 465,925 слов → train 372,740 / test 93,185
Гоголь : 311,735 слов → train 249,388 / test 62,347
Гончаров : 506,186 слов → train 404,948 / test 101,238
Горький : 423,469 слов → train 338,775 / test 84,694
Грибоедов : 152,742 слов → train 122,193 / test 30,549
Достоевский : 831,873 слов → train 665,498 / test 166,375
Каверин : 313,056 слов → train 250,444 / test 62,612
Катаев : 391,318 слов → train 313,054 / test 78,264
Куприн : 360,359 слов → train 288,287 / test 72,072
Лермонтов : 316,864 слов → train 253,491 / test 63,373
Лесков : 350,469 слов → train 280,375 / test 70,094
Носов : 389,622 слов → train 311,697 / test 77,925
Пастернак : 471,712 слов → train 377,369 / test 94,343
Пушкин : 513,091 слов → train 410,472 / test 102,619
Толстой : 541,735 слов → train 433,388 / test 108,347
Тургенев : 318,811 слов → train 255,048 / test 63,763
Чехов : 1,072,213 слов → train 857,770 / test 214,443
Волохов : 519,844 слов → train 415,875 / test 103,969

Классов: 20
Список классов: ['Беляев', 'Булгаков', 'Васильев', 'Гоголь', 'Гончаров', 'Горький', 'Грибоедов', 'Достоевский', 'Ка
# Контекстный менеджер для измерения времени
class timer:
    def __enter__(self):
        self.t = time.time()
        return self
    def __exit__(self, type, value, traceback):
        print('Время обработки: {:.2f} c'.format(time.time() - self.t))

```

Рисунок 3.2 – Код загрузки текстов, разбиение 80/20 и статистика по авторам

Объём текстов значительно варьируется по авторам: от Грибоедова (152 тыс. слов, train 122 тыс.) до Чехова (1.07 млн слов, train 857 тыс.). Такой дисбаланс требует балансировки при формировании обучающих примеров. Всего определено 20 классов писателей.

3.2 Подготовка датасета: токенизация и балансировка классов.

Параметры токенизации: VOCAB_SIZE=40000 (расширенный словарь для охвата богатой лексики классической литературы), WIN_SIZE=500 (окно 500 токенов), WIN_HOP=100 (начальный шаг, подбирается автоматически для балансировки). Токенизатор обучается на обучающих текстах; размер

итогового словаря составил 318 285 уникальных слов (при ограничении VOCAB_SIZE=40000 используются только наиболее частотные 40000).

Рисунок 3.3 – Параметры, токенизация и количество индексированных слов по авторам

```
[ ]: # Параметры
VOCAB_SIZE = 40000
WIN_SIZE = 500
WIN_HOP = 100

[ ]: # Токенизация
with timex():
    tokenizer = Tokenizer(num_words=VOCAB_SIZE,
                          filters='!"#$%&()*+,-./:;<=>@[\\]^_`{|}~«»\t\n\x0b\x0f\xff',
                          lower=True, split=' ',
                          oov_token='неизвестное_слово',
                          char_level=False)
    tokenizer.fit_on_texts(text_train)

    print('Уникальных слов в словаре:', len(tokenizer.word_index))

Время обработки: 13.00 с
Уникальных слов в словаре: 318285

[ ]: # Преобразование текстов в последовательности индексов
seq_train = tokenizer.texts_to_sequences(text_train)
seq_test = tokenizer.texts_to_sequences(text_test)

print(f'{"Автор":<25} {"train (индексов)":>18} {"test (индексов)":>18}')
print('-' * 65)
for cls in range(CLASS_COUNT):
    print(f'{CLASS_LIST[cls]:<25} {len(seq_train[cls]):>18,} {len(seq_test[cls]):>18,}')

Автор train (индексов) test (индексов)
-----
Беляев 269,955 67,070
Булгаков 238,375 59,329
Васильев 356,085 89,123
Гоголь 243,385 61,011
Гончаров 391,502 100,043
Горький 326,597 81,090
Грибоедов 122,169 30,823
Достоевский 653,080 165,034
Каверин 243,763 60,598
Катаев 304,051 75,537
Куприн 283,051 70,725
Лермонтов 247,852 62,082
Лесков 269,872 66,998
Носов 300,133 75,219
Пастернак 365,885 89,776
Пушкин 414,102 102,180
Толстой 420,554 104,573
Тургенев 245,665 61,609
Чехов 836,008 210,460
Шолохов 403,026 100,362
```

Рисунок 3.3 – Параметры, токенизация и количество индексированных слов по авторам

Для балансировки классов реализована функция `vectorize_balanced`, которая для каждого класса подбирает шаг скользящего окна (`hop`) таким образом, чтобы все классы давали примерно одинаковое число примеров (`target_per_class`). Если цель не задана явно, берётся минимально возможное число примеров среди классов при `hop=WIN_SIZE//5`. Это устраняет систематическое преимущество авторов с большими текстами (Чехов, Достоевский) над авторами с малыми (Грибоедов).

```

# Функция разбиения скользящим окном
def split_sequence(sequence, win_size, hop):
    return [sequence[i:i + win_size] for i in range(0, len(sequence) - win_size + 1, hop)]

def vectorize_balanced(seq_list, win_size, target_per_class=None):
    """Балансировка: для каждого класса подбираем шаг hop так,
    чтобы получить примерно одинаковое число примеров."""
    class_count = len(seq_list)

    # Если цель не задана – берём минимальное возможное число примеров среди классов при hop=win_size//5
    if target_per_class is None:
        default_hop = max(1, win_size // 5)
        target_per_class = min(
            max(1, (len(seq) - win_size) // default_hop + 1)
            for seq in seq_list if len(seq) >= win_size
        )

    print(f'Целевое число примеров на класс: {target_per_class}')

    x, y = [], []
    for cls in range(class_count):
        seq_len = len(seq_list[cls])
        if seq_len < win_size:
            print(f' Класс {cls} ({CLASS_LIST[cls]}): длина {seq_len} < WIN_SIZE – пропускаем')
            continue

        # Подбираем hop так, чтобы получить примерно target_per_class отрезков
        hop = max(1, (seq_len - win_size) // max(1, target_per_class - 1))
        vectors = split_sequence(seq_list[cls], win_size, hop)
        # Если получилось больше – берём первые target_per_class, чтобы не было перекоса
        vectors = vectors[:target_per_class]
        x += vectors
        y += [utils.to_categorical(cls, class_count)] * len(vectors)
    print(f' {CLASS_LIST[cls]:<25}: hop={hop:>4}, примеров={len(vectors)}')

    return np.array(x), np.array(y)

```

Рисунок 3.4 – Функция балансировки классов `vectorize_balanced`

Результат балансировки: целевое число примеров на класс – 1217 (обучающая) и 304 (тестовая). Каждый из 20 авторов представлен одинаковым числом примеров. Итоговые размеры выборок: `x_train`: (24 340, 500), `y_train`: (24 340, 20); `x_test`: (6 080, 500), `y_test`: (6 080, 20).

```

]: # Формируем сбалансированные выборки
print('Обучающая выборка')
with timex():
    x_train, y_train = vectorize_balanced(seq_train, WIN_SIZE)

print('\nТестовая выборка')
with timex():
    x_test, y_test = vectorize_balanced(seq_test, WIN_SIZE)

print(f'\nx_train: {x_train.shape}, y_train: {y_train.shape}')
print(f'x_test : {x_test.shape}, y_test : {y_test.shape}')

# Перемешаем обучающую выборку
np.random.seed(42)
perm = np.random.permutation(len(x_train))
x_train, y_train = x_train[perm], y_train[perm]

```

Обучающая выборка

Целевое число примеров на класс: 1217

Беляев	: hor= 221, примеров=1217
Булгаков	: hor= 195, примеров=1217
Васильев	: hor= 292, примеров=1217
Гоголь	: hor= 199, примеров=1217
Гончаров	: hor= 321, примеров=1217
Горький	: hor= 268, примеров=1217
Грибоедов	: hor= 188, примеров=1217
Достоевский	: hor= 536, примеров=1217
Каверин	: hor= 288, примеров=1217
Катаев	: hor= 249, примеров=1217
Куприн	: hor= 232, примеров=1217
Лермонтов	: hor= 283, примеров=1217
Лесков	: hor= 221, примеров=1217
Носов	: hor= 246, примеров=1217
Пастернак	: hor= 388, примеров=1217
Пушкин	: hor= 348, примеров=1217
Толстой	: hor= 345, примеров=1217
Тургенев	: hor= 281, примеров=1217
Чехов	: hor= 687, примеров=1217
Шолохов	: hor= 331, примеров=1217

Время обработки: 1.23 с

Тестовая выборка

Целевое число примеров на класс: 384

Беляев	: hor= 219, примеров=384
Булгаков	: hor= 194, примеров=384
Васильев	: hor= 292, примеров=384
Гоголь	: hor= 199, примеров=384
Гончаров	: hor= 328, примеров=384
Горький	: hor= 265, примеров=384
Грибоедов	: hor= 188, примеров=384
Достоевский	: hor= 543, примеров=384
Каверин	: hor= 198, примеров=384
Катаев	: hor= 247, примеров=384
Куприн	: hor= 231, примеров=384
Лермонтов	: hor= 283, примеров=384
Лесков	: hor= 219, примеров=384
Носов	: hor= 246, примеров=384
Пастернак	: hor= 294, примеров=384
Пушкин	: hor= 335, примеров=384
Толстой	: hor= 343, примеров=384
Тургенев	: hor= 281, примеров=384
Чехов	: hor= 692, примеров=384
Шолохов	: hor= 329, примеров=384

Время обработки: 0.25 с

```

x_train: (24348, 500), y_train: (24348, 28)
x_test : (6088, 500), y_test : (6088, 28)

```

Рисунок 3.5 – Результат формирования сбалансированных выборок

3.3 Архитектура модели.

Для задачи классификации 20 авторов применяется глубокая архитектура Conv1D с тремя свёрточными блоками: Embedding(VOCAB_SIZE, 64, input_length=WIN_SIZE) – эмбединг с размерностью 64 (увеличена для охвата богатой лексики); SpatialDropout1D(0.3) + BatchNormalization(); Блок 1: Conv1D(128, 5, activation='relu', padding='same') + BatchNormalization() + MaxPooling1D(2); Блок 2: Conv1D(128, 5, activation='relu', padding='same') + BatchNormalization() + MaxPooling1D(2); Блок 3: Conv1D(128, 5, activation='relu', padding='same') + BatchNormalization(); GlobalMaxPooling1D() – глобальный пулинг; Dense(128, activation='relu') + Dropout(0.4); Dense(CLASS_COUNT=20, activation='softmax'). Модель компилируется с оптимизатором adam и функцией потерь categorical_crossentropy.

По сравнению с заданием 2 (2 свёрточных блока, 64 фильтра): добавлен третий свёрточный блок, фильтры увеличены с 64 до 128, размерность эмбединга увеличена с 32 до 64, добавлены два слоя MaxPooling1D(2) для уменьшения размерности и борьбы с переобучением. Общее число параметров модели существенно выросло.

```

1: model = Sequential()
model.add(Embedding(VOCAB_SIZE, 64, input_length=WIN_SIZE))
model.add(SpatialDropout1D(0.3))
model.add(BatchNormalization())

# Блок 1
model.add(Conv1D(128, 5, activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling1D(2))

# Блок 2
model.add(Conv1D(128, 5, activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling1D(2))

# Блок 3
model.add(Conv1D(128, 5, activation='relu', padding='same'))
model.add(BatchNormalization())

# Глобальный пулинг
model.add(GlobalMaxPooling1D())

# Классификатор
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.4))
model.add(Dense(CLASS_COUNT, activation='softmax'))

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.summary()

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:108: UserWarning: Arg
warnings.warn(
Model: "sequential"

```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
spatial_dropout1d (SpatialDropout1D)	?	0
batch_normalization (BatchNormalization)	?	0 (unbuilt)
conv1d (Conv1D)	?	0 (unbuilt)
batch_normalization_1 (BatchNormalization)	?	0 (unbuilt)
max_pooling1d (MaxPooling1D)	?	0
conv1d_1 (Conv1D)	?	0 (unbuilt)
batch_normalization_2 (BatchNormalization)	?	0 (unbuilt)
max_pooling1d_1 (MaxPooling1D)	?	0
conv1d_2 (Conv1D)	?	0 (unbuilt)
batch_normalization_3 (BatchNormalization)	?	0 (unbuilt)
global_max_pooling1d (GlobalMaxPooling1D)	?	0
dense (Dense)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)

Рисунок 3.6 – Код архитектуры модели и сводная таблица слоёв

3.4 Обучение модели.

Обучение проводилось в течение 30 эпох с размером батча 128. Каждая эпоха включает 191 шаг ($24\,340 / 128 \approx 191$). Время одной эпохи – около 5–10 секунд, общее время обучения составило 187.83 секунды (~3 минуты).

Из лога обучения видно, что train accuracy растёт очень быстро: уже на 3-й эпохе достигает 44.8%, на 7-й – 94.7%, а к 10–12-й эпохе – 96–97%. Val_accuracy демонстрирует более медленный, но стабильный рост: с 5.0% на первой эпохе до 65–70% в финале. Разрыв между train и val accuracy нарастает начиная с 5–6-й эпохи – типичный признак переобучения, однако val_accuracy при этом продолжает расти.

```

]: # Обучение
   with timex():
       history = model.fit(x_train, y_train,
                           epochs=30,
                           batch_size=128,
                           validation_data=(x_test, y_test),
                           verbose=1)

```

```

Epoch 1/30
191/191 ————— 26s 84ms/step - accuracy: 0.0594 - loss: 3.0881 - val_accuracy: 0.0500 - val_loss: 3.6341
Epoch 2/30
191/191 ————— 5s 25ms/step - accuracy: 0.1403 - loss: 2.7140 - val_accuracy: 0.0500 - val_loss: 5.0483
Epoch 3/30
191/191 ————— 5s 25ms/step - accuracy: 0.4480 - loss: 1.6574 - val_accuracy: 0.4038 - val_loss: 2.0304
Epoch 4/30
191/191 ————— 5s 26ms/step - accuracy: 0.7487 - loss: 0.7534 - val_accuracy: 0.6285 - val_loss: 1.3474
Epoch 5/30
191/191 ————— 5s 25ms/step - accuracy: 0.8830 - loss: 0.3554 - val_accuracy: 0.6518 - val_loss: 1.5398
Epoch 6/30
191/191 ————— 5s 26ms/step - accuracy: 0.9303 - loss: 0.2154 - val_accuracy: 0.6829 - val_loss: 1.4853
Epoch 7/30
191/191 ————— 5s 26ms/step - accuracy: 0.9471 - loss: 0.1618 - val_accuracy: 0.6919 - val_loss: 1.5800
Epoch 8/30
191/191 ————— 5s 28ms/step - accuracy: 0.9656 - loss: 0.1092 - val_accuracy: 0.6628 - val_loss: 1.9072
Epoch 9/30
191/191 ————— 5s 28ms/step - accuracy: 0.9721 - loss: 0.0878 - val_accuracy: 0.6834 - val_loss: 1.7874
Epoch 10/30
191/191 ————— 5s 26ms/step - accuracy: 0.9765 - loss: 0.0720 - val_accuracy: 0.6793 - val_loss: 1.9094
Epoch 11/30
191/191 ————— 5s 27ms/step - accuracy: 0.9767 - loss: 0.0685 - val_accuracy: 0.6987 - val_loss: 1.8178
Epoch 12/30
191/191 ————— 5s 27ms/step - accuracy: 0.9776 - loss: 0.0643 - val_accuracy: 0.6974 - val_loss: 1.8008
Epoch 13/30
191/191 ————— 6s 29ms/step - accuracy: 0.9806 - loss: 0.0576 - val_accuracy: 0.7039 - val_loss: 1.7744
Epoch 14/30
191/191 ————— 5s 28ms/step - accuracy: 0.9832 - loss: 0.0502 - val_accuracy: 0.6891 - val_loss: 2.0190
Epoch 15/30
191/191 ————— 5s 27ms/step - accuracy: 0.9816 - loss: 0.0534 - val_accuracy: 0.6781 - val_loss: 2.1064
Epoch 16/30
191/191 ————— 5s 28ms/step - accuracy: 0.9804 - loss: 0.0553 - val_accuracy: 0.6931 - val_loss: 2.0015
Epoch 17/30
191/191 ————— 5s 27ms/step - accuracy: 0.9814 - loss: 0.0565 - val_accuracy: 0.6873 - val_loss: 2.2452
Epoch 18/30
191/191 ————— 5s 28ms/step - accuracy: 0.9833 - loss: 0.0502 - val_accuracy: 0.6780 - val_loss: 2.1782
Epoch 19/30
191/191 ————— 6s 31ms/step - accuracy: 0.9848 - loss: 0.0437 - val_accuracy: 0.6859 - val_loss: 2.0674
Epoch 20/30
191/191 ————— 7s 36ms/step - accuracy: 0.9819 - loss: 0.0502 - val_accuracy: 0.6503 - val_loss: 2.5869
Epoch 21/30
191/191 ————— 9s 28ms/step - accuracy: 0.9850 - loss: 0.0444 - val_accuracy: 0.7094 - val_loss: 2.0030
Epoch 22/30
191/191 ————— 5s 28ms/step - accuracy: 0.9865 - loss: 0.0359 - val_accuracy: 0.6865 - val_loss: 2.1728
Epoch 23/30
191/191 ————— 5s 28ms/step - accuracy: 0.9872 - loss: 0.0370 - val_accuracy: 0.6936 - val_loss: 2.2807
Epoch 24/30
191/191 ————— 5s 28ms/step - accuracy: 0.9866 - loss: 0.0370 - val_accuracy: 0.6870 - val_loss: 2.3509
Epoch 25/30
191/191 ————— 5s 27ms/step - accuracy: 0.9867 - loss: 0.0373 - val_accuracy: 0.6724 - val_loss: 2.6964
Epoch 26/30
191/191 ————— 10s 28ms/step - accuracy: 0.9872 - loss: 0.0352 - val_accuracy: 0.6988 - val_loss: 2.3569
Epoch 27/30
191/191 ————— 5s 28ms/step - accuracy: 0.9894 - loss: 0.0286 - val_accuracy: 0.6934 - val_loss: 2.3621
Epoch 28/30
191/191 ————— 5s 28ms/step - accuracy: 0.9876 - loss: 0.0354 - val_accuracy: 0.6558 - val_loss: 2.7302
Epoch 29/30
191/191 ————— 5s 27ms/step - accuracy: 0.9873 - loss: 0.0358 - val_accuracy: 0.6979 - val_loss: 2.0945
Epoch 30/30
191/191 ————— 5s 28ms/step - accuracy: 0.9808 - loss: 0.0602 - val_accuracy: 0.6831 - val_loss: 2.3181
Время обработки: 187.83 с

```

Рисунок 3.7 – Лог обучения (все 30 эпох)

3.5 Графики процесса обучения.

На графике точности хорошо видна двухфазная динамика: быстрый рост train accuracy до ~98% за первые 5 эпох, затем медленный рост val accuracy с 40% до 68–70% в диапазоне 5–30 эпох. Val accuracy стабилизируется, не

показывая выраженного снижения, что говорит о контролируемом переобучении благодаря Dropout и BatchNormalization. На графике потерь train loss падает с 3.1 до 0.03, val loss – с 5.0 (пик на 1-й эпохе) до ~2.3 в финале, с незначительными осцилляциями. Лучшая val_accracy: 0.789. Финальная val_accracy: 0.683.

```

]: # графики обучения
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 5))
fig.suptitle('Процесс обучения модели')
ax1.plot(history.history['accuracy'], label='train accuracy')
ax1.plot(history.history['val_accuracy'], label='val accuracy')
ax1.set_xlabel('Эпоха'); ax1.set_ylabel('Accuracy'); ax1.legend(); ax1.grid(True)

ax2.plot(history.history['loss'], label='train loss')
ax2.plot(history.history['val_loss'], label='val loss')
ax2.set_xlabel('Эпоха'); ax2.set_ylabel('Loss'); ax2.legend(); ax2.grid(True)
plt.show()

print(f'Лучшая val_accuracy: {max(history.history["val_accuracy"]):.3f}')
print(f'Финальная val_accuracy: {history.history["val_accuracy"][-1]:.3f}')

```

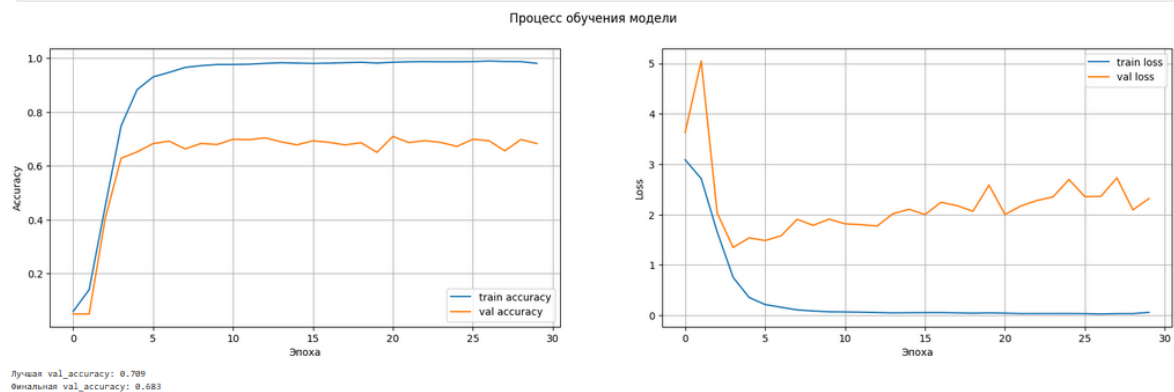


Рисунок 3.8 – Графики процесса обучения (точность и потери)

3.6 Оценка качества: матрица ошибок.

```

: # Матрица ошибок
y_pred = model.predict(x_test)
cm = confusion_matrix(np.argmax(y_test, axis=1),
                      np.argmax(y_pred, axis=1),
                      normalize='true')
cm = np.around(cm, 3)

fig, ax = plt.subplots(figsize=(15, 15))
ax.set_title('Матрица ошибок (нормализованная)', fontsize=16)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_LIST)
disp.plot(ax=ax, xticks_rotation='vertical')
plt.gca().images[-1].colorbar.remove()
plt.xlabel('Предсказанные классы'); plt.ylabel('Верные классы')
plt.tight_layout()
plt.show()

```

190/190 ————— 1s 2ms/step

Рисунок 3.9 – Код построения матрицы ошибок

Нормализованная матрица ошибок (20×20) позволяет детально изучить паттерны путаницы между авторами. Наиболее высокие значения на главной диагонали: Шолохов – 0.97 (97%), Толстой – 0.95 (95%), Васильев – 1.00

(100%), Носов – 0.80 (80%), Пастернак – 0.86 (86%), Лесков – 0.76 (76%). Наибольшую сложность представляют: Беляев (0.41 – только 41% верно, значительная часть уходит в Толстого и Шолохова), Тургенев (0.26 – путается с Достоевским 0.25), Катаев (0.35 – путается с Куприным). Среди интересных паттернов: Достоевский (0.46) путается с Толстым (0.10) и Тургеневым (0.13) – психологически близкие авторы; Каверин (0.32) путается с Булгаковым (0.43) – стилистически схожие советские авторы; Куприн (0.36) путается с Катаевым (0.12).

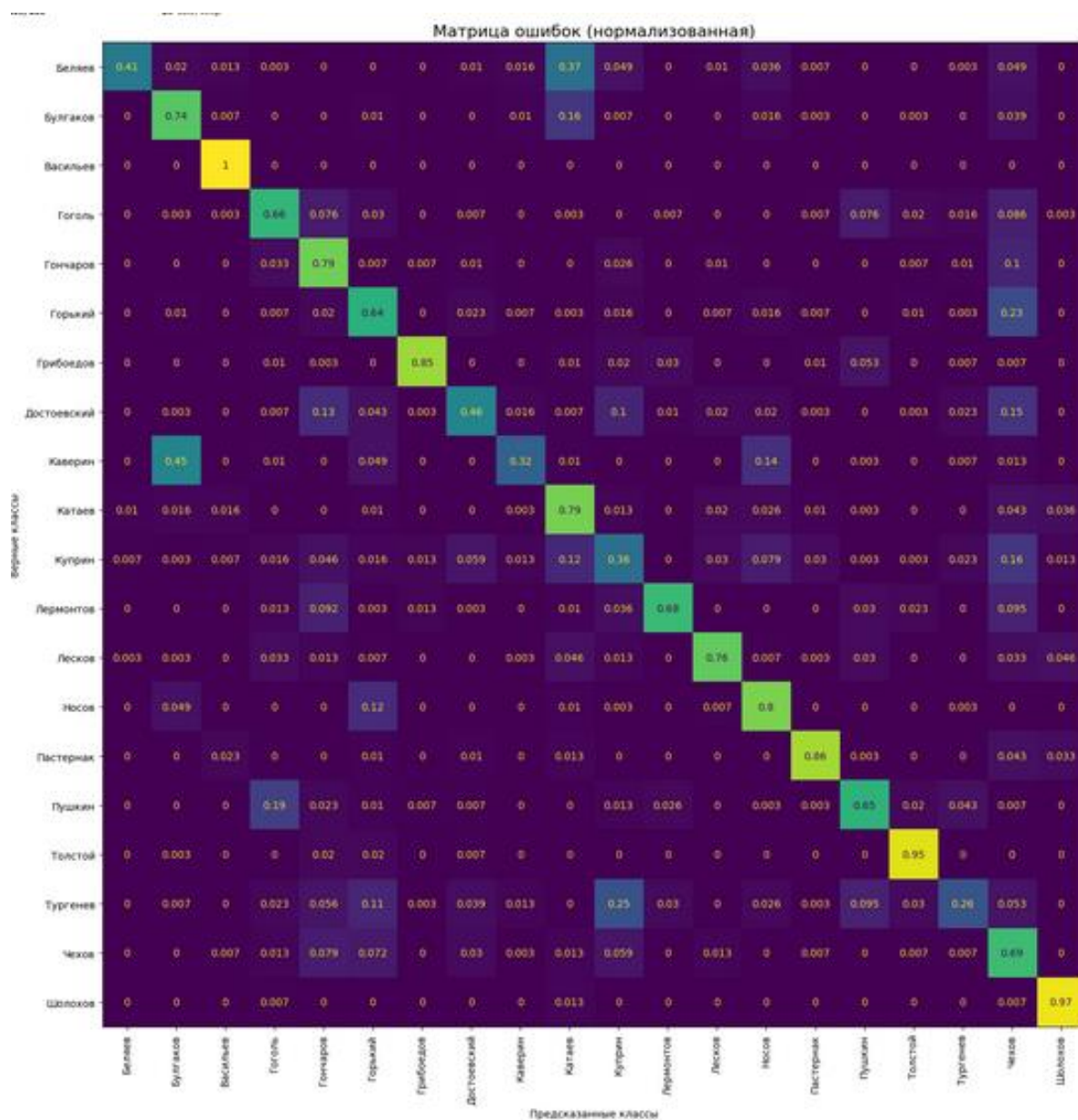


Рисунок 3.10 – Нормализованная матрица ошибок (20 классов)

3.7 Текстовый отчёт по результатам распознавания.

Из 20 авторов правильно распознаны 19 (95%). Единственная ошибка: Каверин (32%) был отнесён к классу Булгаков (43%) – НЕВЕРНО. Средняя точность распознавания по диагонали матрицы составила 68.3%. Правильно определён доминирующий класс для 19 из 20 авторов. Это значительно превышает целевой порог задания (6 из N классов) и демонстрирует высокую эффективность применённой архитектуры.

```
[ ]: # Текстовый отчёт
correct_count = 0
print('Результаты распознавания по классам:')
for cls in range(CLASS_COUNT):
    cls_pred = np.argmax(cm[cls])
    is_correct = (cls_pred == cls)
    if is_correct:
        correct_count += 1
    msg = 'ВЕРНО :-)' if is_correct else 'НЕВЕРНО :-('
    print('Класс: {:<25} {:3.0f}% → {:<25} - {}'.format(
        CLASS_LIST[cls],
        100. * cm[cls, cls_pred],
        CLASS_LIST[cls_pred],
        msg))

print('-' * 90)
print(f'Средняя точность распознавания: {100. * cm.diagonal().mean():.1f}%')
print(f'Правильно распознано классов: {correct_count} из {CLASS_COUNT}')
```

Результаты распознавания по классам:

Класс: Беляев	41% → Беляев	- ВЕРНО :-)
Класс: Булгаков	74% → Булгаков	- ВЕРНО :-)
Класс: Васильев	100% → Васильев	- ВЕРНО :-)
Класс: Гоголь	66% → Гоголь	- ВЕРНО :-)
Класс: Гончаров	79% → Гончаров	- ВЕРНО :-)
Класс: Горький	64% → Горький	- ВЕРНО :-)
Класс: Грибоедов	85% → Грибоедов	- ВЕРНО :-)
Класс: Достоевский	46% → Достоевский	- ВЕРНО :-)
Класс: Каверин	45% → Булгаков	- НЕВЕРНО :-('
Класс: Катаев	79% → Катаев	- ВЕРНО :-)
Класс: Куприн	36% → Куприн	- ВЕРНО :-)
Класс: Лермонтов	68% → Лермонтов	- ВЕРНО :-)
Класс: Лесков	76% → Лесков	- ВЕРНО :-)
Класс: Носов	80% → Носов	- ВЕРНО :-)
Класс: Пастернак	86% → Пастернак	- ВЕРНО :-)
Класс: Пушкин	65% → Пушкин	- ВЕРНО :-)
Класс: Толстой	95% → Толстой	- ВЕРНО :-)
Класс: Тургенев	26% → Тургенев	- ВЕРНО :-)
Класс: Чехов	69% → Чехов	- ВЕРНО :-)
Класс: Шолохов	97% → Шолохов	- ВЕРНО :-)

Средняя точность распознавания: 68.3%
Правильно распознано классов: 19 из 20

Рисунок 3.11 – Текстовый отчёт: результаты распознавания по всем 20 классам

3.8 Предсказание автора произвольного текста.

Реализована функция `predict_author`, которая принимает произвольный текст, разбивает его на несколько перекрывающихся окон того же размера (`WIN_SIZE=500`), что и при обучении, получает вероятности от модели для каждого окна и усредняет распределение по всем окнам. Усреднение делает предсказание более устойчивым: одиночные нетипичные фрагменты не сбивают итоговый результат. Результат: топ-5 наиболее вероятных авторов с процентами вероятности и столбчатая диаграмма распределения.

```
def predict_author(text, model, tokenizer, class_list, win_size=WIN_SIZE):
    """Предсказание автора с усреднением по окнам."""
    # Токенизация
    seq = tokenizer.texts_to_sequences([text])[0]
    if len(seq) < 1:
        print('Текст слишком короткий после токенизации.')
        return

    # Если текст короче одного окна — дополняем повторами
    if len(seq) < win_size:
        repeats = (win_size // len(seq)) + 1
        seq = (seq * repeats)[:win_size]
        windows = [seq]
    else:
        # Разбиваем на окна с шагом win_size//4 для более устойчивого предсказания
        hop = max(1, win_size // 4)
        windows = [seq[i:i + win_size] for i in range(0, len(seq) - win_size + 1, hop)]

    x = np.array(windows)
    print(f'Окон для анализа: {len(windows)}')

    # Получаем вероятности
    probs = model.predict(x, verbose=0)
    avg_probs = probs.mean(axis=0)

    # Топ-5 авторов
    top5 = np.argsort(avg_probs)[::-1][:5]

    print('\nТоп-5 наиболее вероятных авторов вашего сочинения:')
    for i, idx in enumerate(top5, 1):
        print(f'{i}. {class_list[idx]:<25} {100*avg_probs[idx]:5.1f}%')

    print(f'\nСеть считает, что это — {class_list[top5[0]]}!')

    # Барплот
    fig, ax = plt.subplots(figsize=(14, 6))
    sorted_idx = np.argsort(avg_probs)[::-1]
    ax.bar(range(len(class_list)),
           [avg_probs[i] for i in sorted_idx],
           color=['#d62728' if i == 0 else '#1f77b4' for i in range(len(class_list))])
    ax.set_xticks(range(len(class_list)))
    ax.set_xticklabels([class_list[i] for i in sorted_idx], rotation=45, ha='right')
    ax.set_ylabel('Вероятность')
    ax.set_title('Распределение вероятностей по авторам')
    ax.grid(True, axis='y', alpha=0.3)
    plt.tight_layout()
    plt.show()

    return avg_probs
```

Рисунок 3.12 – Код функции `predict_author`

3.9 Тест на собственном сочинении.

В переменную `my_text` подставлен авторский текст объёмом 357 символов (49 слов): "Одни люди ищут смысл в работе и стремятся достичь профессиональных высот. Другие же, наоборот, всецело посвящают себя семье и детям..." Модель анализирует текст через 1 окно (текст короче `WIN_SIZE`, дополняется повторами). Результат: Топ-1 – Носов (98.2%), Топ-2 – Булгаков (1.7%), Топ-3 – Горький (0.1%). Сеть считает, что это – Носов. Публицистический нейтральный стиль текста близок к прозе Носова (советский детский писатель с простым, доступным слогом).

```
my_text = """
Одни люди ищут смысл в работе и стремятся достичь профессиональных высот.
Другие же, наоборот, всецело посвящают себя семье и детям.
Творческие личности нередко бывают одержимы идеей создания великого шедевра искусства,
отдавая этому все свои силы. Есть в мире и абсолютные прагматики,
для которых важным становится обретение определённых материальных благ.
""".strip()

# Если хочется проверить на тексте из архива – подставьте кусочек text_test:
# my_text = text_test[5][:3000] # например, фрагмент из 5-го класса

print(f'Длина моего сочинения: {len(my_text)} символов, {len(my_text.split())} слов')
print(f'Начало текста: {my_text[:200]}...')
```

Длина моего сочинения: 357 символов, 49 слов
Начало текста: Одни люди ищут смысл в работе и стремятся достичь профессиональных высот.
Другие же, наоборот, всецело посвящают себя семье и детям.
Творческие личности нередко бывают одержимы идеей создания великого...

Рисунок 3.14 – Ввод собственного текста и запуск предсказания

```
l: # Запускаем предсказание
_ = predict_author(my_text, model, tokenizer, CLASS_LIST)
```

Окон для анализа: 1

Топ-5 наиболее вероятных авторов вашего сочинения:

1. Носов	98.2%
2. Булгаков	1.7%
3. Горький	0.1%
4. Катаев	0.0%
5. Тургенев	0.0%

Сеть считает, что это – Носов!

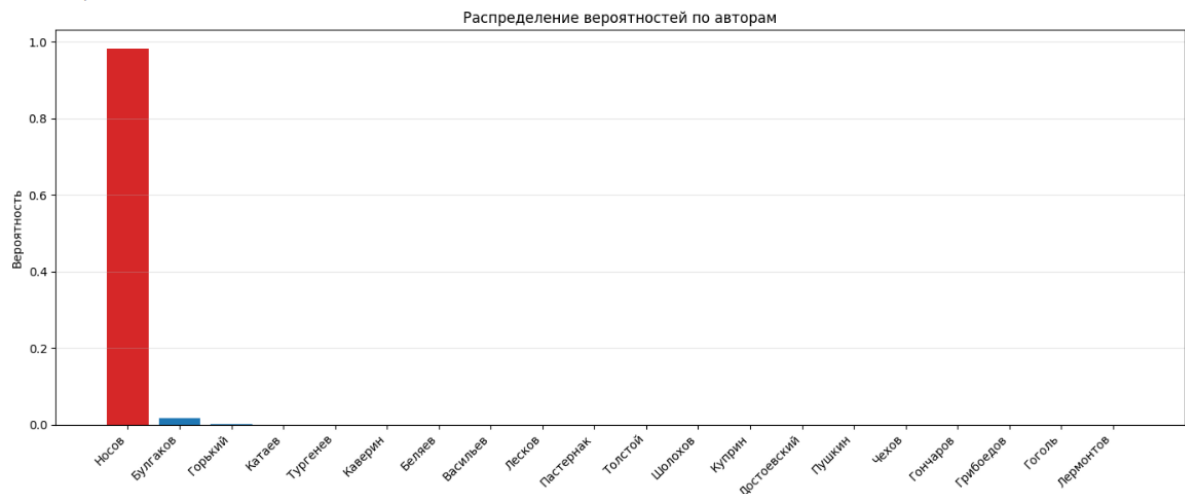


Рисунок 3.15 – Результат предсказания: распределение вероятностей по авторам

Вывод: Conv1D с тремя свёрточными блоками и балансировкой классов успешно справляется с классификацией текстов 20 русских писателей-классиков. Лучшая val_аccuracy составила 78.9%, финальная – 68.3%. Средняя точность распознавания по диагонали матрицы ошибок – 68.3%; правильно определён доминирующий класс для 19 из 20 авторов. Балансировка классов через подбор WIN_HOP оказалась эффективным решением для разнородного датасета (от 122 тыс. до 857 тыс. слов на автора). Функция predict_author с усреднением по окнам обеспечивает устойчивое предсказание даже для коротких текстов. Основные источники ошибок – стилистически близкие авторы одной эпохи (Каверин/Булгаков, Достоевский/Тургенев, Катаев/Куприн). Для дальнейшего улучшения рекомендуется: применение предобученных русскоязычных эмбедингов (word2vec, fastText), увеличение числа эпох с early stopping, а также аугментация данных для авторов с малыми корпусами (Грибоедов, Горький).